Preprints.org

Review

Not peer-reviewed version

Recursive Self-Improvement in AI: A Survey

Hao Wu , Fan Xu , Fan Zhang , Zhipeng Xu , Yongheng Zhang , Yinghui Li , Yizhou Zhao , Penghao Zhao , Weiran Yao , Zengxiang Li , Yuan Gao , Zhangyang Gao , Hanbin Wang , Ruihan Tao , Wenjie Wang , Qi Li , Junfeng Fang , Xingjun Ma , Qiankun Li , Xiao Luo , Yefeng Zheng , Zhiwei Liu , Stephen Wang , Kun Wang [†] , Yang Liu , Yugang Jiang , Liang Wang , Nan Liu , Philip S. Yu , Caiming Xiong , Qiang Yang , Qingsong Wen [*]

Posted Date: 30 September 2026

Keywords: Recursive self-improvement; artificial intelligence; survey

Preprints.org is a free multidisciplinary platform providing preprint service that is dedicated to making early versions of research outputs permanently available and citable. Preprints posted at Preprints.org appear in Web of Science, Crossref, Google Scholar, Scilit, Europe PMC, OpenAlex.

Copyright: This open access article is published under a Creative Commons CC BY 4.0 license, which permit the free download, distribution, and reuse, provided that the author and preprint are cited in any reuse.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Disclaimer/Publisher's Note: The statements, opinions, and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions, or products referred to in the content.

Review

Recursive Self-Improvement in AI: A Survey

Hao Wu [1], Fan Xu [2], Fan Zhang [3], Zhipeng Xu [1], Yongheng Zhang [1], Yinghui Li [1], Yizhou Zhao [4], Penghao Zhao [5], Weiran Yao [6], Zengxiang Li [7], Yuan Gao [1], Zhangyang Gao [8], Hanbin Wang [5], Ruihan Tao [9], Wenjie Wang [10], Qi Li [11], Junfeng Fang [12], Xingjun Ma [13], Qiankun Li [14], Xiao Luo [15], Yefeng Zheng [16], Zhiwei Liu [17], Stephen Wang [18], Kun Wang [14,†], Yang Liu [14], Yugang Jiang [13], Liang Wang [11], Nan Liu [7], Philip S. Yu [19], Caiming Xiong [17], Qiang Yang [20] and Qingsong Wen [21,*]

[1] Tsinghua University
[2] Shenzhen Loop Area Institute
[3] The Chinese University of Hong Kong
[4] Carnegie Mellon University
[5] Peking University
[6] actAVA.ai
[7] Duke-NUS Medical School, National University of Singapore
[8] Shanghai AI Laboratory
[9] University of California, Berkeley
[10] Huazhong University of Science and Technology
[11] Institute of Automation, Chinese Academy of Sciences
[12] National University of Singapore
[13] Fudan University
[14] Nanyang Technological University
[15] University of Wisconsin–Madison
[16] Westlake University
[17] Recursive
[18] Abel.ai
[19] University of Illinois Chicago
[20] The Hong Kong Polytechnic University
[21] Squirrel Ai Learning
* Correspondence: qingsongedu@gmail.com
† Project Leader.

Abstract

Recursive self-improvement (RSI) seeks to turn gains in an AI system into a stronger capacity to produce subsequent gains. Foundation models make parts of this process practical through generated training data, program revision and automated experimentation, but repeated optimization alone does not establish recursive improvement. This survey organizes the field around two overlapping goals: automated design of AI infrastructure, and repair and optimization of existing systems. We relate these goals to AutoML, meta-learning and program search, and formulate their objectives alongside criteria for task gains, retained capabilities and improved update procedures. A common account of update targets, feedback, inheritance and external control connects self-training, memory and skills, agent design, editable improvers and automated research. We synthesize methods by shared mechanisms and compare their evidence under independent testing, resource controls, transfer probes and multi-round evaluation. Existing studies show that systems can retain useful changes and, in limited settings, improve how they produce subsequent updates. They do not yet establish unrestricted or reliably accelerating self-improvement. We identify feedback quality, interacting updates and independent validation as central constraints, and develop testable directions for more reliable, transferable improvement loops.

Keywords: recursive self-improvement; artificial intelligence; survey

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

1. Introduction

Improving an AI system usually requires a division of labor: a model solves problems, while researchers design its data, training procedure and operating environment. Foundation models increasingly participate in both roles. They produce training examples, diagnose failures, construct tools and run experiments. Recursive self-improvement (RSI) asks whether the benefits of these activities can return to the process that produces the next improvement. A stronger solver is one possible outcome; a more effective way of building stronger solvers is the further object of interest.

The practical work falls into two overlapping goals. *Automated design* constructs AI infrastructure: data pipelines, learning algorithms, model architectures, agent workflows and their evaluation machinery. *Repair and optimization* methods modify existing systems by correcting bugs, recovering from failed interactions, maintaining memory and skills, or updating parameters and configurations. These goals have different success criteria. A design should generalize beyond the experiments that select it; a repair should resolve its target failure while preserving other useful behavior. Both can support RSI when retained changes improve later update production. Neither is recursive simply because it runs repeatedly.

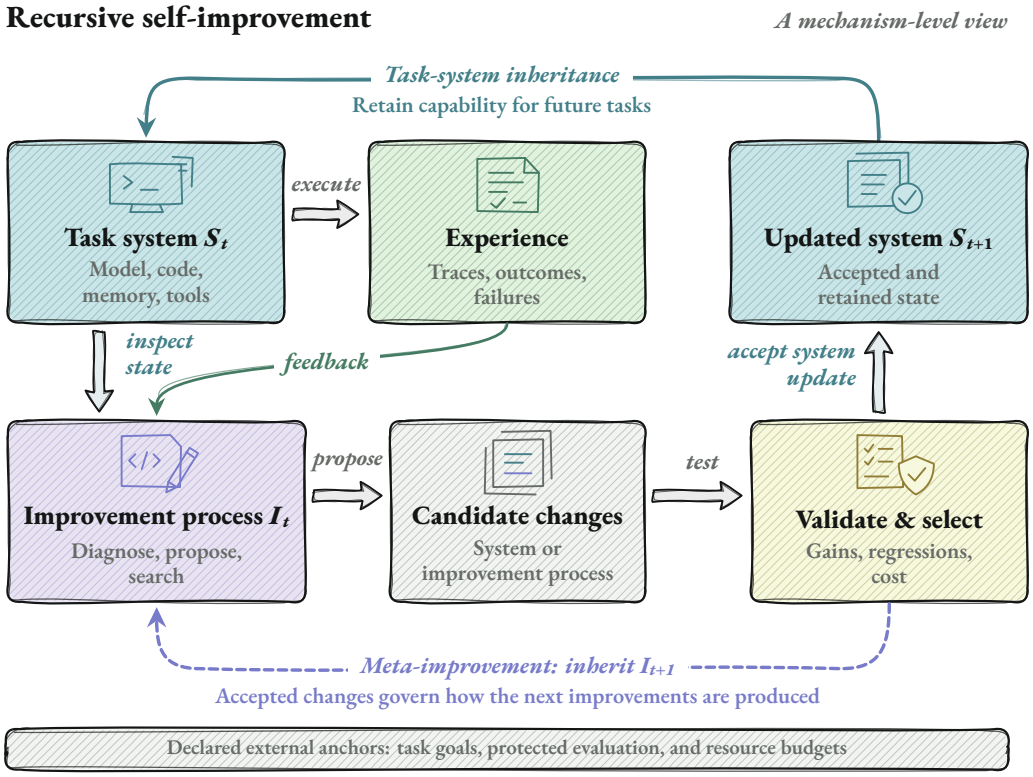


Figure 1. A mechanism-level view of recursive self-improvement. Experience informs candidate changes. Validation determines what is retained: the upper loop carries task-system state into later tasks; the dashed lower loop carries changes to the process that produces later improvements. External anchors specify the conditions outside the update scope. The diagram describes possible dependencies, rather than a guarantee of gains.

The literature connects these goals through several mechanisms. Generated examples and rewards support parameter updates; memories and skills retain experience; prompt, workflow and program search alter how agents act; and editable improvers make the update procedure itself a search object [1–6]. Recursive context provides another route: Meta^n keeps a meta-operation fixed while adding earlier programs and traces to the material it processes [7]. The resulting systems differ in what changes, what survives a reset, and what remains externally fixed. Figures 3 and 4 give an early view of these mechanisms and their historical research routes.

Three questions organize this survey. **What is being improved?** The target may be an answer, a persistent solver, or the process that generates updates. **What evidence supports the improvement?**

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Independent task gains, retention across rounds and transfer of improvement ability require different tests. **What keeps the loop productive?** Repeated updates can reinforce errors, lose earlier capabilities or exploit a proxy reward; execution checks, new data and human judgment can supply information the loop does not generate reliably on its own. The amount, timing and cost of that outside feedback belong in the experimental description.

These questions connect RSI to an established history of AutoML, meta-learning, program search and adaptive agents. Automated search already uses feedback, and learned optimizers already make learning rules trainable. The relevant distinction is the role of inherited changes in later improvement, rather than a categorical divide between older optimization and modern agents. Recent surveys offer complementary accounts of evolving agents, model-development lifecycles and coupled improvement components [8–11]. We build on them through three contributions:

1. A formulation that separates infrastructure design from repair and optimization, and connects both to persistent state and improver quality.
2. A synthesis by method clusters, with historical comparisons, diagrams and configuration-level tables that expose shared mechanisms and exceptions.
3. An evidence framework covering independent tests, full budgets, retention, improver ablations and external feedback, together with the limits of currently demonstrated results.

Figure 2 maps the article. Scope and foundations lead to a mechanism taxonomy; Sections 5 and 6 then develop the two engineering goals. Evaluation and open questions cut across both.

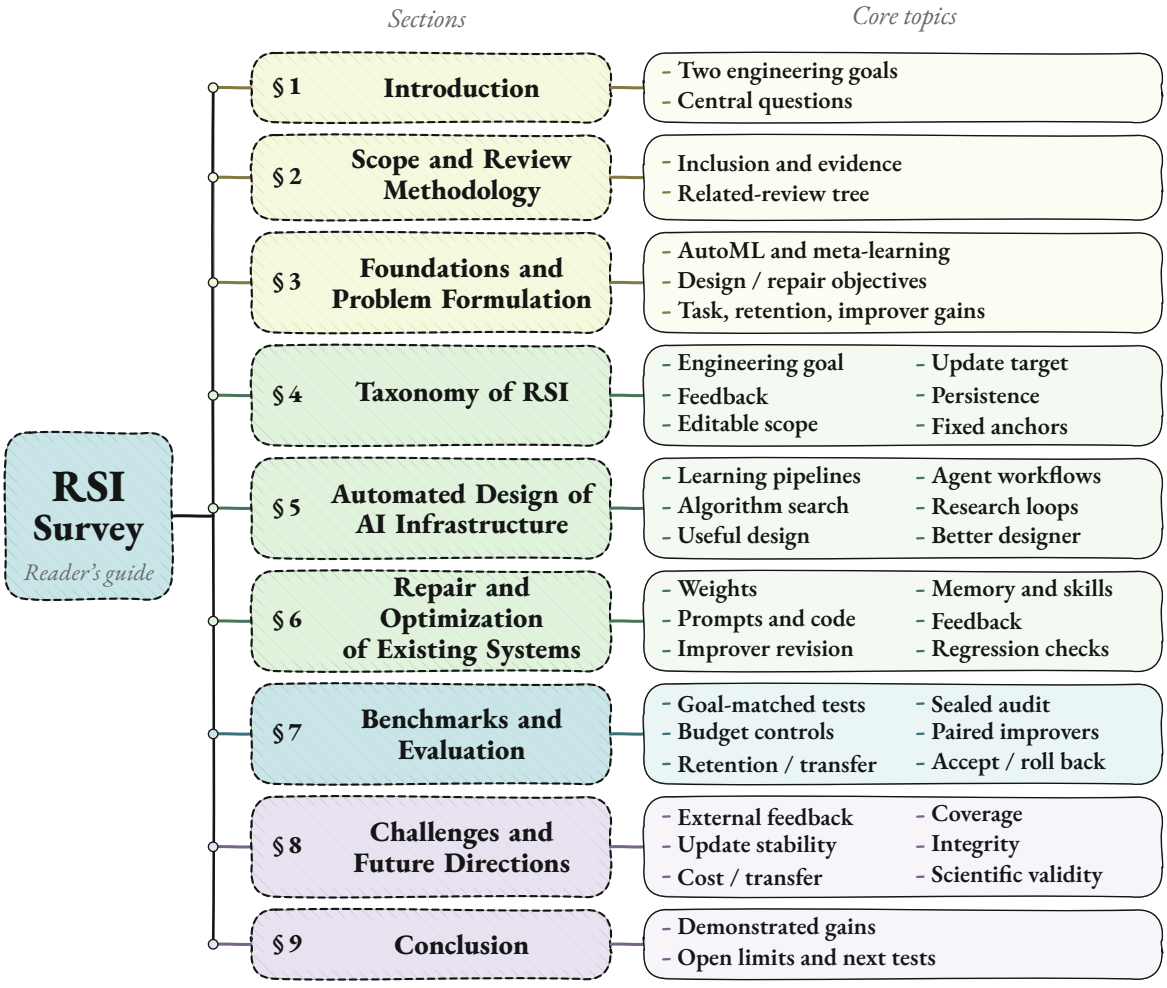


Figure 2. Organization of the survey. The nine sections connect two engineering goals to common mechanisms and tests. Infrastructure design and repair can share the same update operators. Branches describe this article's organization, rather than technical ancestry or a ranking of approaches.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

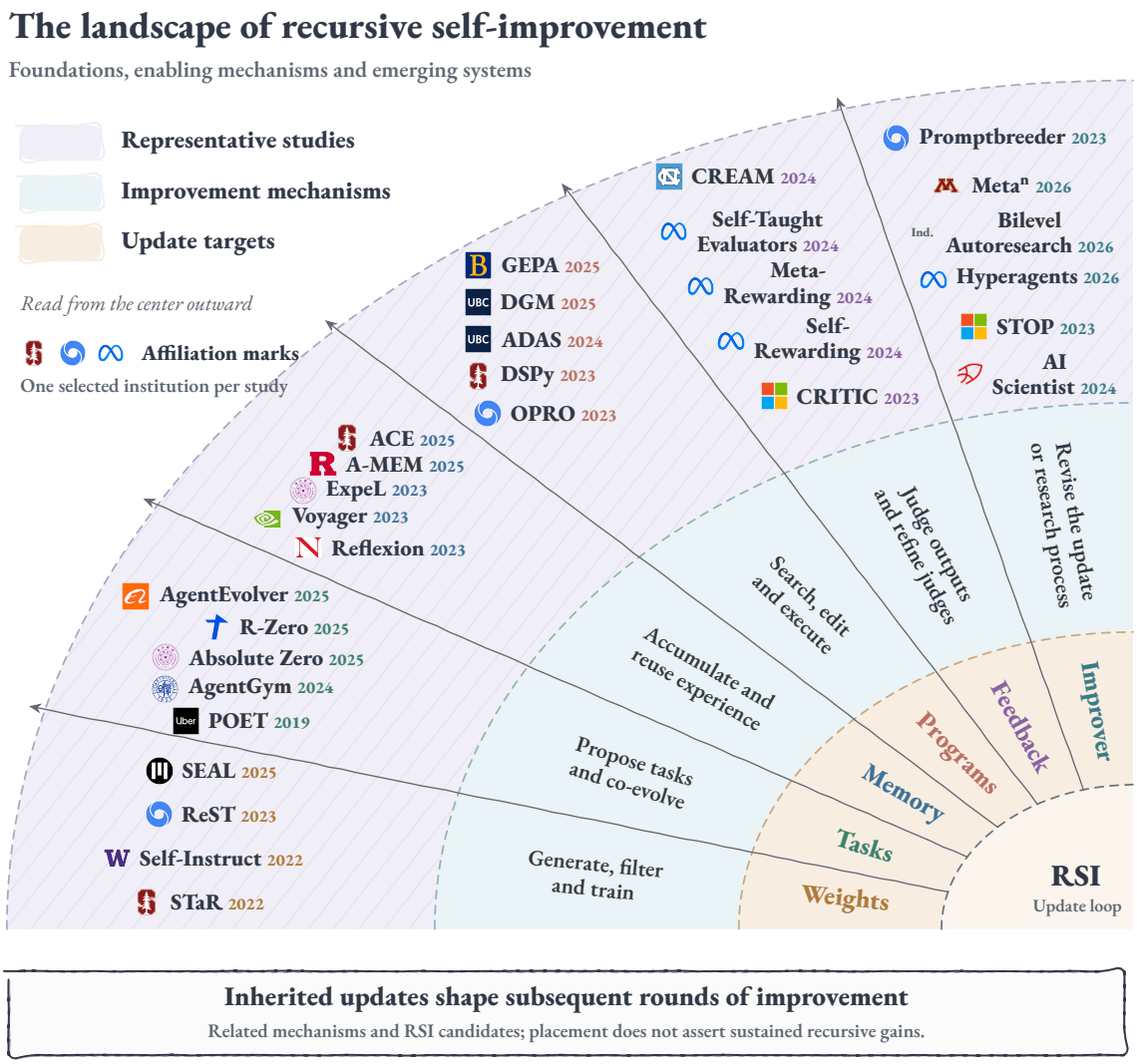


Figure 3. Landscape of mechanisms relevant to recursive self-improvement. The map connects update targets in the inner band, improvement mechanisms in the middle band, and representative studies in the outer band. The six sectors cover weights, tasks, memory, programs, feedback, and the improvement process. A work may involve several targets; its placement highlights one connection. The map includes historical foundations, enabling methods, and RSI-oriented systems. Logos denote selected author or work-done affiliations, rather than base models; other collaborating institutions may be omitted. The accompanying source register records each selection.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

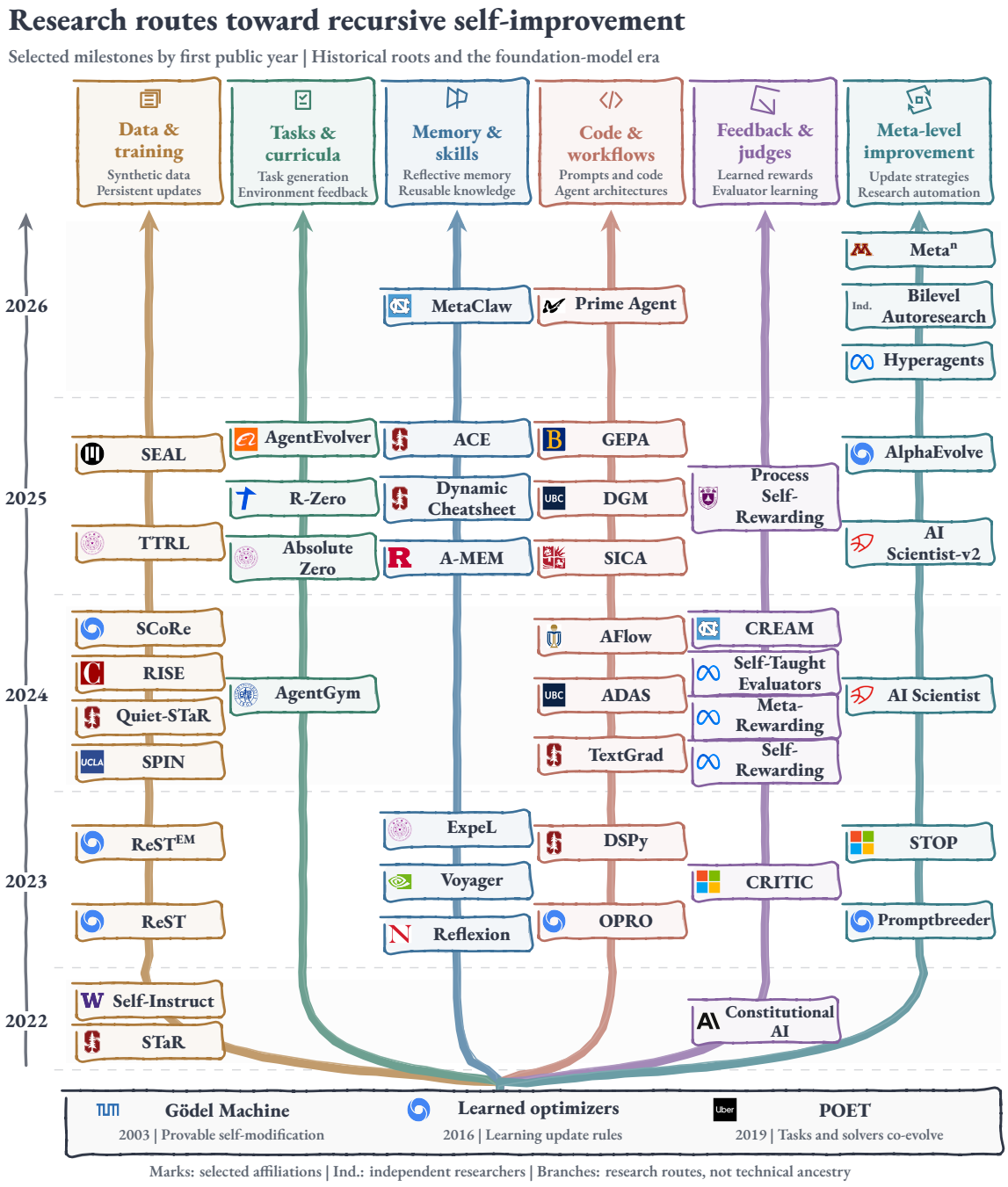


Figure 4. Research routes toward recursive self-improvement. Forty-five representative method papers are grouped into six research routes, with three historical anchors shown separately. Year bands use the first public dates in the checked source records; spacing within each band is adjusted for legibility. Branch lines denote research families. Technical ancestry, cross-route transfer, and sustained recursive gains require separate evidence. The selection reflects the literature search through September 12, 2026. Logos identify one selected author or work-done affiliation per study; "Ind." denotes independent researchers. Affiliation and logo sources are documented separately.

2. Scope and Review Methodology

2.1. Research Scope and Inclusion Criteria

We review foundation models and agents that use generated data, interaction experience or evaluation feedback to change later behavior. We focus on methods that modify the process used to generate subsequent improvements. Self-training, memory, tool construction and automated design supply enabling mechanisms; local refinement and failure studies clarify boundaries. Historical work

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

enters where it explains a mechanism or supplies a necessary comparison. Inclusion in the survey does not imply that a method demonstrates RSI.

Our unit of analysis is a concrete configuration, with a specified update target, feedback source, retained artifact and externally fixed conditions. A paper can describe several configurations, and one configuration can serve both design and repair. We distinguish an implemented recursive dependency from evidence that this dependency produces useful gains.

2.2. Literature Discovery and Evidence Organization

We identify relevant studies through an initial set of papers, citation tracing and targeted searches for related mechanisms. We check publication details against paper pages on arXiv, ACL Anthology, Preprints.org and conference proceedings. The main search covers work available by 12 September 2026. To expand the initial bibliography of 125 works, we screen the bibliographic records and abstracts of 442 distinct arXiv papers and include 275 additional works. A historical supplement dated 20 September adds four references on AutoML and architecture search. The bibliography therefore contains 404 distinct works, including those cited in figures. The supplement does not extend the cutoff for recent work.

For each entry, we retain the source link and version information. When a preprint and a published paper are confirmed to describe the same work, we count them once. We distinguish screening bibliographic records and abstracts from examining methods and experiments in the full text. Not all included papers receive the same depth of full-text review. We use the broader collection to identify related approaches and restrict detailed numerical comparisons to experiments we inspect. This is a narrative survey, not an exhaustive systematic review or a meta-analysis. We mark protocol details as unconfirmed when the inspected material does not specify them. The selected samples do not support estimates of how common a practice is across the field.

The synthesis links three levels: *mechanisms* (what changes and persists), *protocols* (splits, resources, controls and horizons), and *claims* (task gains, retention, transfer or stronger improvers). This separation prevents a descriptive method label from substituting for an experimental result.

2.3. Relationship to Existing Surveys

The neighboring survey literature can be viewed through three intersecting perspectives, summarized in Figure 5. Lifecycle and agent surveys organize the objects, stages, and application settings of self-improvement [8,9]. Component surveys examine how particular mechanisms operate and what state they preserve [12–14]. RSI-oriented syntheses focus more directly on how the improvement loop is closed and what evidence supports recursive improvement [10,11]. These perspectives are thematic rather than mutually exclusive, so a survey may contribute to more than one branch.

Table 1 compares existing surveys along three groups of dimensions derived from our research questions. **System boundary** asks which components can be updated and what information is retained or reset between improvement rounds. **Recursive agency** asks whether the improver that generates updates and the judge or criterion that evaluates them remain fixed or can change across rounds. **Evaluation evidence** asks what evidence supports a claimed improvement, including evaluation data kept outside the update process, controls for attributing gains to the update, full accounting of the resources used, longitudinal tests of retention and transfer, and matched comparisons between improvers.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

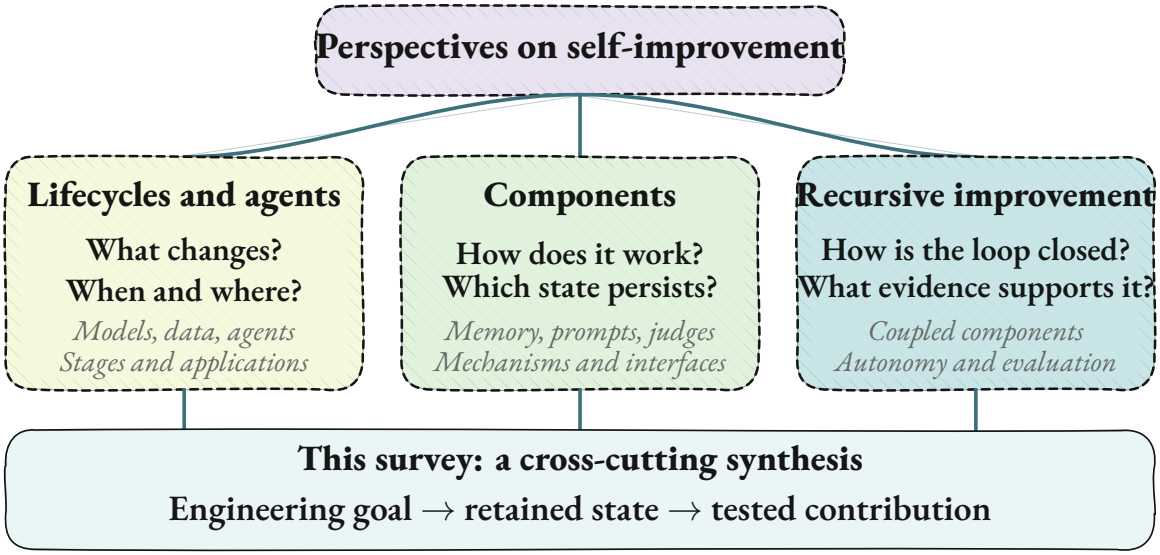


Figure 5. How neighboring surveys organize the field. The three branches summarize complementary lifecycle- and agent-level, component-level, and RSI-oriented perspectives. Overlap between the branches is expected. The lower band states the cross-cutting questions adopted in this survey. The figure is a thematic reading guide rather than a citation genealogy or priority claim.

We assign full coverage when a dimension forms a central or systematic part of a survey's framework, partial coverage when it is discussed explicitly but only to a limited extent, and no systematic coverage when it falls outside the survey's main scope. The assessment reflects the role of each dimension in the survey's overall framework rather than isolated mentions. It characterizes differences in scope, not the overall quality of the surveyed work.

Early general surveys primarily organize self-improvement around the model and its immediate update process. The initial self-evolution perspective follows the acquisition, refinement, updating, and evaluation of experience [15]. Agent-level surveys broaden the unit of analysis to models, memory, tools, architectures, environments, and optimizers [8,16]. Lifecycle and system-oriented views further connect data, training, inference, and evaluation with the operational scaffold surrounding the foundation model [9,17].

Recent RSI-oriented syntheses shift attention from individual updates to the structure of the improvement loop, including persistent system changes and the evolution of the improver itself [10,11,18]. They further examine the autonomy and reliability conditions required to sustain and verify improvement across successive rounds [19–21].

A second group studies self-evolution within broader agent architectures or particular application settings. Surveys of persistent autonomous agents, foundation agents, and lifelong learning emphasize workspaces, reusable skills, agent components, and the retention and transfer of experience across tasks [22–24]. More specialized surveys examine how the available update targets, feedback channels, and evaluation procedures are shaped by multimodal interaction or software-engineering environments [25,26].

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 1. Coverage of RSI-relevant dimensions in existing surveys. Surveys are grouped according to their primary organizing scope.

Survey	System boundary		Recursive agency		Evaluation evidence				
	Sys	Ret	Meta	Judge	Ext	Attr	Cost	Long	Pair
A. General RSI and self-evolution surveys									
Tao et al. (2024) [15]	△	△	—	—	△	—	—	△	—
Gao et al. (2025) [8]	✓	✓	△	△	△	—	△	✓	—
Fang et al. (2025) [16]	✓	✓	△	△	△	—	—	✓	—
Yang et al. (2026) [9]	✓	△	△	△	△	—	△	△	—
Ren et al. (2026) [17]	✓	✓	△	△	△	△	△	✓	—
Jiang et al. (2026) [18]	✓	✓	✓	△	✓	✓	△	✓	△
Liu et al. (2026) [10]	✓	✓	✓	△	✓	✓	△	✓	△
Chen et al. (2026) [11]	✓	△	△	✓	✓	△	△	△	—
Wu et al. (2026) [19]	✓	✓	✓	✓	✓	✓	✓	✓	△
Duan et al. (2026) [20]	✓	✓	✓	△	△	△	△	✓	—
Wang et al. (2026) [21]	✓	✓	✓	✓	✓	✓	△	✓	△
B. Domain-specific or structurally constrained surveys									
Zhang et al. (2026) [22]	✓	✓	△	△	△	△	△	✓	—
Liu et al. (2025) [23]	✓	△	△	△	△	—	—	✓	—
Zheng et al. (2025) [24]	✓	✓	—	—	△	—	—	✓	—
Deng et al. (2025) [25]	△	△	—	—	△	—	—	△	—
Zhou et al. (2026) [26]	✓	✓	△	△	△	△	△	✓	—
C. Boundary, component, and supporting surveys									
Wang et al. (2025) [27]	✓	△	—	—	△	—	—	—	—
Dong et al. (2024) [28]	—	✓	—	—	△	—	△	—	—
Zhang et al. (2024) [12]	—	✓	—	—	△	—	—	✓	—
Wu et al. (2025) [29]	—	✓	—	—	—	—	—	✓	—
Ramnath et al. (2025) [13]	—	△	—	—	△	—	△	—	—
You et al. (2026) [14]	—	—	—	△	△	—	—	—	—
This survey	✓	✓	✓	✓	✓	✓	✓	✓	✓

✓ central and systematic treatment; △ explicit but partial treatment; — outside the main scope or not treated systematically.
Sys: system-level update scope; **Ret**: retention/reset boundary; **Meta**: improver evolution; **Judge**: judge/criterion evolution; **Ext**: protected external evidence; **Attr**: causal-attribution controls; **Cost**: full-budget accounting; **Long**: longitudinal retention/transfer; **Pair**: paired-improver test.

A third group of surveys focuses on boundary conditions or offers detailed analyses of individual components. Full-stack safety and inference-time self-improvement respectively clarify the risks associated with mutable systems and the distinction between temporary inference gains and persistent updates [27,28]. Memory surveys examine how experience is represented, retrieved, consolidated, and revised over time [12,29], while surveys of automatic prompt optimization and agentic judging provide more focused accounts of prompt-level updates and evaluator agents [13,14]. These specialized surveys offer depth on individual mechanisms and boundaries, whereas the present survey examines how these mechanisms interact within a recursive improvement process.

Across these perspectives, our unit of analysis is an inherited change and its effect on subsequent improvement. We ask what is updated, what persists into later rounds, whether the improver or evaluation process changes, and what evidence attributes later gains to those changes. This framing distinguishes persistent and recursive improvement from better execution produced by an unchanged procedure. Table 2 provides a finer-grained comparison of ten representative surveys and roadmaps, examining their scopes, organizing perspectives, and evaluation emphases. Together, Figure 5, Table 1, and Table 2 position this survey at the intersection of persistent system change, evolving improvement processes, and cross-road evidence of improvement.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 2. Related surveys and roadmaps. Dates denote checked versions. Fang et al. and Wang et al.: abstracts only; others: selected full-text passages. Relationships indicate emphasis, not priority.

Survey / version	Scope	Evaluation focus	Our emphasis
Gao et al. [8] v4 · Jan 2026	Agent evolution: objects, timing, mechanisms, domains	Adaptation; retention; transfer; efficiency; safety	Retained state; fixed drivers
Fang et al. [16] v2 · Aug 2025	Lifelong agents: inputs, system, environment, optimizers	Evaluation; safety; ethics (abstract only)	Component changes; tested claims
Deng et al. [25] EMNLP Findings 2025	Multimodal improvement: data collection, organization, optimization	Response refinement vs. training; multimodal evaluation	Memory and improver state
Yang et al. [9] v3 · Sep 2026	LLM lifecycle: data, training, inference, evaluation	Collapse; feedback reliability; evaluation costs	Retained outputs; subsequent updates
Chen et al. [11] v2 · Sep 2026	Improvement targets × loop closure; automated research	Self-evaluation; result vs. process improvement	Configuration comparisons; paired-improver tests
Zhou et al. [26] v1 · Aug 2026	Coding agents: framework, memory, tools, models, collaboration	Software benchmarks; feedback reliability; overfitting	Execution feedback vs. independent testing
Liu et al. [10] v1 · Aug 2026	Coupled model, harness, data, trainer and improver	Five capability grades; validation; integration	Evidence for retained changes
Wu et al. [19] v1 · Aug 2026	AI4AI; research stages; self-reference; grounded signals	Gains; retention; human comparison; held-out transfer	Retained state; improver interventions
Duan et al. [20] v1 · Sep 2026	Autonomy: execution, strategy, experience and meta-improvement	Level-specific tests; successor quality; industrial evidence	Paper-specific evidence; no maturity score
Wang et al. [21] v1 · Sep 2026	Output, model, scaffold, improver and criterion changes	Reliability ladder; external evaluation (abstract only)	Reliability framing; full-text comparison pending
This survey Current draft	Retained state; editable improvers; mechanism families	Test boundaries; costs; retention; controls; paired-improver tests	Mechanisms linked to evidence; coding ongoing

3. Foundations and Problem Formulation

3.1. Definitions and Historical Continuity

The term self-improvement covers several claims. In its broad usage, a system uses its outputs or experience to improve later performance. A structural RSI claim adds a return path through which an

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

improvement procedure acts on itself or on inherited artifacts that govern subsequent improvement. An empirical RSI claim additionally requires evidence that this return path makes later improvement more effective. We use these distinctions rather than assign every included method one binary RSI label. Shared weights or code can implement several roles, and a fixed operator can process an evolving recursive context.

The Gödel Machine supplies a formal reference: self-rewrites depend on proofs of utility under specified axioms, with the proof searcher itself eligible for rewriting [30]. This conditional guarantee is not a guarantee for an empirical LLM agent that accepts edits by testing. Bounded RSI and seed-AI analyses also expose assumptions about resources, representation and the ability to recognize useful changes [31,32]. Modern systems replace some formal requirements with executable feedback and statistical evaluation, making the resulting evidence conditional on the tasks and tests available.

From AutoML to editable improvement processes.

Bayesian optimization adapts its proposals to earlier evaluations; Auto-WEKA applies automated search to joint algorithm and hyperparameter selection, while auto-sklearn adds meta-learning and ensemble construction [33–35]. Neural architecture search learns a controller that proposes architectures from validation rewards [36]. These are already adaptive, feedback-driven procedures. A fixed outer algorithm does not imply that its search state is static, and a learned controller is already an improvement component. The additional RSI question is whether inherited changes improve that component's ability to produce successors, and whether this advantage survives new tasks and a full accounting of cost.

Meta-learning makes the overlap explicit. Learned optimizers change update rules; RL^2 learns adaptation through recurrent dynamics; MAML learns an initialization [37–39]. AutoML-Zero searches programs representing learning algorithms, while POET couples solver evolution, environment generation and transfer [40–42]. Evolution through Large Models connects program mutation, archives and model training [43]. These traditions supply the representations, search operators and transfer tests that modern RSI inherits. Language models broaden the editable objects to natural-language instructions and general programs; that broader interface does not remove the need to validate the resulting updates.

Table 3. Historical connections. A fixed outer algorithm can still maintain adaptive search state. The final column specifies an additional question, not a property absent from every method in a family.

Tradition	Search / learning target	Retained object	Additional RSI question
AutoML / NAS [34–36]	Pipelines; architectures; configurations	Models; search history; learned controller	Better future design at matched cost?
Meta-learning [37–39]	Update rule; dynamics; initialization	Meta-learned state	Can its production or revision improve?
Program / open-ended search [40,41]	Algorithms; environments; solvers	Programs; populations; archives	Does discovery improve later discovery?
Tool-using agents [44,45]	Actions; tool use	Traces or learned policies	What survives, and affects later updates?
Self-modification [30,46]	Solver and/or improver code	Revised executable state	Do tested or proved gains survive reuse?

3.2. Two Engineering Goals and Their Objectives

We organize applications by their immediate engineering goal. The equations below are analytical templates for comparison, not objectives claimed to be implemented by every paper. Let S denote a task system and let

$$Q(S; D) = \mathbb{E}_{z \sim D}[u(S, z)] \tag{1}$$

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

measure utility on a task distribution D, with higher values preferred. The utility u can encode correctness or a declared combination of quality and deployment cost. Training, development, validation and sealed test distributions serve different roles; D_{test} never selects an update.

Goal A: automated design of AI infrastructure.

Infrastructure includes learning pipelines and agent architectures, as well as the tools and evaluators that support them. Let a specify a design, λ its configuration, and θ trainable parameters when present. A common formulation is

$$\begin{aligned} (a^*, \lambda^*) &\in \arg\max_{a,\lambda} Q(S(a, \lambda, \theta^*(a, \lambda)); D_{\text{val}}), \\ \theta^*(a, \lambda) &\in \arg\min_{\theta} L_{\text{train}}(\theta; a, \lambda, D_{\text{train}}), \qquad C_{\text{search}} \leq B. \end{aligned} \tag{2}$$

Here L_{train} is the specified training loss, C_{search} counts all candidate construction and evaluation, and B is the search budget. For training-free agent design, the inner training problem is absent and a can specify a workflow, tool library or prompt program. A new design is an output of the search. A recursive return path exists only when the retained result also affects how later designs are generated or selected.

Goal B: repair and optimization of an existing system.

Let $S_t \oplus \delta$ apply an allowed edit δ to the incumbent. Repair emphasizes a target failure distribution D_{target} and protected capabilities D_j. One useful constrained formulation is

$$\begin{aligned} \delta_t^* \in \arg\max_{\delta \in \mathcal{A}_t} \quad & Q(S_t \oplus \delta; D_{\text{target}}) - \lambda_e d(\delta), \\ \text{subject to} \quad & Q(S_t \oplus \delta; D_j) \geq Q(S_t; D_j) - \epsilon_j \quad \forall j, \\ & C_{\text{update}}(\delta) \leq B_t. \end{aligned} \tag{3}$$

The allowed edit set $\mathcal{A}_t$ can contain patches, memory operations, prompt changes or parameter updates. The optional penalty d discourages unnecessary change, $\lambda_e \geq 0$ weights it, and $\epsilon_j \geq 0$ sets an allowed regression tolerance. All selection data here are development or validation data. An independent audit tests whether the selected edit also works outside them. The mathematical optimum is a specification; an actual search can fail to find a feasible or useful edit.

Design and repair overlap. Replacing a workflow after a failure is both a repair of its current behavior and a design decision. The distinction lies in the experimental goal and constraints, not the format of the code or whether model weights change. We retain both labels when needed.

3.3. A Unified Description of the Improvement Loop

Following Figure 1, let $X_t = (S_t, I_t)$ denote the retained state at the start of round t. Here, S_t represents the state that supports task solving, including model parameters, prompts, memory, skills and executable programs. I_t represents the process that produces or selects subsequent updates, including its programs, parameters, strategies and persistent context. This is a distinction between functional roles: one model or program can support both task solving and improvement. The roles can therefore share an implementation and need not have independently separable parameters.

Let E_t denote the experience and feedback available for improvement in the current round, V_t the information used to validate candidates and select updates, and $\mathcal{C}$ the conditions declared to remain externally fixed. These conditions can include task objectives, data-access rules, resource constraints, and models, tools and evaluation interfaces outside the update scope. An update takes the following general form:

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

$$\begin{aligned} \widetilde{X}_{t+1} &\sim P_{I_t}(\cdot \mid X_t, E_t; \mathcal{C}), \\ X_{t+1} &= \mathcal{R}_t(X_t, \widetilde{X}_{t+1}, V_t; \mathcal{C}). \end{aligned} \tag{4}$$

Here, P_{I_t} describes candidate generation under the current improvement process, $\widetilde{X}_{t+1}$ is a candidate state, and $\mathcal{R}_t$ denotes the actual validation and retention mechanism. Updatable parts of the retention mechanism fall within the analysis of I_t, while its externally fixed constraints belong to $\mathcal{C}$. Validation can accept a candidate, retain the existing task system and improver, or place a candidate in an archive for later selection; failed attempts and their costs also require recording. For evolution with multiple branches, X_t can include the candidate archive and selection state, allowing this representation to accommodate non-greedy retention and multiple exploration paths.

This representation emphasizes what persists across rounds and how it serves subsequent activity. Figure 6 illustrates four typical patterns. Local refinement changes the current answer. Memory accumulation writes experience into external state available for later access. Parameter learning changes model weights through training. An improver update retains a change in the improvement process as I_{t+1}, which participates in generating changes in the next round. In the local-refinement pattern shown in the figure, answer revisions do not become persistent updates to the solver or improver. The other patterns can combine, as when a system updates its skill library, model parameters and improvement prompts together.

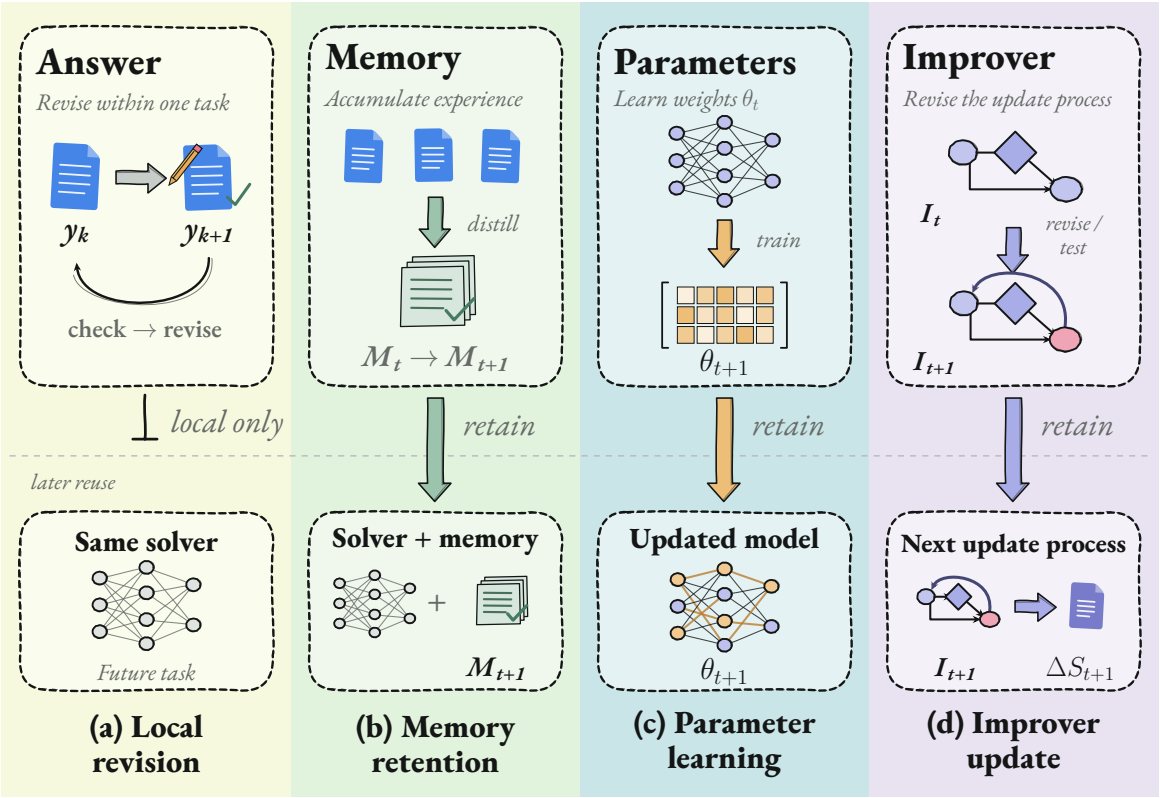


Figure 6. What survives a round of improvement? Four patterns distinguished by the state carried forward. (a) Local checking and revision change an answer without retaining a change to the solver. (b) Memory M is retained while the model parameters and the chosen update procedure are held fixed. (c) Parameters θ are updated under a fixed training procedure. (d) An accepted change to the improver I is retained and governs a subsequent task-system update ΔS_{t+1}. The horizontal gray dashed line separates current-task or current-round processing from later reuse; black dashed enclosures group components. Colored arrows denote retained updates, and a capped line denotes a change that is not carried forward. Networks, memory stacks, and procedure graphs are schematic representations. Solver and improver are functional roles that may share an implementation. These analytical patterns may coexist and do not form a capability ranking; persistent or meta-level changes alone do not establish sustained recursive gains.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Retention requires an explicit task, session or training boundary. Context that grows within one run has a different persistence scope from memory that remains available after a task reset. Parameter updates likewise require specifying whether the next task restores the previous parameters [47, 48]. Similarly, fixed external conditions delimit the update scope under study without implying independence from human design, pretraining knowledge or environmental feedback. Our subsequent method comparisons record both the components that change and the conditions that continue to support those changes.

3.4. Three Distinct Empirical Claims

The loop supports three questions with different observables. At a scheduled checkpoint t, task gain is

$$\Delta_t = Q(S_t; D_{\text{test}}) - Q(S_0; D_{\text{test}}). \tag{5}$$

For a task group j first acquired at round t_j, later retention and out-of-distribution transfer can be recorded as

$$\rho_{t,j} = Q(S_t; D_j^{\text{test}}) - Q(S_{t_j}; D_j^{\text{test}}), \qquad \tau_t = Q(S_t; D_{\text{new}}^{\text{test}}) - Q(S_0; D_{\text{new}}^{\text{test}}). \tag{6}$$

The reference checkpoint t_j is predeclared; $\rho_{t,j} < 0$ indicates a loss relative to it. The new distribution specifies the intended shift. Neither positive Δ_t nor positive τ_t isolates the improver.

For that question, let $S^+(I; S, B)$ be the successor selected by improver I from the same starting solver S within budget B. Define

$$g(I; S, B) = \mathbb{E}\big[Q(S^+(I; S, B); D_{\text{test}}) - Q(S; D_{\text{test}})\big], \tag{7}$$

where the expectation covers update randomness and sampled tasks. A paired probe on held-out starting states drawn from $\mathcal{P}_{\text{start}}$ tests

$$\Gamma_t(B) = \mathbb{E}_{S \sim \mathcal{P}_{\text{start}}}[g(I_t; S, B) - g(I_0; S, B)]. \tag{8}$$

Positive Γ_t with appropriate uncertainty supports a stronger improver under that probe. Controls must match starting states, feedback access and resource accounting. The expense of obtaining I_t is additional and belongs in an end-to-end comparison. If solver and improver share inseparable state, a swap can change both roles; component ablations then support a narrower claim, and the coupling must remain explicit.

STOP's self-modifying program and Hyperagents' editable meta-agent expose update-producing components; Meta^n instead retains recursive context for a fixed operation [6,7,46]. Their structure motivates a test, rather than supplies its outcome. Even increasing task scores over a finite horizon do not demonstrate accelerating improvement: that stronger claim requires gains per unit cost over longer horizons, without changing the criterion that defines success.

3.4.1. What Theoretical Results Establish

Recent theory addresses more specific mechanisms. A framework for self-play theorem proving models relations among theorems as a graph and derives expansion results under assumptions about connectivity, the prover and the conjecturer [49]. Work on iterative self-rewarding derives guarantees within an explicit statistical model, including dependence on initialization and sample size [50]. Such results explain possible sources of improvement under stated conditions. They do not imply that arbitrary language-model judges or generated curricula satisfy those conditions.

The information carried by a synthetic training distribution is another useful analytical object. The reverse-bottleneck account relates post-training generalization to information gain from the generative model [51]. The power-distribution analysis connects sampling, self-reward optimization and distillation, while distinguishing an increase in self-reward from an increase in the downstream

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

reward of interest [52]. Together, these perspectives motivate measuring what new training material makes learnable, rather than using sample count or internal confidence as a sufficient account of progress.

Learning an update procedure also predates current agent scaffolds. Automated Continual Learning trains self-referential networks to acquire in-context learning algorithms that balance old and new tasks [53]. Its controlled setting offers a concrete example of optimizing how learning proceeds. Conversely, reward-overoptimization studies show that stronger optimization of a proxy can reduce performance against a separate reference criterion [54]. For RSI, these observations belong together: making an update process more effective requires specifying the objective against which its effectiveness is judged.

3.4.2. Local Revision and Learned Correction

Self-correction is an informative boundary case because repeated calls can change an answer without changing the system that answers the next question. Confidence Matters and the uncertainty-based account of intrinsic self-correction investigate when revisiting a response can help; a more recent task-sensitive analysis relates its usefulness to the structure of the task [55–57]. These findings caution against treating self-correction as a uniform capability across problems. They also motivate controls that preserve already-correct responses, rather than assuming every revision deserves acceptance.

Error recognition and correction activation are separable. Self-Correction Bench compares otherwise matched errors attributed to the model or to an external speaker, while DISC uses a judgment gate to limit harmful rewrites [58,59]. Symbolic feedback for planning and LLM-Augmenter's externally grounded feedback show how additional evidence changes the correction problem [60,61]. The information introduced by a verifier, retrieval system or task constraint belongs in the account of the gain; it should not disappear under the label of introspection.

Correction can become a persistent skill when it is trained. STaSC uses self-generated data to learn correction; EVOLVE coordinates training and inference to develop refinement behavior; and Generative Self-Refinement jointly trains generation and aggregation using transferred refinement supervision [62–64]. LADDER instead recursively constructs simpler problem variants for learning and test-time adaptation [65]. Here, recursion over subproblems and recursion over an improvement procedure are different structures even when both support better later predictions.

The intervention level also matters in research. SAGE's multi-hypothesis failure attribution distinguishes changes to a hypothesis, experimental design and implementation [66]. This illustrates why a fluent reflection is an incomplete description of an update. The operational question is which observed failure warrants which change, and whether that change survives verification at the level where the failure occurs.

> **Highlight. Recursion is a dependency; improvement is an observation**
>
> Task gain Δ_t, retention $\rho_{t,j}$ and improver gain Γ_t answer different questions. Automated design and repair can contribute to all three, but a good final artifact alone establishes none of the additional claims about persistence or successor production.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 4. Conceptual boundaries of RSI. Patterns overlap. The final column poses evaluation questions, not claims established by every cited example.

Concept / examples	Updated state	RSI boundary	Key question
Local self-refinement [67,68]	Task-local responses, critiques and traces	Repeated correction need not change the solver	Retention after reset? Reliable correction?
Inference-time search [69–71]	Reasoning paths; search statistics; local reflections	Recursive search can use fixed models/rules	Equal-budget gains? Retention beyond the episode?
Self-training [1,72,73]	Synthetic examples; filtered rationales; weights	Generator can change; training/filtering may stay fixed	Transfer beyond generated data? External supervision?
Self-play / curriculum [74–76]	Policies; opponents; tasks; experience distribution	Adaptive curricula need not change learning rules	Which components change? Who fixes the objective?
Memory / skill retention [3,4,77,78]	Reflections; summaries; executable skills/tools	Persistent state; weights/update rule may stay fixed	When reused? Cross-task retention ablation?
Self-reward / meta-reward [2,79]	Preferences; judging behavior; actor/judge weights	Judge improvement differs from criterion/update-rule changes	Independent gains? Actor/judge role ablations?
Automated agent design [80–82]	Prompts; workflows; agent programs	Improved agents can have a fixed search driver	Artifact transfer? Is the driver updated/inherited?
Meta-learning / learned updates [37–39,83]	Initialization; learning dynamics; learned update rules	Adaptation can rely on fixed external meta-training	Which level learns? Is meta-training itself modified?
Improver self-modification [6,46,84]	Mutation prompts; improver code	Editable improvers do not establish sustained gains	Better successors at matched initial states and cost?
Recursive context / nested research [7,85]	Meta-context; evidence; unresolved constraints	Meta^n: fixed Ω; AREX: answer/context refinement	What persists? Which process changes?

4. A Taxonomy of Recursive Self-Improvement

4.1. Two Axes: Engineering Goal and Update Target

The engineering goal explains *why* a change is made; the update target explains *what* changes. Infrastructure design seeks a useful pipeline, architecture or improvement procedure. Repair and optimization start from an incumbent and a failure, performance gap or resource constraint. Either can change weights, data, memory or code. We therefore use the two goals to organize the methods discussion and retain the six overlapping target labels in Figure 3 for mechanism comparison.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Each configuration also receives three descriptors. *Feedback* identifies answers, execution, environmental outcomes, model judgments or human input, and whether each signal trains, proposes or selects an update. *Persistence* specifies the task, session or training boundary across which state survives. *Control* identifies editable proposal, training and selection components alongside fixed objectives, tools and budget rules. A label such as self-training or self-modification leaves these descriptors undetermined; they must come from the implementation.

4.2. Six Overlapping Mechanism Families

Parameters and data.

Parameter updates internalize generated or selected experience. Data and task updates alter the examples, curricula or task-generation policy that make learning possible. These mechanisms often coexist: STaR returns selected reasoning traces to training, while Absolute Zero learns task proposal and solving with executable feedback [1,75]. Their classification must keep the generator, acceptance criterion and trainer separate. Learning a task proposal policy does not imply learning the rule that trains that policy.

Memory and skills.

Persistent text, graphs, latent states and executable libraries retain experience without necessarily changing the backbone. Reflection, experience abstraction and skill reuse differ in what later computation accesses [3,4,77]. A stored record belongs to the solver when it guides task execution and to the improver when it guides diagnosis or update selection; the same record can serve both. Changing a memory's contents and changing its management algorithm are separate interventions.

Prompts, programs and agent harnesses.

Textual instructions, tools and orchestration code govern how a model uses its capabilities. A searched prompt, generated workflow and revised agent program therefore share an operational target, despite different search operators [80,82,86,87]. The scope ranges from the current problem to a deployed agent family. A recursive call graph describes execution; it does not by itself show that an improved artifact survives to the next task.

Feedback and evaluation.

Judges, critics, tests and reward models affect what the loop accepts. Self-rewarding and meta-rewarding make this role explicit, but actor and judge can share the same parameters [2,79]. Feedback quality should be measured against an independent reference, and its effect on downstream updates tested separately. A changed evaluator can alter the scale on which apparent progress is reported.

The improvement process.

A prompt that generates mutations, a program that edits programs, a memory-management policy or a learned workflow designer can all act as an improver. Promptbreeder, STOP and Hyperagents expose different forms of editable update production [6,46,84]. Meta^n illustrates recursive conditioning under a fixed operation [7]. These functional labels overlap with the state carriers above: an improver may be stored in weights, code or context. The relevant evidence traces the changed object into the next update.

4.3. Reading Combinations Without Imposing a Capability Ladder

The four patterns in Figure 6 distinguish local answer revision, memory retention, parameter learning and improver updates. They can coexist in one system and do not establish an ordering of intelligence. A small memory update can be more useful than a large training run; an editable improver can repeatedly regress. The task, budget and retention boundary determine which comparison is meaningful.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

The research routes in Figure 4 likewise denote families, not verified lines of technical descent. Their chronology helps locate shared ideas, while the two engineering goals organize what the systems attempt to accomplish. Sections 5 and 6 synthesize these goals; Section 7 tests the claims that follow from them.

5. Automated Design of AI Infrastructure

Infrastructure design determines the machinery with which an AI system learns, acts and evaluates alternatives. Its outputs include data pipelines, learning algorithms, agent workflows, tools and research procedures. Equation 2 places these outputs under a shared design-and-evaluation view. We group the literature by the design decision it automates, then ask what the result contributes to the next design round.

5.1. Learning Pipelines and Task Curricula

A learning pipeline chooses what experience to collect, which examples to admit and how to turn them into updates. Generated instructions expand task coverage; rationales and executable solutions provide more structured training material [1,72,88]. The design problem is consequently broader than producing more samples. A pipeline must preserve informative variation while controlling error, redundancy and the cost of verification. Its output should be judged by the successor it trains, rather than the fluency of its synthetic data.

Curriculum methods adapt the difficulty, composition or source of tasks. Self-play couples this choice to the learner's current behavior. Absolute Zero makes task proposal and solving learnable but retains an external executor and training rule [75]; POET couples evolving tasks, solvers and transfer [41,42]. These systems expose a common tension: tasks should be solvable enough to yield feedback and diverse enough to extend capability. Raising difficulty alone can eliminate the learning signal. A fixed independent test distribution is needed to separate better learning from a task generator's changing preferences.

Curriculum mechanisms.

Three clusters refine this choice. Scheduling methods select or order preference pairs, reasoning difficulty and learning stages [89–93]. Co-evolution methods couple task proposal with a changing solver or critic [94–99]. Population and multi-agent approaches instead vary opponents, collaborators or model populations [100–105]. These mechanisms change the supply of experience in different ways; a harder task pool and a more diverse population are not interchangeable measures of learning opportunity.

A second axis concerns grounding. Verified problem construction, learned critics and solver–verifier interaction differ in who can certify an answer [106–108]. Executable software tasks and formal mathematics offer narrower checks than unrestricted language interaction [109,110]. Task construction from tools, self-challenges and interactive experience broadens coverage but also changes the sources of uncertainty [111–113]. Test-time curriculum and self-play methods additionally make the persistence boundary task-dependent [114,115]. Across these clusters, the common test is whether generated experience improves a learner on an independent task distribution after accounting for generation and checking.

Evaluator design is another part of the pipeline. Constitutional principles, model preferences and process supervision supply different forms of feedback [116–119]. Their external information differs: a human-written principle, a reference answer and a tool outcome cannot all be counted as an absence of supervision. Changing an internal judge is compatible with RSI, but independent task checks must remain outside the criterion it can rewrite. Section 6.4 examines the maintenance of these learned feedback components.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

5.2. Agent Architectures, Workflows and Tool Interfaces

Agent design searches the arrangement of reasoning, tool use, memory and verification. ReAct and Toolformer establish complementary interfaces between language models and external actions [44,45]. Tool construction extends this interface from choosing existing actions to writing new executable capabilities [120,121]. Self-Discover, AgentSquare and ADAS make reasoning structures, modular compositions or agent programs explicit design objects [80,81,122]. The shared search space is an executable organization of capabilities, rather than only the parameters of one model.

Workflow methods cluster around three choices: how to represent a design, how to allocate roles and communication, and whether to train the designer. A natural-language program admits broad edits but can hide interface changes; a graph exposes dependencies but constrains the operators available. Learned designers can reuse experience across searches, while a fixed optimizer can still adapt its candidate distribution within a run. Useful comparisons therefore report both the artifact and the state of its designer.

Workflow design mechanisms.

Representation-oriented methods manipulate language programs, coding workflows or collaboration graphs, using execution feedback to assign responsibility for failure [123–126]. Designer-oriented methods train proposal or role policies, retain useful experience, or combine optimizers in a common platform [127–130]. The distinction is the learned output: one workflow, a distribution over workflows, or reusable optimization infrastructure.

Population search and communication-aware design preserve alternatives or adapt team topology [131–134]. Model heterogeneity, population size and communication density must remain separate budget variables. Instance-specific adaptation can instead revise roles, scaffolds or tools during a problem; such changes have different retention horizons from reusable domain tool collections [135–138]. Bounded condition–action rules with temporally ordered validation offer a more restricted, inspectable alternative in domain settings [139]. Expressive search spaces permit broader changes, while restricted ones make interfaces and regression tests easier to specify.

Runtime adaptation needs a separate persistence account. ToolSelf permits configuration changes during execution and also studies a separate training stage [140]. Orchestra-o1 combines multimodal subtask assignment with training of the orchestrator through DA-GRPO [141]. A topology selected for one query and a trained policy for selecting future topologies have different return paths. A clear experiment tests the former on task outcomes and the latter on new design problems, with communication, model calls and tool execution included in cost.

5.3. Algorithms, Search Operators and Reusable Design Knowledge

Algorithm design ranges from architecture search to executable learning and optimization procedures. AutoML-Zero searches learning algorithms, and AlphaEvolve uses model-guided program proposals, execution and an archive to search for useful programs [40,142]. A fixed search engine can produce a substantially better algorithm. That result becomes evidence about recursive improvement only when the discovery also changes later search or the model that conducts it.

Textual optimization offers a related space for studying proposal quality. APE, automatic prompt optimization, EvoPrompt and OPRO use different forms of generation, feedback and selection; TextGrad and GEPA use textual or execution-grounded diagnoses to guide revisions [82,86,87,143–145]. These methods connect design to repair: the same optimizer can construct a new instruction or correct an incumbent's failure. Their common limitation is that a high validation score can reflect specialization to the search population. Held-out artifacts, tasks and starting programs test different kinds of reuse.

Search control and reusable operators.

Prompt search varies the source of feedback: error traces, synthetic comparisons and environment rollouts offer different independence and cost [146–149]. History-sensitive textual optimization adds

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

credit assignment and control of revision magnitude [150–152]. These are improvements to a search design even when its outer implementation remains fixed.

Executable search introduces archives, diversity, uncertainty and crossover as mechanisms for retaining useful alternatives [153–156]. Returning search traces to proposal training differs from evolving models through weight-space operators [157,158]. Reusable transformations, typed skills and tool libraries narrow future search by preserving executable design knowledge [159–162]. Their transfer should be tested on new combinations, not inferred from library size.

Budget allocation, model routing and proxy evaluation alter which candidates receive expensive checks [163–166]. A proxy can reduce search cost while misranking the eventual deployment candidates. Independent execution and separate reporting of proposal, verification and deployment expenditure therefore remain necessary for claims about both efficiency and discovery quality.

Highlight. Design quality and designer quality need separate tests

A discovered architecture, tool or workflow is a useful artifact. To show that its discovery improves the designer, retain the relevant change and measure the next search on new starting systems at a matched budget. The learning and verification costs of closing that return path count too.

5.4. Automated Research as Infrastructure Design

Research agents extend design to proposing hypotheses, implementing changes, conducting experiments and interpreting results. The AI Scientist reports span related stages of this workflow and do not constitute independent replications [167–169]. ASI-Evolve connects experimental analysis and accumulated priors to changes in data pipelines, architectures and learning algorithms [170]. The important inherited object is an executable finding together with its evidence and conditions of use, rather than the research report alone.

Research mechanisms.

Hypothesis-generation systems use knowledge representations, critique, tournaments or physical experiments to refine proposals [171–173]. Persistent research agents additionally retain ideation records, tools, strategies or scientific skills [174–177]. The central distinction is whether feedback selects a current hypothesis or improves how the next study is conducted.

For machine-learning research, experiment loops search algorithms and data pipelines while preserving evaluations and retrospective lessons [178–181]. Execution permissions, budgets, artifact management and explicit evidence records support these loops by making an experiment reproducible and its intended mechanism inspectable [182,183]. A run that completes successfully can still fail to implement the proposed idea, so operational success and scientific support need separate checks.

Training proposal generators and comparing research-search strategies also expose a tradeoff among quality, novelty and diversity [184,185]. A productive loop should retain negative results and uncertainty alongside successful recipes. Otherwise, later experiments inherit a selectively favorable record rather than reliable knowledge about where a design works.

Research assistance also includes task-local loops. AREX refines evidence and constraints through a learned context-update tool; OMNIFLOW uses a frozen multimodal model with physical consistency checks and rollback [85,186]. These mechanisms can improve an answer or experiment without training a successor researcher. SGI-Bench's novelty-based test-time learning provides a different return path from retrieved literature to future proposals, but novelty alone does not establish scientific validity [187].

An infrastructure-design claim is strongest when a proposed algorithm survives independent reimplementation or rerunning. A recursive claim adds a further step: the resulting system participates in subsequent discovery, and does so more effectively. Human review, physical measurements and

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

negative results remain legitimate inputs to that process. Their roles and costs should be reported, particularly when an inexpensive exploratory run precedes a costly test of a trained successor.

6. Repair and Optimization of Existing Systems

Repair begins with an incumbent and a diagnosed shortfall. Optimization can also target accuracy, efficiency or robustness without an explicit software bug. In both cases, Equation 3 asks for useful change subject to resource and regression constraints. We organize this literature by the state that receives the repair: parameters, experience stores, programs, feedback components and the improver itself.

6.1. Parameter Updates from Generated Experience

Self-training converts outputs selected under a feedback rule into new parameters. The central decisions concern generation, admission and the starting checkpoint for retraining. STaR iteratively improves a rationale pool while restarting each round's fine-tuning from the original pretrained model; its continuity resides partly in the evolving training data [1]. Other methods use generated reasoning, offline experience reuse or preference training with human anchors [73,188,189]. Thus, an iteration label does not specify either the inherited state or the source of supervision.

Executable and formal feedback makes errors more observable in code and mathematics. DeepSeek-Prover, ReST-MCTS* and DeepSeek-R1 illustrate different ways of combining generation, verification, search and training [190–192]. These results support learning within the reported regimes; they do not show that the outer training algorithm improves itself. Evidence of stronger reasoning must also distinguish better sampling or selection from gains in the trained policy under a fixed evaluation budget.

Self-training mechanisms.

The literature groups naturally by the bottleneck it addresses. Instruction and reasoning synthesis broadens available supervision [193–195]. Verification, reflection and retry mechanisms refine which generated traces are admitted [196–198]. Mathematical and verifier-supported pipelines use more structured correctness signals [199–202]. These clusters concern different parts of one loop; better filtering need not supply missing task diversity.

Diversity-aware selection and adaptive training address that latter problem [203–205], while noise-aware preference and self-play optimization modify the learning signal [206,207]. Synthetic pretraining broadens the scale at which generated data enters the pipeline [208,209]. Across these regimes, self-training can also degrade [210]. Comparisons should therefore expose real-data retention, admission criteria and teacher dependence rather than treat all generated-data loops as one mechanism with one expected outcome.

The same return path appears beyond language reasoning. BeamVQ combines a vector-quantized code bank, beam search and physics-based selection, then uses selected forecasts in self-training [211]. Search improves a candidate prediction; returning selected candidates to training changes future generation. The physical criterion remains an external anchor. RoboAlign-R1 instead uses a frozen multimodal teacher to refresh a student's reward proxy [212]. In both settings, the loop must retain its verification source in the description and test generalization beyond the conditions used for filtering or reward construction.

6.2. Memory and Skill Maintenance

Memory repair changes what the agent retains and how it retrieves or uses it. Generative Agents and MemGPT support persistent context, while Reflexion, ExpeL and Voyager emphasize reflective experience, extracted lessons and executable skills [3,4,77,213,214]. These forms differ in granularity and reuse: a successful trace preserves circumstances, a lesson abstracts them, and a skill specifies behavior. A useful memory can outlive one task without any change to the language-model weights.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Representation and management.

Memory systems vary along three dimensions. Conversational and experiential stores consolidate facts, temporal relations, beliefs or multiple memory types [215–218]. Graph, hierarchical and distributed structures change what relationships retrieval can recover [219–223]. Experience abstraction instead extracts workflows, reasoning strategies and subtask-specific lessons [224–228]. The right representation depends on whether the task needs exact recall, temporal consistency or procedural transfer.

Management methods train or revise admission, retrieval, augmentation and consolidation [229–233]. Latent pools, soft prompts and rehearsal-based consolidation blur the boundary between external storage and parameter learning [234–238]. The report must identify both the information that persists and the operations or parameters that change.

Cross-system memory records tool behavior, shares experience or preserves artifact lineage for later executors [239–243]. Provenance and applicability become central: a retrieved strategy is useful only if its assumptions still hold. Retrieval accuracy alone does not test whether memory helps the agent diagnose failures or produce better updates.

Skill creation and maintenance.

Skill construction converts interaction into APIs, tools and composable behaviors [244–247]. Lifecycle methods add validation, bounded edits and trace-based revision [248–251]. Learning the manager or generator further makes later skill quality part of the training objective [252–255]. These are distinct capabilities: creating a plausible skill, maintaining a useful library and improving its maintenance procedure.

Skills can guide new software tasks, supervise policy updates or generate robot-training curricula [256–258]. They also interact: complementary and conflicting tools, retirement under noisy judgments, acquisition constraints and transfer granularity all affect library utility [259–262]. Failure-driven code improvement with light human verification gives a further reminder to report outside intervention explicitly [263]. The pertinent endpoint is successful reuse under stated conditions, together with regressions and maintenance cost.

Training and deployment boundaries.

Web-agent training uses generated trajectories, search and critique, or failure-driven curricula [264–266]. World-model experience and environment assessment add further sources of feedback for adaptation [267,268]. Their value depends on the observation interface, action space and validity of simulated transitions. Agent learning frameworks place self-generated experience and feedback into training or evolving execution procedures [269–271]. Coaching, reflection and experience reuse provide complementary paths to later adaptation [272–276]. Operating-system agents, tool-building systems and symbolic agent learning extend the editable objects beyond model weights [277–279]. Test-time self-improvement further varies when an update occurs [280]. A common comparison records which state crosses the next task boundary and whether that state changes future update production.

The critical intervention separates stored content from the mechanism that manages it. Removing both at once cannot establish which produces the gain. Mem2Evolve and dynamic memory-graph approaches make the interaction between experience and its organization especially relevant [78,281]. A practical comparison freezes one of admission, retrieval, consolidation or content while varying the others. Tests should include shifted tasks and obsolete memories, because a library can grow while becoming less useful. An improved memory policy becomes an improver component when it determines how future experience produces future updates.

6.3. Prompt, Tool and Code Repair

Execution traces often localize a failure more clearly than a final reward. Prompt and program optimizers translate that evidence into revisions of instructions, tools or agent code. The design

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

methods in Section 5.3 supply many of these operators; their use for repair adds a requirement to preserve previously correct behavior. Tests that trigger an edit are development evidence, while regression tests and sealed tasks probe the scope of the repair.

A local correction is valuable without being persistent self-improvement. Self-Refine and tool-grounded correction operate on current outputs [67,68]; SICA instead targets agent code used by later executions [282]. The distinction is what the next task inherits. Program repair also needs a distinction between a patch to a target repository and a patch to the agent that writes repository patches. Only the latter directly changes the update-producing machinery; even then, a later controlled test is needed to establish improved repair ability.

6.4. Maintaining Feedback and Evaluators

A feedback provider can improve, drift or be exploited as the actor changes. Self-Rewarding returns model preferences to training, while Meta-Rewarding also targets judging behavior; SaFeR-VLM studies self-feedback in a multimodal setting [2,79,283]. Shared actor–judge weights complicate attribution: a single training step can change both the answers and their assessment. Track actor outcomes, judge calibration and the downstream usefulness of feedback as separate measurements.

Feedback mechanisms.

Self-judgment, latent reasoning, trajectory consistency and endogenous reward estimation extract different proxies from model behavior [284–287]. Semantic agreement, reconstruction and peer prediction introduce other assumptions and auxiliary components [288–290]. Agreement can indicate consistency without establishing correctness; the reference used to calibrate each proxy must remain visible.

Verifier training and selective revision connect reference-supported error detection, step critiques and tool feedback [291–295]. Trajectory credit assignment instead estimates which actions contribute to successful outcomes [296–298]. These tasks require different calibration: detecting one wrong step, ranking candidate solutions and producing a stable training reward are related but separate objectives.

Explicit rules, meta-verification, research rubrics and structured attribution make feedback more inspectable [299–302]. Co-evolving tests, transition-based value estimates and critiques rewarded through their consequences evaluate feedback by what it helps improve [303–305]. This downstream test is particularly relevant to RSI: an explanation's readability or a judge's agreement with itself cannot substitute for evidence that its feedback produces a better successor.

Feedback sources are not interchangeable. Reference answers, environment outcomes, peer agreement and verbal critique reveal different information. A judge that ranks a fixed candidate pool well may still give unstable signals during repeated training. Likewise, refining an internal reward is an allowed update, but improvement under the revised reward must be kept separate from improvement under the original independent task criterion.

6.5. Repairing and Revising the Improver

Three clusters expose increasingly direct return paths without forming a universal capability ladder. *Operator adaptation* changes how edits are proposed: Promptbreeder evolves mutation prompts together with task prompts, and MetaRSI-v1 studies scheduling across data, harness and model operators [84,306]. *Executable self-modification* retains revised agent or improver code. *Recursive conditioning* retains material that a fixed meta-operation uses in later rounds. The comparisons below concern their different causal structures.

Executable self-modification.

STOP evaluates a program improver through the downstream utility of the programs it improves, and lets the procedure revise its own implementation [46]. DGM lets a coding agent revise its agent code while keeping the foundation model frozen; an archive and parent selection preserve alternatives to one lineage [5]. Hyperagents places task-agent and meta-agent code in one editable program; its

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

principal DGM-H experiments retain handcrafted parent selection, with editable selection studied as an extension [6]. All three expose an implemented return path, but its scope differs: the program improver, shared agent logic or dedicated meta-agent can change, while other parts of the search remain external.

The value of a return path is a separate question. DGM's fixed-modifier and archive ablations test specific contributions, but an archive-best score need not describe every descendant. Hyperagents measures successor gains and transfers an evolved system to new domains; transferring both task-agent and meta-agent code leaves starting-state attribution partly coupled. STOP's transfer probes similarly concern selected evolved improvers. Section 7.6 preserves these distinctions when reading the results.

Recursive context under a fixed operator.

Meta^n applies a fixed operation Ω to earlier code, traces and context, producing new strategy layers and callable helpers [7]. Recursion lies in what the operator processes, rather than edits to its implementation. More depth, more candidate chains and more evaluation are separate resource changes. Context and library ablations can test the inherited material; matched-budget comparisons test whether its contribution survives control of extra search. Reported layer interference also shows why deeper recursion need not improve every chain.

Selection, shared state and changing objectives.

Improver methods differ in what makes a candidate worth retaining. Self-editing agents and descendant-aware selection expose the difference between current task quality and the quality of future offspring [307,308]. Memory-architecture search and guided mutation with evolution memory instead alter how experience informs future proposals [309,310]. Evaluating descendants can improve selection but also consumes the very budget against which improvement efficiency is measured.

Joint scaffold–weight updates, architecture-design agents and coupled populations of task agents and optimizers create additional attribution paths [311–313]. An architecture's inner training gain is not automatically a gain in its designer; a changing task-agent population also changes the distribution on which optimizer quality is assessed. Held-out starting agents help test whether the advantage survives that change.

Finally, evolving utilities and aggregated objective proposals let the criterion change [314,315]. Record progress under both the revised objective and a stable external reference. Otherwise, a change in what the loop values becomes indistinguishable from improved ability to achieve a fixed goal.

Highlight. A repair should improve later behavior without hiding its cost

A successful edit, a useful archive and a stronger modifier are different outcomes. Follow the retained object into the next round, test preserved capabilities, and count failed proposals and recovery. These checks turn self-modification from a structural property into an assessable claim.

6.6. Comparing Complete Configurations

Table 5 compares sixteen configurations by their inherited state, feedback and reported horizon. Its rows span both design and repair. The unit is the configuration, rather than a paper title or backbone alone: the same model can support local correction, persistent memory and code revision under different access rules. These descriptors make the later protocol comparison possible without assigning a single maturity score.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 5. Representative methods. Checked configurations; icons denote model families/providers. Run lengths retain source units, not comparable recursion depths. Code: author-provided link; – means not recorded, not unavailable.

Method / model	Retained state	Feedback	Update process	Reported run	Code
STaR [1] GPT-J 6B	Rationales; weights	Given answers; correctness	Fixed SFT recipe	16 arithmetic rounds (no rationalization)	–
Self-Rewarding [2] Llama 2 70B	Preferences; actor/judge weights	Self-ranked responses	Shared actor/judge; fixed DPO	Seed + 2 DPO rounds	–
Meta-Rewarding [79] Llama 3 8B Instruct	Preferences; shared actor/judge weights	Judgments; meta-judgments	Learned judge; fixed DPO	4 DPO rounds; judge pairs in 1-2	–
Absolute Zero [75] Qwen2.5 7B / Coder 7B	Task buffers; proposer/solver weights	Execution; validity; learnability	Learned proposer; fixed RL	500 RL steps	link
Reflexion [3] GPT-3 / GPT-4 (task-dependent)	Episodic reflections	Outcomes; execution; self-evaluation	Fixed reflection procedure	Task retries; no weight updates	link
Voyager [4] GPT-4; GPT-3.5 retrieval	Skill code; curriculum history	Environment; errors; self-checks	Fixed curriculum and verification	Exploration: up to 160 prompting iterations	–
ExpeL [77] GPT-4 insights; GPT-3.5 actions	Insights; successful traces	Training successes / failures	Fixed insight extraction	Collect; distill; retrieve on test	link
ADAS [80] GPT-4 search; GPT-3.5 agents	Programs; prompts; archive	Validation scores	Fixed meta-agent	25 ARC / 30 other search iterations	link
GEPA [82] Qwen3 8B / GPT-4.1 Mini targets	Prompts; candidate ancestry	Traces; metrics; text	Fixed reflective optimizer	Task-specific rollout caps	link
AlphaEvolve [142] Gemini 2.0 Flash + Pro	Algorithm code; program database	Executable evaluators	Fixed search pipeline	Asynchronous evolution; task-specific budgets	–

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 5. *Cont.*

Method / model	Retained state	Feedback	Update process	Reported run	Code
Promptbreeder [84] PaLM 2-L	Task/mutation prompts; exemplars	Training-batch accuracy	Editable mutation prompts; fixed topology	Population 50; usually 20-30 generations	–
STOP [46] GPT-4; GPT-3.5 / Mixtral comparisons	Improver program	Downstream meta-utility	Editable improver; frozen LM	Transfer study: 4 update rounds	link
SICA [282] Sonnet 3.5 v2; o3-mini subagent	Agent code; tools; control flow	Quality; time; cost	Shared coding/-modifying agent	Coding study: iterations 0-15	link
DGM [5] Sonnet 3.5 editor; Sonnet / o3-mini tasks	Agent code; archive	Staged benchmark scores	Shared agent; fixed outer driver	80 iterations; 1 candidate each	link
Hyperagents [6] Sonnet 3.5 / 4.5 editors; task-specific models	Task/meta-agent code; helper state	Domain scores; archive	Editable meta-agent; fixed parent selection	80 coding / 100 joint iterations; imp@50 probe	link
Meta^{n} [7] Gemma 4 31B-IT / GPT-5.2	Context layers; code libraries; archive	Scores; execution traces	Fixed Ω; evolving context	Nested depth + evolution; task-specific caps	link

7. Benchmarks and Evaluation

An RSI experiment tests a sequence of proposals, acceptance decisions and retained changes. A final score summarizes only one outcome. The protocol in Figure 7 separates the development loop from independent audit, then compares task gain, retained behavior and successor production. The rules below are our synthesis of an experimental design, not a claim that every reviewed method implements them.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

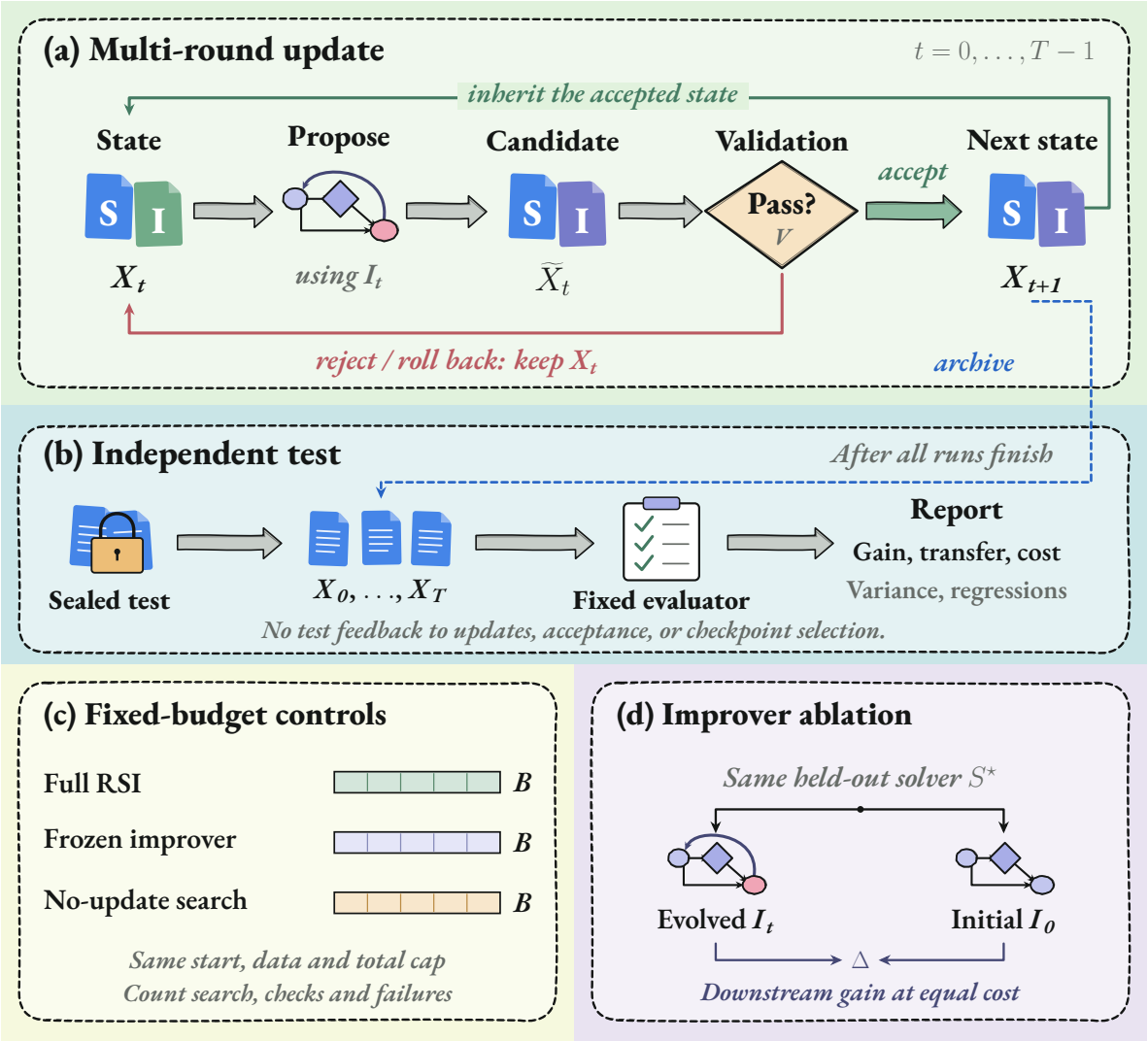


Figure 7. A proposed evaluation protocol for RSI. (a) At round t, state $X_t = (S_t, I_t)$ produces a candidate. A predeclared validation rule on V accepts it or restores the previous state; the retained state drives the next round. (b) Once all runs finish, an independent evaluator audits archived checkpoints chosen by a schedule fixed before search, using sealed test tasks and labels. Test feedback cannot guide updates, acceptance, or checkpoint selection. Report task gains, transfer, cumulative cost, uncertainty across repeated runs, and regressions, rather than assuming monotonic progress. (c) Compare full RSI with a frozen improver and a no-update search baseline under the same total budget cap B, initialization, and data access. Charge proposal generation, training, validation, inference, and rejected attempts; match audit calls and inference limits, and report realized costs. (d) To isolate improver quality, apply I_t and I_0 to the same held-out solver states and compare downstream gains on new tasks at equal cost. These roles may share an implementation; the paired intervention applies where they can be separated. The diagram synthesizes an experimental design, not measured results or a universal standard. It builds on validation/test separation in ADAS and the fixed-improver control in DGM [5,80].

7.1. Match the Benchmark to the Engineering Goal

Interactive benchmarks expose task outcomes: AgentBench, GAIA and SWE-bench cover agent interaction, assistance and repository-level issue resolution [316–318]. Infrastructure-design benchmarks instead need executable artifacts and their downstream utility. MLAgentBench, MLE-bench and ScienceAgentBench probe machine-learning engineering and scientific experimentation [319–321]. RewardBench and ProcessBench test evaluators and reasoning-error detection [322,323]. A good score on one of these endpoints does not specify the improvement history that produces it.

Scientific and multimodal tasks broaden what an update must preserve. SGI-Bench separates research, idea generation and experimental reasoning; VideoGAIA tests multi-turn video understanding with tools [187,324]. Longitudinal environments make the history visible: SEA-Eval records success

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

and cost along task streams, while SEAGym separates training, frozen update-validation, held-out tests, replay and cost records [325,326]. The benchmark supplies tasks and measurement; the researcher still specifies update rounds, retained state and acceptance.

7.1.1. Matching Benchmarks to the Claimed Update

A benchmark's task realism and its suitability for measuring improvement are different properties. WebArena and Windows Agent Arena provide interactive environments, while τ-bench tests tool use, policy compliance and consistency across repeated interactions [327–329]. These settings can host an improvement experiment, but the experiment must add an explicit sequence of updates and a rule for retained state. Repeated attempts by an unchanged agent measure a different quantity from repeated attempts after a learned update.

Software engineering also needs strong fixed-design comparisons. SWE-agent shows the effect of the agent–computer interface, and Agentless establishes a simpler localization, repair and validation baseline [330,331]. SWE-Bench Pro broadens the scope toward longer-horizon repository work; benchmark mutation studies examine how formal issue descriptions differ from realistic user requests [332,333]. These comparisons help prevent an interface advantage or an easier prompt distribution from being attributed to recursive improvement.

Training environments and test environments play different roles. SWE-Gym and R2E-Gym supply executable software tasks and support work on agent training and verification. EnvScaler synthesizes tool-interaction sandboxes [334–336]. Their usefulness for producing experience does not make the same tasks independent tests of a trained successor. Repository overlap, task-generation seeds and shared verification code should therefore be recorded when such infrastructure appears on both sides of an evaluation.

Persistent state requires longitudinal tests. TRACE evaluates continual learning across tasks; LifelongAgentBench uses interdependent agent tasks; and AutoEnv studies learning across heterogeneous environments [337–339]. GateMem adds governance requirements to shared memory, including state updates, access boundaries and forgetting [340]. A protocol can improve recall while regressing on other retained requirements, so longitudinal utility should include both newly acquired behavior and obligations carried from earlier interactions.

Tool and algorithm creation can be probed directly. Tool-Genesis separates interface compliance, functional correctness and downstream utility. The Meta-Agent Challenge tests autonomous construction of agent artifacts, while AI4AI-Bench targets changes to training algorithms and independently reruns the resulting code [341–343]. These are useful instruments for measuring capabilities required by RSI. A successful artifact remains one link in the chain; testing an improved procedure on the next construction round supplies a further link.

Dynamic benchmarks address saturation but complicate comparability. Benchmark Self-Evolving transforms existing instances, TRACE's validate-by-reproduce framework checks generated task trajectories, and BenchEvolver grounds harder coding tasks in evolved solutions [344–346]. An evolving task stream should be paired with a stable reference distribution. Otherwise, a lower score can reflect harder tasks and a higher score can reflect a generator's changing preferences, rather than a change in the system being assessed.

Finally, resource controls must follow the operation being measured. Compute-optimal test-time scaling studies allocation across inference strategies, while SWE-Effi accounts for the costs of both successes and expensive failures [347,348]. For an RSI comparison, this means charging unsuccessful candidate updates, verifier calls, environment construction and retraining to the same declared budget. Reporting a final solver's inference cost alone omits the investment that produces it.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 6. Match objectives to tests. Examples supply task endpoints; an RSI study must add update histories and independent audit.

Goal	Example setting	Endpoint	Additional probe
Infrastructure design	MLE-bench; AI4AI-Bench	Rerun artifact quality; cost	New tasks and training runs
System repair	SWE-bench; tool environments	Resolved failures; regressions	Sealed tasks; protected behaviors
Persistent adaptation	SEAGym; continual tasks	Gain; retention; transfer	Reset boundaries; replay; cost
Improver update	Held-out starting solvers	$\Gamma_t(B)$; successor quality	Same states, feedback and budget
Research validity	Reproduction and scientific tasks	Reproducibility; supported claims	Independent checks; negative results

7.2. Separate Development, Selection and Sealed Testing

Development feedback guides proposals or training; validation selects updates; sealed tests measure the resulting states. ADAS illustrates validation-guided search with held-out testing [80]. Repeatedly selecting against a fixed validation set still adapts the search to that set. Aggregate scores also convey information, so withholding individual labels is insufficient if test averages guide future edits.

Before a run, declare checkpoint times, budget milestones and selection rules. After all runs finish, an independent evaluator audits the scheduled checkpoints without returning scores to proposal, acceptance, stopping or checkpoint selection. Report the final checkpoint and any validation-selected checkpoint under their declared roles. Selecting the best checkpoint after inspecting its test performance compromises the audit. A procedural split also cannot rule out pretraining contamination; task release dates and model versions remain relevant, as release-aware testing such as LiveCodeBench emphasizes [349].

A deployment-oriented gate can formalize acceptance as

$$A_t = \mathbf{1}\Big\{\widehat{\Delta}_{\mathrm{val}} \geq \eta_t, \quad \widehat{r}_{t,j} \leq \epsilon_j \,\forall j, \quad C_{\leq t+1} \leq B\Big\}, \qquad X_{t+1} = \begin{cases} \widetilde{X}_{t+1}, & A_t = 1, \\ X_t, & A_t = 0. \end{cases} \tag{9}$$

Here $\widehat{\Delta}_{\mathrm{val}}$ estimates the candidate's validation gain, $\widehat{r}_{t,j}$ estimates loss on protected group j, η_t is a predeclared gain threshold and ϵ_j a regression tolerance. These estimates can be noisy; the rule does not guarantee an independent test gain. Uncertainty-aware thresholds, repeated checks and recovery records make its behavior assessable. An exploratory archive can retain weaker candidates as future parents, so archive admission, parent selection and deployment acceptance must be reported separately.

Staged screening can save resources but changes score coverage. DGM first checks candidates on smaller subsets [5]; a candidate rejected early and one evaluated broadly have different observed evidence. Unevaluated tasks should not silently become measured failures. Preserve the screening history and audit selected artifacts on a common population.

7.3. Account for Full Development and Deployment Cost

An update round can mean a prompt edit, a population search or a training run. Use a non-overlapping cost ledger:

$$C_{\leq T} = \sum_{t=0}^{T-1} \left(c_t^{\mathrm{proposal}} + c_t^{\mathrm{train}} + c_t^{\mathrm{validation}} + c_t^{\mathrm{execution}}\right). \tag{10}$$

Include retries, rejected candidates and environmental checks; proposal inference belongs to the proposal term. Record independent audit cost separately under matched test-time limits. When

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

GPU time, API expenditure and physical experiments lack a defensible common conversion, report a resource vector rather than add unlike units.

A shared budget cap does not ensure equal realized spending. Report quality at common cost milestones, together with per-task deployment cost. GEPA's rollout budget controls one scarce resource, but does not equalize all tokens used to propose edits [82]. Similarly, matching tokens can leave model prices and tool latency unmatched. The relevant idea is skill-acquisition efficiency as well as task competence [350]; RLVR analyses using large-sample pass@k also show why inference limits affect the interpretation of learned gains [351].

For a deployment horizon of N future tasks, report $C_{\text{total}}(N) = C_{\text{development}} + N\bar{c}_{\text{task}}$ alongside quality, where $\bar{c}_{\text{task}}$ is the measured average serving cost. Reuse can amortize an expensive update, but only on tasks to which it applies. A cheaper search that produces an expensive agent need not be the more economical system over that horizon.

7.4. Isolate the Contribution of the Improver

Compare the full loop with a *frozen-improver* control that permits solver updates using I_0, and a *no-persistent-update* control that permits inference-time search within the same declared limits. DGM's fixed-modifier baseline supplies a precedent for the first comparison [5]. The controls separate extra search, persistent adaptation and adaptation of the update procedure.

The paired probe in Equation 8 applies I_t and I_0 to copies of the same held-out starting states. It tests the successors they produce at matched cost, with the probe isolated from the primary trajectory. STOP and Hyperagents motivate such downstream tests [6,46]. A swap is interpretable only if interfaces and dependencies remain compatible. Otherwise, failure may reflect a missing memory schema or broken tool interface rather than worse improvement ability.

When roles share code or weights, report the coupling and use the narrowest valid intervention. Removing both memory and its manager cannot identify the contribution of either alone. Factorial comparisons can separate their interaction when feasible. Baselines also need a functional opportunity to improve: repairing an invalid output format is useful, but differs from improving reasoning in an already functioning agent. Record these failure categories rather than collapse them into one score.

7.5. Measure Trajectories, Retention and Transfer

For each scheduled checkpoint, report quality versus cumulative cost, acceptance rates, new successes and regressions on earlier groups. An archive can preserve a strong candidate while its latest branch deteriorates; archive-best, deployed and final-state curves therefore answer different questions. Replay and transfer tests instantiate Equation 6; SEAGym's replay view makes capability turnover visible [326].

Repeated runs should vary update generation and selection, not only final inference. State whether uncertainty summarizes tasks, search seeds, environments or task orders. A conservative gate that rejects almost every change can be stable but unproductive; an aggressive gate can gain quickly and incur costly recovery. Track accepted gains, rejected useful candidates, recovery cost and retained constraints together.

Transfer has several axes: tasks, backbones, tool interfaces and feedback sources. Varying them separately helps locate the inherited advantage. Transferring a solver tests a different object from transferring an improver. Figure 8 accordingly separates update targets from protocol evidence; its selected sample supports neither a prevalence estimate nor a common ranking of sustained RSI.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

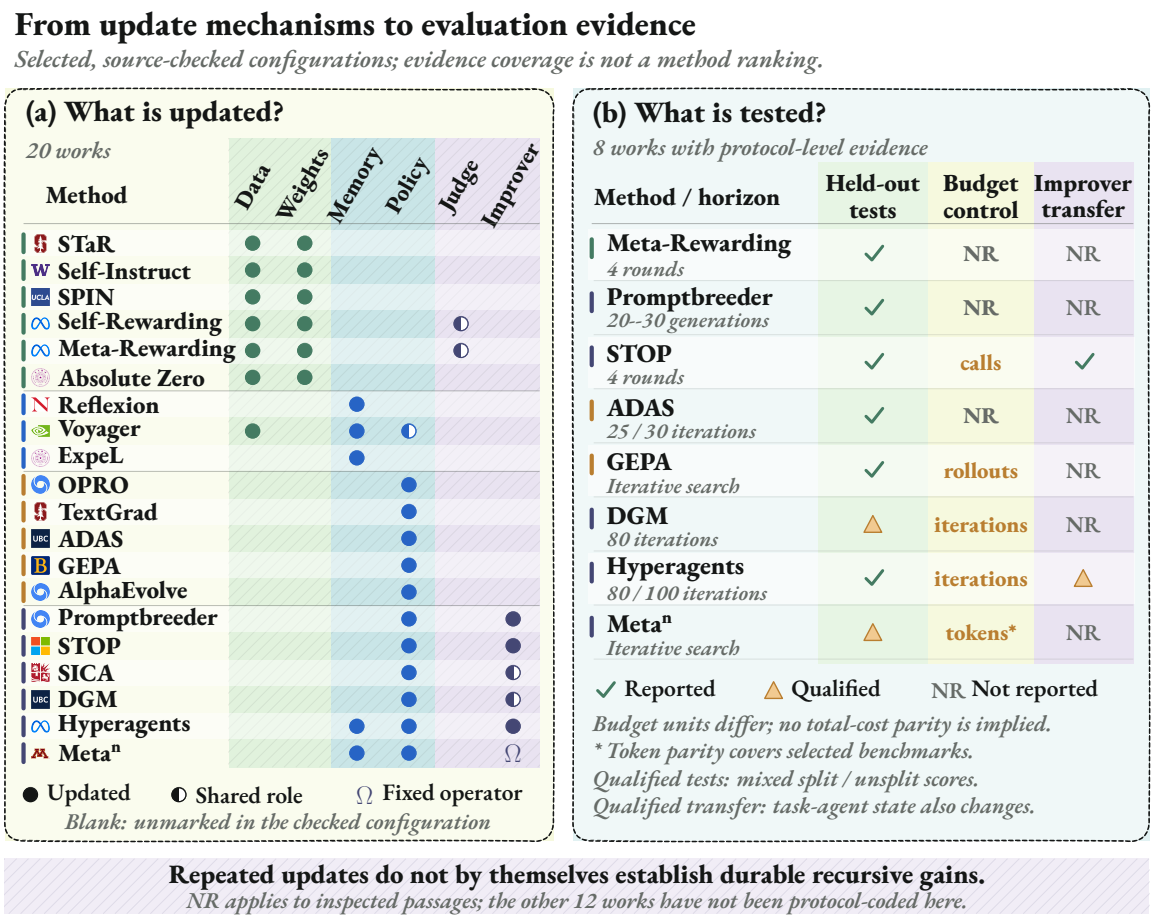


Figure 8. Update mechanisms and the evidence used to evaluate them. (a) Six update objects across twenty selected works, coded from abstracts and relevant method passages [1–7,46,72,75,77,79,80,82,84,86,87,142,189,282]. Filled circles denote updates; half-filled circles denote changed artifacts serving shared roles. Ω is Meta^n's fixed operator with growing context. Data includes curricula, memory includes reusable skills, and policy includes prompts and code; neither an archive alone nor use of a judge implies a learned memory or an updated judge. Logos identify one selected affiliation. (b) Targeted protocol coding for the eight works in Table 7. Horizons use each paper's own units. Checks denote reported tests; triangles qualify mixed held-out/unsplit scores or transfer that also changes task-agent state. Budget labels name the resource unit for specified comparisons, not equal total expenditure; token parity applies to selected settings. NR means the specified protocol is not reported in inspected passages. The other twelve works are not protocol-coded here. Neither blank cells nor NR establish universal absence, and the samples support neither prevalence estimates nor a ranking of sustained recursive improvement.

7.6. What the Current Evidence Supports

Table 7 compares eight protocols. The evidence clusters into artifact gains, transfer of update-producing procedures, and measurement limits. ADAS and GEPA test discovered designs or prompts with fixed outer optimizers; STOP tests an evolved improver; Meta^n tests recursive context and helpers [7,46,80,82]. They do not run one common RSI benchmark.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Table 7. Evaluation evidence. Selected reported protocols; budgets retain source units. The final column summarizes evidence and its limits. Figure 7 presents our proposed protocol.

Work	Test boundary	Budget / controls	Selection	Evidence / limits
Meta-Rewarding [79]	AlpacaEval 2; Arena-Hard; MT-Bench; separate judge prompts	4 rounds; 5,000 prompts × 7 responses; matched length control	Filtered pairs; DPO; no rollback	Actor/judge gains; GPT-4 proxies; later human-agreement gains fade
Promptbreeder [84]	Train-batch fitness; test splits (some random halves)	50 individuals; usually 20-30 generations; operator ablations	Tournaments; task/mutation prompt co-evolution	Prompt co-evolution; fixed topology; cost parity unestablished
STOP [46]	50 unseen LPN cases; 5 runs; 5 transfer tasks	LM/utility-call caps; seed/greedy baselines; 4-round transfer improver	Meta-utility search; nonmonotonic rounds	Improver transfer; costly utility; weaker-model regressions
ADAS [80]	ARC: 20 validation / 60 test tasks; 5 repeats	25 ARC / 30 other iterations; shared baseline framework	Validation-guided search; agent archive	Solver transfer; fixed meta-agent; end-to-end cost parity unestablished
GEPA [82]	Train/validation/test; restricted validation-content access	Rollout counts; optimizer/GRPO controls; sampling/merge ablations	Minibatch acceptance; best validation candidate	Prompt transfer; fixed optimizer; rollouts ≠ compute parity
DGM [5]	Staged search; cross-benchmark/language transfer; full scores include search tasks	80 iterations; frozen-modifier/no-archive controls; separate cost reports	Archive sampling; stepping stones; no strict rollback	Self-modification; solver transfer; shared roles; fixed outer driver
Hyperagents [6]	Held-out domain tasks; cross-domain imp@50	Frozen-meta/no-archive controls; 50-candidate probe; separate token estimates	Archive sampling; fixed parent selection	Meta-level transfer; evolved task-agent state not matched
Meta^n [7]	CO-Bench, S2D, LawBench held out; other benchmarks include unsplit scores	Selected token-parity tests; depth/context/library ablations; usually 3 seeds	Archive/chain best; patience; optional consolidation guard	Context gains; fixed Ω; parity is configuration-specific

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

Artifact gains depend on the evaluated population.

DGM reports gains from 20.0% to 50.0% on its 200-task SWE-bench Verified subset after 80 iterations. Its Polyglot scores increase from 14.0% to 38.0% on a 50-task subset and from 14.2% to 30.7% on the full benchmark. These assessments include search tasks and are not wholly independent holdouts [5]. These figures describe different populations. Similarly, an archive's per-task best score need not describe any deployable single agent. On CO-Bench with Gemma 4, Meta^n reports 0.851 ± 0.014 for archive-best, 0.782 ± 0.016 for its best single chain, and 0.814 ± 0.022 for OpenEvolve (standard deviations over three seeds) [7]. The comparator ordering changes with the selected object; these values alone do not establish statistical significance.

Bounded evidence exists for improved successor production.

STOP tests 50 unseen learning-parity-with-noise instances over five runs and transfers a selected evolved improver to five additional utility settings. Its tested GPT-4 configuration improves early, whereas tested GPT-3.5 and Mixtral configurations can regress [46]. Hyperagents transfers evolved task-agent and meta-agent code to math grading; with the transferred meta-agent frozen for 50 new iterations, the validation-selected agents achieve a reported test score of 0.630 (confidence interval 0.540–0.630), from an initial zero score [6]. This supports useful transferred generation ability under that setup. It does not isolate meta-agent quality from the transferred starting agent, and the initial grader's invalid format is a material qualification. In the source-domain comparison, DGM-H's gains over DGM-custom are not statistically significant at the paper's stated threshold, although comparisons with its frozen self-improvement control are significant.

Budget and feedback controls narrow the conclusions.

Meta^n adds token-parity tests on two prompt tasks, supporting a contribution beyond extra tokens in those settings; this does not equalize total cost across all experiments [7]. Meta-Rewarding reports an AlpacaEval 2 length-controlled win rate of 39.44% after four iterations, versus 35.49% for its length-controlled Self-Rewarding baseline [79]. Its separate judge tests show that early gains in correlation with human rankings do not persist in all later rounds. Automatic actor preference gains and stable evaluator improvement are thus different observations.

These examples show practical progress on bounded tasks and some direct probes of improvement ability. They also show regressions, selection effects, shared-state confounds and uneven budget controls. The evidence does not establish indefinite capability growth, reliable acceleration, or complete independence from externally supplied objectives and feedback.

> **Highlight. The tested claim is narrower than the loop's ambition**
>
> Persistent task gains are widely studied; an improved improver requires a separate successor-production test. Stronger claims about sustained or accelerating RSI require longer, independently audited, cost-controlled trajectories than a favorable final checkpoint supplies.

8. Challenges and Future Directions

Useful feedback loops can also preserve mistakes. Figure 9 connects six constraints to testable questions. The common concern is whether a local gain remains useful as tasks, components and evaluation signals change.

8.1. External Feedback and the Risk of Drift

Self-Refine and studies of intrinsic self-correction reach different outcomes under different models, tasks and feedback access [67,352]. Repeated critique is therefore an operation to evaluate, rather than a generally reliable source of new information. Model-judge biases and shared actor–judge updates can make internal rewards increasingly favorable without a corresponding task gain [79,353]. Evidence

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

graphs can make support and contradiction inspectable within a research task, but their improved belief state still differs from improved learning across tasks [354].

Outside feedback can take several forms: executable tests, fresh observations, trusted reference data, human preferences, expert review or physical experiments. Its purpose is to add information or constraints unavailable from the loop's current proxy alone. No single source is infallible: tests have coverage limits, humans disagree and environments can shift. A useful study varies the source, frequency and cost of external feedback while holding other resources fixed. Track independent quality, acceptance errors and recovery across rounds, including a no-new-feedback condition. This tests when external intervention is helpful, rather than assuming that all RSI requires continuous human supervision or that none requires it.

8.2. Experience Selection and Coverage

Recursive synthetic-data training can degrade under particular replacement regimes, while retaining and accumulating real data changes the outcome in other tested settings [355–357]. These findings identify data composition and selection as experimental variables; they do not establish inevitable collapse for every feedback loop. For agents, old lessons and executable skills can preserve rare successes or stale assumptions [4,77]. Compare replay, selective retention and refresh on both earlier and shifted tasks, with storage, retrieval and maintenance costs. A larger store is useful only if its contents remain applicable.

8.3. Interacting Updates and Recovery

A new prompt can change a retrieval cue; a tool revision can invalidate a skill; training can alter a formerly successful workflow. Layer interference and replay regressions make these dependencies visible [7,326]. Archives preserve alternatives, rollback restores an earlier state and consolidation attempts to retain useful behavior in a new representation. Compare these policies under common budgets and preserve complete dependencies in checkpoints. Recovery time, lost skills and post-recovery quality matter alongside the latest task score. Stability is a measured property over a stated horizon, not a consequence of recursion.

8.4. Evaluation Integrity Under Self-Modification

An editable system can change how a development score is obtained. DGM's objective-hacking example and contamination concerns illustrate distinct routes to misleading gains [5,349]. The broader lifecycle perspective includes risks introduced through data, training and deployed components [27]. An experimental boundary should protect test labels, audit code and accounting from the editable state. Internal judges can evolve while the independent reference remains fixed. Varying the allowed edit scope then tests both performance and preservation of the conditions under which that performance is meaningful.

8.5. Discovery and Verification Compete for Resources

More proposals can consume the budget on weak candidates; excessive early verification can eliminate exploration. Reflective search uses detailed feedback to target edits, but obtaining and interpreting that feedback also costs resources [82]. Compare proposal breadth, verification depth and staged screening at matched total cost. Record useful candidates lost to screening and regressions admitted by cheap proxies. Deployment horizons should be explicit when assessing whether the cost of training or tool construction pays for itself through reuse.

8.6. Transfer and the Limits of Current Demonstrations

Transferring a discovered design and transferring a procedure that discovers better designs are different experiments [6,80]. The paired probe controls starting states within each new domain; separate shifts in task, backbone, interface and reward help identify what transfers. New task generation can

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

sustain learning opportunities, as POET and Absolute Zero motivate, but it also needs independent measures of coverage and value [41,75].

Formal self-modification results depend on their axioms and utility conditions [30]. Finite empirical trajectories cannot prove indefinite progress, and the absence of such evidence does not prove RSI impossible. The immediate research target is narrower: reproducible gains that persist under controlled shifts, at a known cost, while retaining the ability to recognize and recover from harmful updates.

8.7. Failure Mechanisms and Scientific Validity

Optimization, information and behavioral drift.

Campaign-level self-regression, loss of training knowledge and exploration collapse concern different states and timescales [358–360]. Reward bias, overconfidence and ungrounded reflection expose errors in the proxy itself [361–363]. Data admission, task design and verifier quality can be varied independently of the reward assigned to admitted examples [364–366]. A verified search artifact can also fail to teach the unaided model the computation it performs [367]. These mechanisms call for different interventions; grouping them all as collapse hides where the feedback loop fails.

Persistence can preserve vulnerabilities.

Model, memory, tool and workflow updates can carry undesirable behavior across rounds, including alignment drift under particular training regimes [368–371]. Memory manipulation, extraction and poisoning create distinct admission, retrieval and access-control problems; validation defenses address only specified parts of that threat model [372–375]. Executable skills and agent access-control proposals extend the boundary from information storage to permitted action [376,377]. Report tested attacks separately from proposed governance mechanisms, and check retained constraints alongside retained capability.

Scientific claims need evidence beyond a completed report.

Integrity, methodological soundness and replication benchmarks test different failure modes in automated research [378–380]. Case studies of failed autonomous experiments and active reverse engineering further distinguish plausible explanations from interventions that identify a mechanism [381,382]. Claim-level evidence records and judge characterization support auditability [383,384]. Broader critiques of missing practical knowledge and publication bias identify information that a research loop may never receive [385,386]. They motivate retaining negative results, uncertainty and reasons for rejection, rather than treating a favorable paper-review score as proof of scientific discovery.

Highlight. External checks are part of the mechanism

Specify what new information reaches the loop, when it arrives and what it costs. A useful next experiment varies that access and measures progress, regression and recovery together. This makes dependence on tests or human judgment observable instead of hiding it behind a claim of autonomy.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

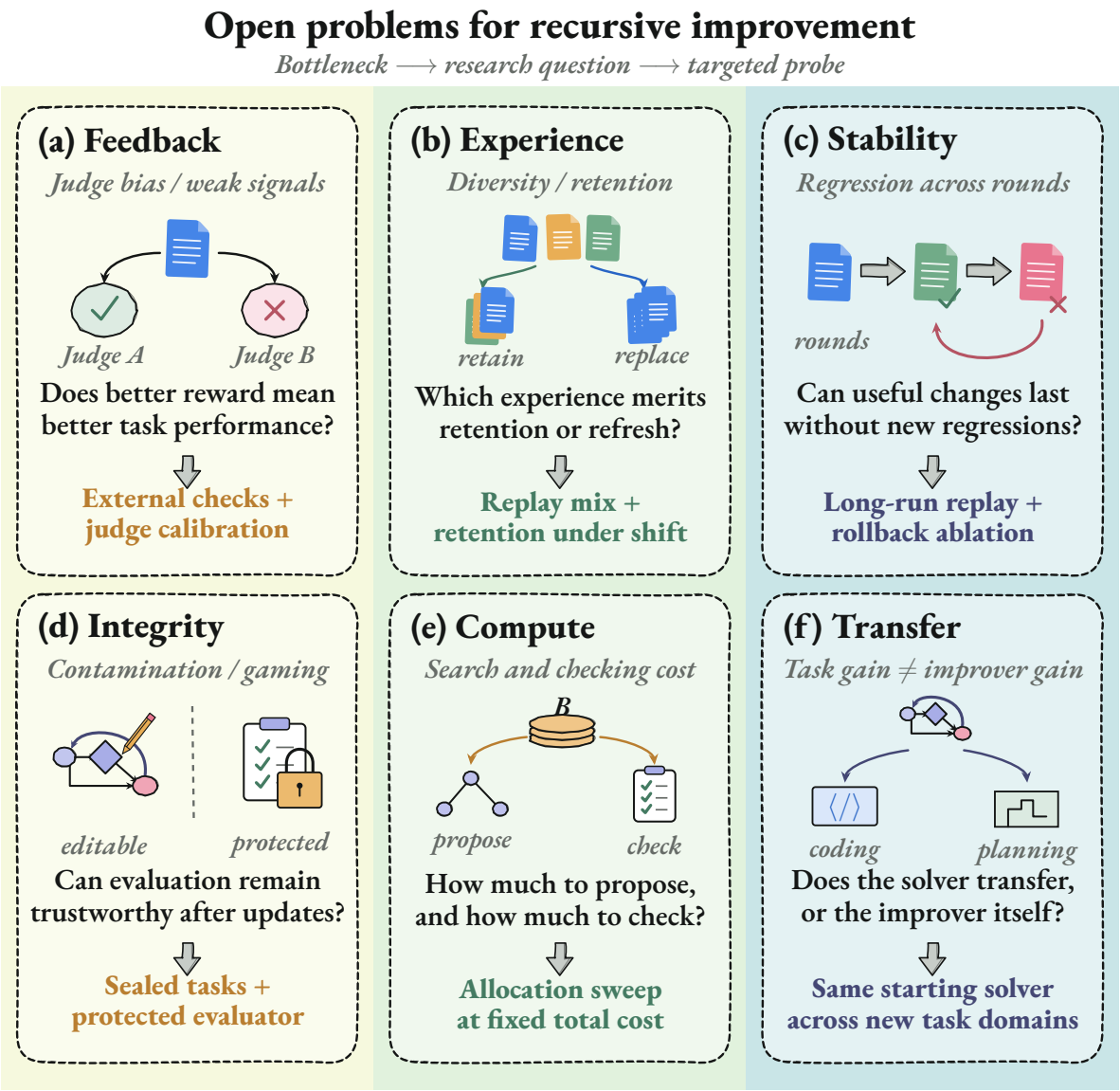


Figure 9. Open problems for recursive improvement. Each panel connects a bottleneck, a research question, and a proposed probe. (a) Judge biases motivate calibrated feedback and external checks [79,353]. (b) Data accumulation and reusable experience motivate testing selection, refresh, and replay; collapse is not inevitable under every data regime [77,357]. (c) Interference across recursive layers motivates long-run retention and rollback studies [7]. (d) Objective hacking and benchmark contamination motivate evaluation integrity as systems change [5,349]. (e) Search and verification costs motivate budget-allocation studies [5,82]. (f) Transferring a discovered solver differs from transferring an improver; paired probes should hold starting solver states and budgets fixed within each new domain [6,80]. These are literature-motivated questions and proposed tests, not measured outcomes, established guarantees, or a predicted sequence of progress. Illustrations are schematic; mechanisms and failure modes may interact.

9. Conclusion

RSI connects a system's present capabilities to the process that produces its next improvement. This survey separates two practical goals—designing AI infrastructure and repairing or optimizing an existing system—and relates both to inherited state, feedback and update production. The connection to AutoML and meta-learning is substantial: automated search, learned update rules and reusable design experience are established foundations. The additional empirical question concerns the value of the return path into future improvement.

Several parts of this agenda already work in bounded settings. Generated experience supports parameter learning, memories and skills support reuse, and automated search produces useful prompts,

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

workflows and programs. DeepSeek-R1 demonstrates verifiable-reward reasoning training; Llama-based self-rewarding studies examine actor and judge updates; GPT-based program improvers and other frozen-backbone agents test useful changes outside model weights [2,46,79,192]. These successes concern different objects and do not establish that the underlying foundation model autonomously improves its entire training stack. Selected improver-transfer experiments provide a further, qualified step toward reusable improvement ability.

The appropriate tests follow the claim. Task benchmarks measure artifact quality; replay and shifted tasks measure retention and transfer; paired successor-production probes measure improver quality. All need an explicit selection boundary, complete resource accounting and uncertainty over improvement runs. Current evidence leaves unrestricted, indefinitely sustained or reliably accelerating RSI unestablished. It also shows that regression is possible and that execution, trusted data and human expertise can remain important sources of correction.

A productive research agenda therefore asks which inherited changes remain useful, under what feedback and budget, and for how long. Closer links between objective design, independent audit and failure recovery make those questions answerable. Progress consists of reproducible improvements to both systems and their update procedures, with their limits visible.

AI Assistance: ChatGPT (OpenAI) assists with literature searches, manuscript drafting, translation, language editing, and figure preparation. The authors retain responsibility for the content, citations, and conclusions.

Conflicts of Interest: The authors' institutional and corporate affiliations are listed on the first page. The authors declare no additional conflicts of interest.

References

1. Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping Reasoning With Reasoning. arXiv preprint arXiv:2203.14465, 2022. URL https://arxiv.org/abs/2203.14465.
2. Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-Rewarding Language Models. arXiv preprint arXiv:2401.10020, 2024. URL https://arxiv.org/abs/2401.10020v3.
3. Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv preprint arXiv:2303.11366, 2023. URL https://arxiv.org/abs/2303.11366v4.
4. Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291, 2023. URL https://arxiv.org/abs/2305.16291v2.
5. Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv preprint arXiv:2505.22954, 2025. URL https://arxiv.org/abs/2505.22954v3.
6. Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. arXiv preprint arXiv:2603.19461, 2026. URL https://arxiv.org/abs/2603.19461v1.
7. Zae Myung Kim, Young-Jun Lee, Seungyeon Jwa, and Dongyeop Kang. Meta^n: Recursive Self-Improvement through Emergent Depth. arXiv preprint arXiv:2608.24735, 2026. URL https://arxiv.org/abs/2608.24735v1.
8. Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, et al. A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. arXiv preprint arXiv:2507.21046, 2025. URL https://arxiv.org/abs/2507.21046v4.
9. Haoyan Yang, Mario Xerri, Solha Park, Huajian Zhang, Yiyang Feng, Sai Akhil Kogilathota, and Jiawei Zhou. Self-Improvement of Large Language Models: A Technical Overview and Future Outlook. arXiv preprint arXiv:2603.25681, 2026. URL https://arxiv.org/abs/2603.25681v3.
10. Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, et al. The Path to Recursive Self-Improving Agents: Foundation, Framework, and Future Directions. Preprints.org, version 1, 2026. URL https://www.preprints.org/manuscript/202608.0051/v1. Preprint.
11. Mingguang Chen, Licheng Wang, and Bo Qu. Recursive Self-Improvement in AI: From Bounded Self-Refinement to Autonomous Research Loops. arXiv preprint arXiv:2607.07663, 2026. URL https://arxiv.org/abs/2607.07663v2.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

12. Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, et al. A Survey on the Memory Mechanism of Large Language Model based Agents. arXiv preprint arXiv:2404.13501, 2024. URL https://arxiv.org/abs/2404.13501.
13. Kiran Ramnath, Kang Zhou, Sheng Guan, Soumya Smruti Mishra, Xuan Qi, Zhengyuan Shen, et al. A Systematic Survey of Automatic Prompt Optimization Techniques. arXiv preprint arXiv:2502.16923, 2025. URL https://arxiv.org/abs/2502.16923.
14. Runyang You, Hongru Cai, Caiqi Zhang, Qiancheng Xu, Meng Liu, Tiezheng Yu, Yongqi Li, and Wenjie Li. Agent-as-a-Judge. arXiv preprint arXiv:2601.05111, 2026. URL https://arxiv.org/abs/2601.05111.
15. Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, et al. A Survey on Self-Evolution of Large Language Models. arXiv preprint arXiv:2404.14387, 2024. URL https://arxiv.org/abs/2404.14387.
16. Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, et al. A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems. arXiv preprint arXiv:2508.07407, 2025. URL https://arxiv.org/abs/2508.07407.
17. Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, R. B. Xiong, et al. Self-Improvements in Modern Agentic Systems: A Survey. arXiv preprint arXiv:2607.13104, 2026. URL https://arxiv.org/abs/2607.13104.
18. Che Jiang, Jincheng Zhong, Yu Fu, Kai Tian, Junlin Yang, Kaikai Zhao, et al. Self-improving agents in the era of experience: A survey of self- to meta-evolution. *OpenReview Archive*, 2026. URL https://openreview.net/pdf?id=IUltZSgLMm.
19. Kai Wu, Hao Lyu, Zhen Luo, Chaofan Wang, Siyu Ye, Jinghao Lin, et al. AI4AI Survey: From Long-Horizon Agents to Recursive Self-Improvement—Definitions, Reliable Horizons, and Open Problems. Preprints.org, version 1, 2026. URL https://www.preprints.org/manuscript/202608.2108/v1. Preprint.
20. Yi Duan, Ying Liu, Zirui Tang, Haodong Chen, Jun Zhou, Yumou Liu, et al. The Last AI Built by Humans: Toward Genuine Recursive Self-Improvement. arXiv preprint arXiv:2609.11873, 2026. URL https://arxiv.org/abs/2609.11873v1.
21. Kaiqi Wang, Wenjin Hou, Yuchen Yan, Hongrui Jia, Zhisheng Zhong, Botao Ren, et al. Diving into Reliable Self-Evolving Agents: A Survey. Preprints.org, version 1, 2026. URL https://www.preprints.org/manuscript/202609.0913/v1. Preprint.
22. Yongheng Zhang, Ziang Liu, Jiaxuan Zhu, Shuai Wang, Xiangqi Chen, Haojing Huang, et al. From Chatbot to Digital Colleague: The Paradigm Shift Toward Persistent Autonomous AI. arXiv preprint arXiv:2606.14502, 2026. URL https://arxiv.org/abs/2606.14502v2.
23. Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, et al. Advances and Challenges in Foundation Agents: From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems. arXiv preprint arXiv:2504.01990, 2025. URL https://arxiv.org/abs/2504.01990.
24. Junhao Zheng, Chengming Shi, Xidi Cai, Qiuke Li, Duzhen Zhang, Chenxing Li, Dong Yu, and Qianli Ma. Lifelong Learning of Large Language Model based Agents: A Roadmap. arXiv preprint arXiv:2501.07278, 2025. URL https://arxiv.org/abs/2501.07278.
25. Shijian Deng, Kai Wang, Tianyu Yang, Harsh Singh, and Yapeng Tian. Self-Improvement in Multimodal Large Language Models: A Survey. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 1987–2006. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.105. URL https://aclanthology.org/2025.findings-emnlp.105/.
26. Hao Zhou, Haichuan Hu, Ye Shang, and Quanjun Zhang. Self-Evolving Coding Agents. arXiv preprint arXiv:2608.03392, 2026. URL https://arxiv.org/abs/2608.03392v1.
27. Kun Wang, Guibin Zhang, Zhenhong Zhou, Jiahao Wu, Miao Yu, Shiqian Zhao, et al. A Comprehensive Survey in LLM(-Agent) Full Stack Safety: Data, Training and Deployment. arXiv preprint arXiv:2504.15585, 2025. URL https://arxiv.org/abs/2504.15585v4.
28. Xiangjue Dong, Maria Teleki, and James Caverlee. A Survey on LLM Inference-Time Self-Improvement. arXiv preprint arXiv:2412.14352, 2024. URL https://arxiv.org/abs/2412.14352.
29. Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From Human Memory to AI Memory: A Survey on Memory Mechanisms in the Era of LLMs. arXiv preprint arXiv:2504.15965, 2025. URL https://arxiv.org/abs/2504.15965.
30. Juergen Schmidhuber. Goedel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. arXiv preprint arXiv:cs/0309048, 2003. URL https://arxiv.org/abs/cs/0309048.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

31. E. Nivel, K. R. Thórisson, B. R. Steunebrink, H. Dindo, G. Pezzulo, M. Rodriguez, et al. Bounded Recursive Self-Improvement. arXiv preprint arXiv:1312.6764, 2013. URL https://arxiv.org/abs/1312.6764.
32. Roman V. Yampolskiy. From Seed AI to Technological Singularity via Recursively Self-Improving Software. arXiv preprint arXiv:1502.06512, 2015. URL https://arxiv.org/abs/1502.06512.
33. Jasper Snoek, Hugo Larochelle, and Ryan Adams. Practical Bayesian Optimization of Machine Learning Algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2012. URL https://proceedings.neurips.cc/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html.
34. Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. arXiv preprint arXiv:1208.3719, 2012. URL https://arxiv.org/abs/1208.3719.
35. Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and Robust Automated Machine Learning. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL https://proceedings.neurips.cc/paper/2015/hash/11d0e6287202fced83f79975ec59a3a6-Abstract.html.
36. Barret Zoph and Quoc V. Le. Neural Architecture Search with Reinforcement Learning. arXiv preprint arXiv:1611.01578, 2016. URL https://arxiv.org/abs/1611.01578.
37. Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. arXiv preprint arXiv:1606.04474, 2016. URL https://arxiv.org/abs/1606.04474.
38. Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning. arXiv preprint arXiv:1611.02779, 2016. URL https://arxiv.org/abs/1611.02779.
39. Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. arXiv preprint arXiv:1703.03400, 2017. URL https://arxiv.org/abs/1703.03400.
40. Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving Machine Learning Algorithms From Scratch. arXiv preprint arXiv:2003.03384, 2020. URL https://arxiv.org/abs/2003.03384.
41. Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions. arXiv preprint arXiv:1901.01753, 2019. URL https://arxiv.org/abs/1901.01753.
42. Rui Wang, Joel Lehman, Aditya Rawal, Jiale Zhi, Yulun Li, Jeff Clune, and Kenneth O. Stanley. Enhanced POET: Open-Ended Reinforcement Learning through Unbounded Invention of Learning Challenges and their Solutions. arXiv preprint arXiv:2003.08536, 2020. URL https://arxiv.org/abs/2003.08536.
43. Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through Large Models. arXiv preprint arXiv:2206.08896, 2022. URL https://arxiv.org/abs/2206.08896.
44. Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, 2022. URL https://arxiv.org/abs/2210.03629.
45. Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv preprint arXiv:2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.
46. Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. arXiv preprint arXiv:2310.02304, 2023. URL https://arxiv.org/abs/2310.02304.
47. Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. arXiv preprint arXiv:1909.13231, 2019. URL https://arxiv.org/abs/1909.13231.
48. Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, et al. Learning to (Learn at Test Time): RNNs with Expressive Hidden States. arXiv preprint arXiv:2407.04620, 2024. URL https://arxiv.org/abs/2407.04620.
49. Thomas Chen and Zhiyuan Li. A Theoretical Framework for Self-Play Theorem Proving Algorithms. arXiv preprint arXiv:2606.01861, 2026. URL https://arxiv.org/abs/2606.01861.
50. Shi Fu, Yingjie Wang, Shengchao Hu, Peng Wang, and Dacheng Tao. Why Self-Rewarding Works: Theoretical Guarantees for Iterative Alignment of Language Models. arXiv preprint arXiv:2601.22513, 2026. URL https://arxiv.org/abs/2601.22513.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

51. Zeyu Gan and Yong Liu. Towards a Theoretical Understanding of Synthetic Data in LLM Post-Training: A Reverse-Bottleneck Perspective. arXiv preprint arXiv:2410.01720, 2024. URL https://arxiv.org/abs/2410.01720.
52. Akiyoshi Tomihari and Issei Sato. Power Distribution Bridges Sampling, Self-Reward RL, and Self-Distillation. arXiv preprint arXiv:2605.04542, 2026. URL https://arxiv.org/abs/2605.04542.
53. Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. Metalearning Continual Learning Algorithms. arXiv preprint arXiv:2312.00276, 2023. URL https://arxiv.org/abs/2312.00276.
54. Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization. arXiv preprint arXiv:2210.10760, 2022. URL https://arxiv.org/abs/2210.10760.
55. Loka Li, Zhenhao Chen, Guangyi Chen, Yixuan Zhang, Yusheng Su, Eric Xing, and Kun Zhang. Confidence Matters: Revisiting Intrinsic Self-Correction Capabilities of Large Language Models. arXiv preprint arXiv:2402.12563, 2024. URL https://arxiv.org/abs/2402.12563.
56. Guangliang Liu, Haitao Mao, Bochuan Cao, Zhiyu Xue, Xitong Zhang, Rongrong Wang, Jiliang Tang, and Kristen Johnson. On the Intrinsic Self-Correction Capability of LLMs: Uncertainty and Latent Concept. arXiv preprint arXiv:2406.02378, 2024. URL https://arxiv.org/abs/2406.02378.
57. Elroy Stav, Dvir Berlowitz, Maayan Orner, and Sarit Kraus. When Does Intrinsic Self-Correction Help? A Task-Sensitive Analysis. arXiv preprint arXiv:2606.23196, 2026. URL https://arxiv.org/abs/2606.23196.
58. Ken Tsui. Self-Correction Bench: Uncovering and Addressing the Self-Correction Blind Spot in Large Language Models. arXiv preprint arXiv:2507.02778, 2025. URL https://arxiv.org/abs/2507.02778.
59. Shen Yin, David Ken, and Joel Stremmel. Denoising Iterative Self-Correction: Structured Verification Loops for Reliable LLM Reasoning. arXiv preprint arXiv:2606.21724, 2026. URL https://arxiv.org/abs/2606.21724.
60. Jiajing Zhang, Jiamei Jiang, Chenyang Zhang, Feifei Mo, Linjing Li, and Daniel Zeng. Towards Reliable and Robust LLM Planning: Symbolic Feedback-Driven Iterative Self-Refinement Framework. arXiv preprint arXiv:2606.27757, 2026. URL https://arxiv.org/abs/2606.27757.
61. Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, et al. Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback. arXiv preprint arXiv:2302.12813, 2023. URL https://arxiv.org/abs/2302.12813.
62. Viktor Moskvoretskii, Chris Biemann, and Irina Nikishina. Self-Taught Self-Correction for Small Language Models. arXiv preprint arXiv:2503.08681, 2025. URL https://arxiv.org/abs/2503.08681.
63. Yongcheng Zeng, Xinyu Cui, Xuanfa Jin, Qirui Mi, Guoqing Liu, Zexu Sun, et al. Evolving LLMs' Self-Refinement Capability via Synergistic Training-Inference Optimization. arXiv preprint arXiv:2502.05605, 2025. URL https://arxiv.org/abs/2502.05605.
64. Qibin Wang, Pu Zhao, Shaohan Huang, Fangkai Yang, Lu Wang, Furu Wei, et al. Learning to Refine: Self-Refinement of Parallel Reasoning in LLMs. arXiv preprint arXiv:2509.00084, 2025. URL https://arxiv.org/abs/2509.00084.
65. Toby Simonds and Akira Yoshiyama. LADDER: Self-Improving LLMs Through Recursive Problem Decomposition. arXiv preprint arXiv:2503.00735, 2025. URL https://arxiv.org/abs/2503.00735.
66. Jie Ma, Binfei Chu, Jie Gao, Jinlu Zhang, Yiwei Ma, Yi Tan, et al. One Reflection Is Not Enough: Self-Correcting Autonomous Research via Multi-Hypothesis Failure Attribution. arXiv preprint arXiv:2606.31478, 2026. URL https://arxiv.org/abs/2606.31478.
67. Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, et al. Self-Refine: Iterative Refinement with Self-Feedback. arXiv preprint arXiv:2303.17651, 2023. URL https://arxiv.org/abs/2303.17651.
68. Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. arXiv preprint arXiv:2305.11738, 2023. URL https://arxiv.org/abs/2305.11738.
69. Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv preprint arXiv:2305.10601, 2023. URL https://arxiv.org/abs/2305.10601.
70. Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. arXiv preprint arXiv:2310.04406, 2023. URL https://arxiv.org/abs/2310.04406.
71. Yuxi Xie, Kenji Kawaguchi, Yiran Zhao, Xu Zhao, Min-Yen Kan, Junxian He, and Qizhe Xie. Self-Evaluation Guided Beam Search for Reasoning. arXiv preprint arXiv:2305.00633, 2023. URL https://arxiv.org/abs/2305.00633.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

72. Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-Instruct: Aligning Language Models with Self-Generated Instructions. arXiv preprint arXiv:2212.10560, 2022. URL https://arxiv.org/abs/2212.10560v2.
73. Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large Language Models Can Self-Improve. arXiv preprint arXiv:2210.11610, 2022. URL https://arxiv.org/abs/2210.11610.
74. David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv preprint arXiv:1712.01815, 2017. URL https://arxiv.org/abs/1712.01815.
75. Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, et al. Absolute Zero: Reinforced Self-play Reasoning with Zero Data. arXiv preprint arXiv:2505.03335, 2025. URL https://arxiv.org/abs/2505.03335v3.
76. Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, et al. R-Zero: Self-Evolving Reasoning LLM from Zero Data. arXiv preprint arXiv:2508.05004, 2025. URL https://arxiv.org/abs/2508.05004.
77. Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM Agents Are Experiential Learners. arXiv preprint arXiv:2308.10144, 2023. URL https://arxiv.org/abs/2308.10144.
78. Zihao Cheng, Zeming Liu, Yingyu Shan, Xinyi Wang, Xiangrong Zhu, Yunpu Ma, et al. Mem^{2}Evolve: Towards Self-Evolving Agents via Co-Evolutionary Capability Expansion and Experience Distillation. arXiv preprint arXiv:2604.10923, 2026. URL https://arxiv.org/abs/2604.10923.
79. Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason Weston, and Sainbayar Sukhbaatar. Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge. arXiv preprint arXiv:2407.19594, 2024. URL https://arxiv.org/abs/2407.19594v2.
80. Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. arXiv preprint arXiv:2408.08435, 2024. URL https://arxiv.org/abs/2408.08435v2.
81. Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM Agent Search in Modular Design Space. arXiv preprint arXiv:2410.06153, 2024. URL https://arxiv.org/abs/2410.06153.
82. Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv preprint arXiv:2507.19457, 2025. URL https://arxiv.org/abs/2507.19457.
83. Junhyuk Oh, Matteo Hessel, Wojciech M. Czarnecki, Zhongwen Xu, Hado van Hasselt, Satinder Singh, and David Silver. Discovering Reinforcement Learning Algorithms. arXiv preprint arXiv:2007.08794, 2020. URL https://arxiv.org/abs/2007.08794.
84. Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution. arXiv preprint arXiv:2309.16797, 2023. URL https://arxiv.org/abs/2309.16797.
85. Shuqi Lu, Chaofan Li, Kun Luo, Zhang Zhang, Hui Wang, Hongwang Xiao, et al. AREX: Towards a Recursively Self-Improving Agent for Deep Research. arXiv preprint arXiv:2607.21461, 2026. URL https://arxiv.org/abs/2607.21461.
86. Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large Language Models as Optimizers. arXiv preprint arXiv:2309.03409, 2023. URL https://arxiv.org/abs/2309.03409v3.
87. Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "Differentiation" via Text. arXiv preprint arXiv:2406.07496, 2024. URL https://arxiv.org/abs/2406.07496.
88. Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, et al. WizardLM: Empowering large pre-trained language models to follow complex instructions. arXiv preprint arXiv:2304.12244, 2023. URL https://arxiv.org/abs/2304.12244.
89. Xuemiao Zhang, Liangyu Xu, Feiyu Duan, Yongwei Zhou, Sirui Wang, Rongxiang Weng, Jingang Wang, and Xunliang Cai. Preference Curriculum: LLMs Should Always Be Pretrained on Their Preferred Data. arXiv preprint arXiv:2501.13126, 2025. URL https://arxiv.org/abs/2501.13126.
90. Shubham Parashar, Shurui Gui, Xiner Li, Hongyi Ling, Sushil Vemuri, Blake Olson, et al. Curriculum Reinforcement Learning from Easy to Hard Tasks Improves LLM Reasoning. arXiv preprint arXiv:2506.06632, 2025. URL https://arxiv.org/abs/2506.06632.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

91. Zhenting Wang, Guofeng Cui, Yu-Jhe Li, Kun Wan, and Wentian Zhao. DUMP: Automated Distribution-Level Curriculum Learning for RL-based LLM Post-training. arXiv preprint arXiv:2504.09710, 2025. URL https://arxiv.org/abs/2504.09710.
92. Dai Do, Manh Nguyen, Svetha Venkatesh, and Hung Le. SPaCe: Unlocking Sample-Efficient Large Language Models Training With Self-Pace Curriculum Learning. arXiv preprint arXiv:2508.05015, 2025. URL https://arxiv.org/abs/2508.05015.
93. Jiaye Lin, Mengdi Li, Xufeng Zhao, Wenhao Lu, Peilin Zhao, Stefan Wermter, and Di Wang. Curriculum-RLAIF: Curriculum Alignment with Reinforcement Learning from AI Feedback. arXiv preprint arXiv:2505.20075, 2025. URL https://arxiv.org/abs/2505.20075.
94. Peng Xia, Kaide Zeng, Jiaqi Liu, Can Qin, Fang Wu, Yiyang Zhou, Caiming Xiong, and Huaxiu Yao. Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning. arXiv preprint arXiv:2511.16043, 2025. URL https://arxiv.org/abs/2511.16043.
95. Shaobo Wang, Zhengbo Jiao, Zifan Zhang, Yilang Peng, Xu Ze, Boyu Yang, et al. Socratic-Zero : Bootstrapping Reasoning via Data-Free Agent Co-evolution. arXiv preprint arXiv:2509.24726, 2025. URL https://arxiv.org/abs/2509.24726.
96. Wai-Chung Kwan, Joshua Ong Jun Leang, Pavlos Vougiouklis, Jeff Z. Pan, Marco Valentino, and Pasquale Minervini. OpenSIR: Open-Ended Self-Improving Reasoner. arXiv preprint arXiv:2511.00602, 2025. URL https://arxiv.org/abs/2511.00602.
97. Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Ilia Kulikov, et al. SPICE: Self-Play In Corpus Environments Improves Reasoning. arXiv preprint arXiv:2510.24684, 2025. URL https://arxiv.org/abs/2510.24684.
98. Ziyi Yang, Weizhou Shen, Chenliang Li, Ruijun Chen, Fanqi Wan, Ming Yan, Xiaojun Quan, and Fei Huang. SPELL: Self-Play Reinforcement Learning for Evolving Long-Context Language Models. arXiv preprint arXiv:2509.23863, 2025. URL https://arxiv.org/abs/2509.23863.
99. Jinghan He, Junfeng Fang, Feng Xiong, Zijun Yao, Fei Shen, Haiyun Guo, Jinqiao Wang, and Tat-Seng Chua. Active Zero: Self-Evolving Vision-Language Models through Active Environment Exploration. arXiv preprint arXiv:2602.11241, 2026. URL https://arxiv.org/abs/2602.11241.
100. Jakub Grudzien Kuba, Mengting Gu, Qi Ma, Yuandong Tian, Vijai Mohan, and Jason Chen. Language Self-Play For Data-Free Training. arXiv preprint arXiv:2509.07414, 2025. URL https://arxiv.org/abs/2509.07414.
101. Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, et al. SPIRAL: Self-Play on Zero-Sum Games Incentivizes Reasoning via Multi-Agent Multi-Turn Reinforcement Learning. arXiv preprint arXiv:2506.24119, 2025. URL https://arxiv.org/abs/2506.24119.
102. Yixing Chen, Yiding Wang, Siqi Zhu, Haofei Yu, Tao Feng, Muhan Zhang, Mostofa Patwary, and Jiaxuan You. Multi-Agent Evolve: LLM Self-Improve through Co-evolution. arXiv preprint arXiv:2510.23595, 2025. URL https://arxiv.org/abs/2510.23595.
103. Roger Creus Castanyer, Geoffrey Bradway, Lorenz Wolf, Maxwill Lin, Augustine N. Mavor-Parker, and Matthew James Sargent. PopuLoRA: Co-Evolving LLM Populations for Reasoning Self-Play. arXiv preprint arXiv:2605.16727, 2026. URL https://arxiv.org/abs/2605.16727.
104. Jacob Dineen, Aswin RRV, Zhikun Xu, and Ben Zhou. Vocabulary Dropout for Curriculum Diversity in LLM Co-Evolution. arXiv preprint arXiv:2604.03472, 2026. URL https://arxiv.org/abs/2604.03472.
105. Chengsong Huang, Haolin Liu, Tong Zheng, Runpeng Dai, Langlin Huang, Jinyuan Li, et al. G-Zero: Self-Play for Open-Ended Generation from Zero Data. arXiv preprint arXiv:2605.09959, 2026. URL https://arxiv.org/abs/2605.09959.
106. Yuhang Lai, Jiazhan Feng, Yee Whye Teh, and Ning Miao. Verifier-Backed Hard Problem Generation for Mathematical Reasoning. arXiv preprint arXiv:2605.06660, 2026. URL https://arxiv.org/abs/2605.06660.
107. Jiaqi Chen, Bang Zhang, Ruotian Ma, Peisong Wang, Xiaodan Liang, Zhaopeng Tu, Xiaolong Li, and Kwan-Yee K. Wong. SPC: Evolving Self-Play Critic via Adversarial Games for LLM Reasoning. arXiv preprint arXiv:2504.19162, 2025. URL https://arxiv.org/abs/2504.19162.
108. Zi Lin, Sheng Shen, Ilia Kulikov, Jingbo Shang, Jason Weston, and Yixin Nie. Learning to Solve and Verify: A Self-Play Framework for Code and Test Generation. arXiv preprint arXiv:2502.14948, 2025. URL https://arxiv.org/abs/2502.14948.
109. Yuxiang Wei, Zhiqing Sun, Emily McMilin, Jonas Gehring, David Zhang, Gabriel Synnaeve, et al. Toward Training Superintelligent Software Agents through Self-Play SWE-RL. arXiv preprint arXiv:2512.18552, 2025. URL https://arxiv.org/abs/2512.18552.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

110. Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal Mathematics Statement Curriculum Learning. arXiv preprint arXiv:2202.01344, 2022. URL https://arxiv.org/abs/2202.01344.
111. Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, et al. TaskCraft: Automated Generation of Agentic Tasks. arXiv preprint arXiv:2506.10055, 2025. URL https://arxiv.org/abs/2506.10055.
112. Yifei Zhou, Sergey Levine, Jason Weston, Xian Li, and Sainbayar Sukhbaatar. Self-Challenging Language Model Agents. arXiv preprint arXiv:2506.01716, 2025. URL https://arxiv.org/abs/2506.01716.
113. Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Sercan Ö. Arık. Learn-by-interact: A Data-Centric Framework for Self-Adaptive Agents in Realistic Environments. arXiv preprint arXiv:2501.10893, 2025. URL https://arxiv.org/abs/2501.10893.
114. Chengyi Yang, Zhishang Xiang, Yunbo Tang, Zongpei Teng, Chengsong Huang, Fei Long, Yuhan Liu, and Jinsong Su. TTCS: Test-Time Curriculum Synthesis for Self-Evolving. arXiv preprint arXiv:2601.22628, 2026. URL https://arxiv.org/abs/2601.22628.
115. Zhangyi Hu, Chenhui Liu, Tian Huang, Jindong Li, Yang Yang, Jiemin Wu, et al. CoSPlay: Cooperative Self-Play at Test-Time with Self-Generated Code and Unit Test. arXiv preprint arXiv:2605.23491, 2026. URL https://arxiv.org/abs/2605.23491.
116. Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI Feedback. arXiv preprint arXiv:2212.08073, 2022. URL https://arxiv.org/abs/2212.08073.
117. Zhiqing Sun, Yikang Shen, Qinhong Zhou, Hongxin Zhang, Zhenfang Chen, David Cox, Yiming Yang, and Chuang Gan. Principle-Driven Self-Alignment of Language Models from Scratch with Minimal Human Supervision. arXiv preprint arXiv:2305.03047, 2023. URL https://arxiv.org/abs/2305.03047.
118. Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, et al. RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback. arXiv preprint arXiv:2309.00267, 2023. URL https://arxiv.org/abs/2309.00267.
119. Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, et al. Let's Verify Step by Step. arXiv preprint arXiv:2305.20050, 2023. URL https://arxiv.org/abs/2305.20050.
120. Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. arXiv preprint arXiv:2305.14318, 2023. URL https://arxiv.org/abs/2305.14318.
121. Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large Language Models as Tool Makers. arXiv preprint arXiv:2305.17126, 2023. URL https://arxiv.org/abs/2305.17126.
122. Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V. Le, et al. Self-Discover: Large Language Models Self-Compose Reasoning Structures. arXiv preprint arXiv:2402.03620, 2024. URL https://arxiv.org/abs/2402.03620.
123. Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, et al. AutoFlow: Automated Workflow Generation for Large Language Model Agents. arXiv preprint arXiv:2407.12821, 2024. URL https://arxiv.org/abs/2407.12821.
124. Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. SEW: Self-Evolving Agentic Workflows for Automated Code Generation. arXiv preprint arXiv:2505.18646, 2025. URL https://arxiv.org/abs/2505.18646.
125. Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, et al. Self-Evolving Multi-Agent Collaboration Networks for Software Development. arXiv preprint arXiv:2410.16946, 2024. URL https://arxiv.org/abs/2410.16946.
126. Xiaowen Ma, Yunpu Ma, Chenyang Lin, Sikuan Yan, Jinhe Bi, Zixuan Cao, et al. Self-Evolving Multi-Agent Systems via Textual Backpropagation. arXiv preprint arXiv:2506.09046, 2025. URL https://arxiv.org/abs/2506.09046.
127. Yinjie Wang, Ling Yang, Guohao Li, Mengdi Wang, and Bryon Aragam. ScoreFlow: Mastering LLM Agent Workflows via Score-based Preference Optimization. arXiv preprint arXiv:2502.04306, 2025. URL https://arxiv.org/abs/2502.04306.
128. Hongcheng Gao, Yue Liu, Yufei He, Longxu Dou, Chao Du, Zhijie Deng, et al. FlowReasoner: Reinforcing Query-Level Meta-Agents. arXiv preprint arXiv:2504.15257, 2025. URL https://arxiv.org/abs/2504.15257.
129. Wanjia Zhao, Mert Yuksekgonul, Shirley Wu, and James Zou. SiriuS: Self-improving Multi-agent Systems via Bootstrapped Reasoning. arXiv preprint arXiv:2502.04780, 2025. URL https://arxiv.org/abs/2502.04780.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

130. Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX: An Automated Framework for Evolving Agentic Workflows. arXiv preprint arXiv:2507.03616, 2025. URL https://arxiv.org/abs/2507.03616.
131. Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. EvoAgent: Towards Automatic Multi-Agent Generation via Evolutionary Algorithms. arXiv preprint arXiv:2406.14228, 2024. URL https://arxiv.org/abs/2406.14228.
132. Guibin Zhang, Kaijie Chen, Guancheng Wan, Heng Chang, Hong Cheng, Kun Wang, Shuyue Hu, and Lei Bai. EvoFlow: Evolving Diverse Agentic Workflows On The Fly. arXiv preprint arXiv:2502.07373, 2025. URL https://arxiv.org/abs/2502.07373.
133. Shiyuan Li, Yixin Liu, Qingsong Wen, Chengqi Zhang, and Shirui Pan. Assemble Your Crew: Automatic Multi-agent Communication Topology Design via Autoregressive Graph Generation. arXiv preprint arXiv:2507.18224, 2025. URL https://arxiv.org/abs/2507.18224.
134. Siyu Wang, Ruotian Lu, Zhihao Yang, Yuchao Wang, Yanzhou Zhang, Lei Xu, et al. AgentConductor: Topology Evolution for Multi-Agent Competition-Level Code Generation. arXiv preprint arXiv:2602.17100, 2026. URL https://arxiv.org/abs/2602.17100.
135. Yimeng Wang, Jiaxing Zhao, Hongbin Xie, Hexing Ma, Yuzhen Lei, Shuangxue Liu, et al. MetaGen: Self-Evolving Roles and Topologies for Multi-Agent LLM Reasoning. arXiv preprint arXiv:2601.19290, 2026. URL https://arxiv.org/abs/2601.19290.
136. Zixuan Ke, Austin Xu, Yifei Ming, Xuan-Phi Nguyen, Ryan Chin, Caiming Xiong, and Shafiq Joty. MAS-ZERO: Designing Multi-Agent Systems with Zero Supervision. arXiv preprint arXiv:2505.14996, 2025. URL https://arxiv.org/abs/2505.14996.
137. Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? arXiv preprint arXiv:2511.13646, 2025. URL https://arxiv.org/abs/2511.13646.
138. Jiahao Qiu, Xuan Qi, Hongru Wang, Xinzhe Juan, Yimin Wang, Zelin Zhao, et al. Alita-G: Self-Evolving Generative Agent for Agent Generation. arXiv preprint arXiv:2510.23601, 2025. URL https://arxiv.org/abs/2510.23601.
139. Xiwen Chen, Wenhui Zhu, Songzhu Zheng, Kashif Rasul, Yueyue Deng, and Huayu Li. SHARP: A Self-Evolving Human-Auditable Rubric Policy for Financial Trading Agents. arXiv preprint arXiv:2605.06822, 2026. URL https://arxiv.org/abs/2605.06822.
140. Jingqi Zhou, Sheng Wang, Dezhao Deng, Junwen Lu, Junwei Su, Qintong Li, et al. ToolSelf: Unifying Task Execution and Self-Reconfiguration via Tool-Driven Emergent Adaptation. arXiv preprint arXiv:2602.07883, 2026. URL https://arxiv.org/abs/2602.07883v4.
141. Fan Zhang, Vireo Zhang, Shengju Qian, Haoxuan Li, Hao Wu, Jinyang Wu, et al. Orchestra-o1: Omnimodal Agent Orchestration. arXiv preprint arXiv:2606.13707, 2026. URL https://arxiv.org/abs/2606.13707.
142. Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025. URL https://arxiv.org/abs/2506.13131.
143. Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large Language Models Are Human-Level Prompt Engineers. arXiv preprint arXiv:2211.01910, 2022. URL https://arxiv.org/abs/2211.01910.
144. Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. arXiv preprint arXiv:2305.03495, 2023. URL https://arxiv.org/abs/2305.03495.
145. Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, et al. EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers. arXiv preprint arXiv:2309.08532, 2023. URL https://arxiv.org/abs/2309.08532.
146. Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, et al. PromptAgent: Strategic Planning with Language Models Enables Expert-level Prompt Optimization. arXiv preprint arXiv:2310.16427, 2023. URL https://arxiv.org/abs/2310.16427.
147. Rithesh Murthy, Ming Zhu, Liangwei Yang, Jielin Qiu, Juntao Tan, Shelby Heinecke, et al. Promptomatix: An Automatic Prompt Optimization Framework for Large Language Models. arXiv preprint arXiv:2507.14241, 2025. URL https://arxiv.org/abs/2507.14241.
148. Jinyu Xiang, Jiayi Zhang, Zhaoyang Yu, Xinbing Liang, Fengwei Teng, Jinhao Tu, et al. Self-Supervised Prompt Optimization. arXiv preprint arXiv:2502.06855, 2025. URL https://arxiv.org/abs/2502.06855.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

149. Rean Clive Fernandes, Lukas Fehring, Theresa Eimer, Marius Lindauer, and Matthias Feurer. Environment-Grounded Automated Prompt Optimization for LLM Game Agents. arXiv preprint arXiv:2606.17838, 2026. URL https://arxiv.org/abs/2606.17838.
150. Dengyun Peng, Yuhang Zhou, Qiguang Chen, Jinhao Liu, Jingjing Chen, and Libo Qin. DLPO: Towards a Robust, Efficient, and Generalizable Prompt Optimization Framework from a Deep-Learning Perspective. arXiv preprint arXiv:2503.13413, 2025. URL https://arxiv.org/abs/2503.13413.
151. Li Yin and Zhangyang Wang. LLM-AutoDiff: Auto-Differentiate Any LLM Workflow. arXiv preprint arXiv:2501.16673, 2025. URL https://arxiv.org/abs/2501.16673.
152. Peiyan Zhang, Haibo Jin, Leyang Hu, Xinnuo Li, Liying Kang, Man Luo, Yangqiu Song, and Haohan Wang. REVOLVE: Optimizing AI Systems by Tracking Response Evolution in Textual Optimization. arXiv preprint arXiv:2412.03092, 2024. URL https://arxiv.org/abs/2412.03092.
153. Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. arXiv preprint arXiv:2401.02051, 2024. URL https://arxiv.org/abs/2401.02051.
154. Zijie Chen, Zhanchao Zhou, Yu Lu, Renjun Xu, Lili Pan, and Zhenzhong Lan. QUBE: Enhancing Automatic Heuristic Design via Quality-Uncertainty Balanced Evolution. arXiv preprint arXiv:2412.20694, 2024. URL https://arxiv.org/abs/2412.20694.
155. Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. HSEvo: Elevating Automatic Heuristic Design with Diversity-Driven Harmony Search and Genetic Algorithm Using LLMs. arXiv preprint arXiv:2412.14995, 2024. URL https://arxiv.org/abs/2412.14995.
156. Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. CodeEvolve: an open source evolutionary coding agent for algorithmic discovery and optimization. arXiv preprint arXiv:2510.14150, 2025. URL https://arxiv.org/abs/2510.14150.
157. Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI. arXiv preprint arXiv:2507.14172, 2025. URL https://arxiv.org/abs/2507.14172.
158. Yiqun Zhang, Peng Ye, Xiaocui Yang, Shi Feng, Shufei Zhang, Lei Bai, Wanli Ouyang, and Shuyue Hu. Nature-Inspired Population-Based Evolution of Large Language Models. arXiv preprint arXiv:2503.01155, 2025. URL https://arxiv.org/abs/2503.01155.
159. Zimu Wang, Yuling Shi, Mengfan Li, Zijun Liu, Jie M. Zhang, Chengcheng Wan, and Xiaodong Gu. EffiSkill: Agent Skill Based Automated Code Efficiency Optimization. arXiv preprint arXiv:2603.27850, 2026. URL https://arxiv.org/abs/2603.27850.
160. Xinyuan Song, Zekun Cai, and Liang Zhao. AlgoSkill: Learning to Design Algorithms by Scheduling Human-Like Skills. arXiv preprint arXiv:2606.29999, 2026. URL https://arxiv.org/abs/2606.29999.
161. Jiaxuan Lu, Ziyu Kong, Yemin Wang, Rong Fu, Haiyuan Wan, Cheng Yang, et al. Beyond Static Tools: Test-Time Tool Evolution for Scientific Reasoning. arXiv preprint arXiv:2601.07641, 2026. URL https://arxiv.org/abs/2601.07641.
162. Murong Yue, Zhiwei Liu, Liangwei Yang, Jianguo Zhang, Zuxin Liu, Haolin Chen, et al. ToolLibGen: Scalable Automatic Tool Creation and Aggregation for LLM Reasoning. arXiv preprint arXiv:2510.07768, 2025. URL https://arxiv.org/abs/2510.07768.
163. Yoichi Ishibashi, Taro Yano, and Masafumi Oyamada. Effective Harness Engineering for Algorithm Discovery with Coding Agents. arXiv preprint arXiv:2605.15221, 2026. URL https://arxiv.org/abs/2605.15221.
164. Temoor Tanveer. LEVI: Stronger Search Architectures Can Substitute for Larger LLMs in Evolutionary Search. arXiv preprint arXiv:2605.09764, 2026. URL https://arxiv.org/abs/2605.09764.
165. John Donaghy and Shikhar Rastogi. DEI: Diversity in Evolutionary Inference for Quality-Diversity Search. arXiv preprint arXiv:2605.27130, 2026. URL https://arxiv.org/abs/2605.27130.
166. Yuanshuo Zhang, Yuchen Hou, Bohan Tang, Shuo Chen, Muhan Zhang, Xiaowen Dong, and Siheng Chen. GNNs as Predictors of Agentic Workflow Performances. arXiv preprint arXiv:2503.11301, 2025. URL https://arxiv.org/abs/2503.11301.
167. Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv preprint arXiv:2408.06292, 2024. URL https://arxiv.org/abs/2408.06292.
168. Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv preprint arXiv:2504.08066, 2025. URL https://arxiv.org/abs/2504.08066.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

169. Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Chris Lu, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards End-to-End Automation of AI Research. arXiv preprint arXiv:2606.15497, 2026. URL https://arxiv.org/abs/2606.15497.
170. Weixian Xu, Tiantian Mi, Yixiu Liu, Yang Nan, Zhimeng Zhou, Lyumanshan Ye, et al. ASI-Evolve: AI Accelerates AI. arXiv preprint arXiv:2603.29640, 2026. URL https://arxiv.org/abs/2603.29640.
171. Alireza Ghafarollahi and Markus J. Buehler. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning. arXiv preprint arXiv:2409.05556, 2024. URL https://arxiv.org/abs/2409.05556.
172. Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Petar Sirkovic, Artiom Myaskovsky, et al. Accelerating scientific discovery with Co-Scientist. arXiv preprint arXiv:2502.18864, 2025. URL https://arxiv.org/abs/2502.18864.
173. Xianrui Zeng, Pengfei Liu, Yirui Zang, Yang Shen, Fei Yu, Chunlei Yu, Minghao Liu, and Yang Du. Socratic agents for autonomous scientific discovery in high-dimensional physical systems. arXiv preprint arXiv:2606.26722, 2026. URL https://arxiv.org/abs/2606.26722.
174. Yougang Lyu, Xi Zhang, Xinhao Yi, Yuyue Zhao, Shuyu Guo, Wenxiang Hu, et al. EvoScientist: Towards Multi-Agent Evolving AI Scientists for End-to-End Scientific Discovery. arXiv preprint arXiv:2603.08127, 2026. URL https://arxiv.org/abs/2603.08127.
175. Ruofan Jin, Zaixi Zhang, Mengdi Wang, and Le Cong. STELLA: Self-Evolving LLM Agent for Biomedical Research. arXiv preprint arXiv:2507.02004, 2025. URL https://arxiv.org/abs/2507.02004.
176. NexusAgent Team. S1-NexusAgent: a Self-Evolving Agent Framework for Multidisciplinary Scientific Research. arXiv preprint arXiv:2602.01550, 2026. URL https://arxiv.org/abs/2602.01550.
177. Xinyu Zhu, Yuzhu Cai, Zexi Liu, Cheng Wang, Fengyang Li, Wenkai Jin, et al. EvoMaster: A Foundational Evolving Agent Framework for Agentic Science at Scale. arXiv preprint arXiv:2604.17406, 2026. URL https://arxiv.org/abs/2604.17406.
178. InternAgent Team, Bo Zhang, Shiyang Feng, Xiangchao Yan, Jiakang Yuan, Runmin Ma, et al. InternAgent: When Agent Becomes the Scientist – Building Closed-Loop System from Hypothesis to Verification. arXiv preprint arXiv:2505.16938, 2025. URL https://arxiv.org/abs/2505.16938.
179. Shangheng Du, Xiangchao Yan, Jinxin Shi, Zongsheng Cao, Shiyang Feng, Zichen Liang, et al. MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery. arXiv preprint arXiv:2606.06473, 2026. URL https://arxiv.org/abs/2606.06473.
180. Sirui Xia, Yikai Zhang, Aili Chen, Siye Wu, Siyu Yuan, and Yanghua Xiao. From AI Assistant to AI Scientist: Autonomous Discovery of LLM-RL Algorithms with LLM Agents. arXiv preprint arXiv:2603.23951, 2026. URL https://arxiv.org/abs/2603.23951.
181. Chao Deng, Shaolei Zhang, Ju Fan, and Xiaoyong Du. DataEvolver: Automatic Data Preparation for Large Language Models through Multi-Level Self-Evolving. arXiv preprint arXiv:2606.07001, 2026. URL https://arxiv.org/abs/2606.07001.
182. Amy Xin, Jiening Siow, Junjie Wang, Zijun Yao, Fanjin Zhang, Jian Song, Lei Hou, and Juanzi Li. EurekAgent: Agent Environment Engineering is All You Need For Autonomous Scientific Discovery. arXiv preprint arXiv:2606.13662, 2026. URL https://arxiv.org/abs/2606.13662.
183. Zijian Wang, Hanqi Li, Ziyue Yang, Zijian Hu, Shenghan Zuo, Yunzhe Zhang, et al. Externalizing Research Synthesis and Validation in AI Scientists through a Research Harness. arXiv preprint arXiv:2606.18874, 2026. URL https://arxiv.org/abs/2606.18874.
184. Andreas Sauter, Yuyue Zhao, Jacopo Urbani, Wenxiang Hu, Zaiqiao Meng, Lun Zhou, Xiaohui Yan, and Yougang Lyu. EvoIdeator: Evolving Scientific Ideas through Checklist-Grounded Reinforcement Learning. arXiv preprint arXiv:2603.21728, 2026. URL https://arxiv.org/abs/2603.21728.
185. Antonis Antoniades, Deepak Nathani, Ritam Saha, Alfonso Amayuelas, Ivan Bercovich, Zhaotian Weng, et al. Heuresis: Search Strategies for Autonomous AI Research Agents Across Quality, Diversity and Novelty. arXiv preprint arXiv:2606.25198, 2026. URL https://arxiv.org/abs/2606.25198.
186. Hao Wu, Yongheng Zhang, Yuan Gao, Fan Xu, Fan Zhang, Ruobing Xie, et al. OMNIFLOW: A Physics-Grounded Multimodal Agent for Generalized Scientific Reasoning. arXiv preprint arXiv:2603.15797, 2026. URL https://arxiv.org/abs/2603.15797v2.
187. Wanghan Xu, Yuhao Zhou, Yifan Zhou, Qinglong Cao, Shuo Li, Jia Bu, et al. Probing Scientific General Intelligence of LLMs with Scientist-Aligned Workflows. arXiv preprint arXiv:2512.16969, 2025. URL https://arxiv.org/abs/2512.16969.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

188. Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, et al. Reinforced Self-Training (ReST) for Language Modeling. arXiv preprint arXiv:2308.08998, 2023. URL https://arxiv.org/abs/2308.08998.
189. Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models. arXiv preprint arXiv:2401.01335, 2024. URL https://arxiv.org/abs/2401.01335.
190. Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, et al. DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data. arXiv preprint arXiv:2405.14333, 2024. URL https://arxiv.org/abs/2405.14333.
191. Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search. arXiv preprint arXiv:2406.03816, 2024. URL https://arxiv.org/abs/2406.03816.
192. DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv preprint arXiv:2501.12948, 2025. URL https://arxiv.org/abs/2501.12948.
193. Jianqiao Lu, Wanjun Zhong, Wenyong Huang, Yufei Wang, Qi Zhu, Fei Mi, et al. SELF: Self-Evolution with Language Feedback. arXiv preprint arXiv:2310.00533, 2023. URL https://arxiv.org/abs/2310.00533.
194. Chenyang Zhao, Xueying Jia, Vijay Viswanathan, Tongshuang Wu, and Graham Neubig. SELF-GUIDE: Better Task-Specific Instruction Following via Self-Synthetic Finetuning. arXiv preprint arXiv:2407.12874, 2024. URL https://arxiv.org/abs/2407.12874.
195. Ping Yu, Jack Lanchantin, Tianlu Wang, Weizhe Yuan, Olga Golovneva, Ilia Kulikov, et al. CoT-Self-Instruct: Building high-quality synthetic prompts for reasoning and non-reasoning tasks. arXiv preprint arXiv:2507.23751, 2025. URL https://arxiv.org/abs/2507.23751.
196. Arian Hosseini, Xingdi Yuan, Nikolay Malkin, Aaron Courville, Alessandro Sordoni, and Rishabh Agarwal. V-STaR: Training Verifiers for Self-Taught Reasoners. arXiv preprint arXiv:2402.06457, 2024. URL https://arxiv.org/abs/2402.06457.
197. Zi-Yi Dou, Cheng-Fu Yang, Xueqing Wu, Kai-Wei Chang, and Nanyun Peng. Re-ReST: Reflection-Reinforced Self-Training for Language Agents. arXiv preprint arXiv:2406.01495, 2024. URL https://arxiv.org/abs/2406.01495.
198. Shelly Bensal, Umar Jamil, Christopher Bryant, Melisa Russak, Kiran Kamble, Dmytro Mozolevskyi, Muayad Ali, and Waseem AlShikh. Reflect, Retry, Reward: Self-Improving LLMs via Reinforcement Learning. arXiv preprint arXiv:2505.24726, 2025. URL https://arxiv.org/abs/2505.24726.
199. Joykirat Singh, Tanmoy Chakraborty, and Akshay Nambi. Self-Evolved Preference Optimization for Enhancing Mathematical Reasoning in Small Language Models. arXiv preprint arXiv:2503.04813, 2025. URL https://arxiv.org/abs/2503.04813.
200. Dian Yu, Baolin Peng, Ye Tian, Linfeng Song, Haitao Mi, and Dong Yu. SIaM: Self-Improving Code-Assisted Mathematical Reasoning of Large Language Models. arXiv preprint arXiv:2408.15565, 2024. URL https://arxiv.org/abs/2408.15565.
201. Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. arXiv preprint arXiv:2501.04519, 2025. URL https://arxiv.org/abs/2501.04519.
202. Tony Lee and Percy Liang. Self-Verified Distillation: Your Language Model Is Secretly Its Own Synthetic Data Pipeline. arXiv preprint arXiv:2605.26132, 2026. URL https://arxiv.org/abs/2605.26132.
203. Yiwei Qin, Yixiu Liu, and Pengfei Liu. DIVE: Diversified Iterative Self-Improvement. arXiv preprint arXiv:2501.00747, 2025. URL https://arxiv.org/abs/2501.00747.
204. Woosung Koh, Wonbeen Oh, Jaein Jang, MinHyung Lee, Hyeongjin Kim, Ah Yeon Kim, et al. AdaSTaR: Adaptive Data Sampling for Training Self-Taught Reasoners. arXiv preprint arXiv:2505.16322, 2025. URL https://arxiv.org/abs/2505.16322.
205. Hao Chen, Abdul Waheed, Xiang Li, Yidong Wang, Jindong Wang, Bhiksha Raj, and Marah I. Abdin. On the Diversity of Synthetic Data and its Impact on Training Large Language Models. arXiv preprint arXiv:2410.15226, 2024. URL https://arxiv.org/abs/2410.15226.
206. Haoyan Yang, Khiem Le, Ting Hua, Shangqian Gao, Binfeng Xu, Zheng Tang, et al. Dynamic Noise Preference Optimization: Self-Improvement of Large Language Models with Self-Synthetic Data. arXiv preprint arXiv:2502.05400, 2025. URL https://arxiv.org/abs/2502.05400.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

207. Xiaohang Tang, Sangwoong Yoon, Seongho Son, Huizhuo Yuan, Quanquan Gu, and Ilija Bogunovic. RSPO: Regularized Self-Play Alignment of Large Language Models. arXiv preprint arXiv:2503.00030, 2025. URL https://arxiv.org/abs/2503.00030.
208. Zitong Yang, Aonan Zhang, Hong Liu, Tatsunori Hashimoto, Emmanuel Candès, Chong Wang, and Ruoming Pang. Synthetic bootstrapped pretraining. arXiv preprint arXiv:2509.15248, 2025. URL https://arxiv.org/abs/2509.15248.
209. DatologyAI, :, Pratyush Maini, Vineeth Dorna, Parth Doshi, Aldo Carranza, et al. BeyondWeb: Lessons from Scaling Synthetic Data for Trillion-scale Pretraining. arXiv preprint arXiv:2508.10975, 2025. URL https://arxiv.org/abs/2508.10975.
210. Sheikh Shafayat, Fahim Tajwar, Ruslan Salakhutdinov, Jeff Schneider, and Andrea Zanette. Can Large Reasoning Models Self-Train? arXiv preprint arXiv:2505.21444, 2025. URL https://arxiv.org/abs/2505.21444.
211. Hao Wu, Xingjian Shi, Ziyue Huang, Penghao Zhao, Wei Xiong, Jinbao Xue, et al. BeamVQ: Aligning Space-Time Forecasting Model via Self-training on Physics-aware Metrics. arXiv preprint arXiv:2405.17051, 2024. URL https://arxiv.org/abs/2405.17051.
212. Hao Wu, Yuqi Li, Yuan Gao, Fan Xu, Fan Zhang, Kun Wang, et al. RoboAlign-R1: Distilled Multimodal Reward Alignment for Robot Video World Models. arXiv preprint arXiv:2605.03821, 2026. URL https://arxiv.org/abs/2605.03821.
213. Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. arXiv preprint arXiv:2304.03442, 2023. URL https://arxiv.org/abs/2304.03442.
214. Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560, 2023. URL https://arxiv.org/abs/2310.08560.
215. Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv preprint arXiv:2504.19413, 2025. URL https://arxiv.org/abs/2504.19413.
216. Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint arXiv:2501.13956, 2025. URL https://arxiv.org/abs/2501.13956.
217. Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. arXiv preprint arXiv:2512.12818, 2025. URL https://arxiv.org/abs/2512.12818.
218. Yu Wang and Xi Chen. MIRIX: Multi-Agent Memory System for LLM-Based Agents. arXiv preprint arXiv:2507.07957, 2025. URL https://arxiv.org/abs/2507.07957.
219. Yaxiong Wu, Yongyue Zhang, Sheng Liang, and Yong Liu. SGMem: Sentence Graph Memory for Long-Term Conversational Agents. arXiv preprint arXiv:2509.21212, 2025. URL https://arxiv.org/abs/2509.21212.
220. Haoran Sun and Shaoning Zeng. Hierarchical Memory for High-Efficiency Long-Term Reasoning in LLM Agents. arXiv preprint arXiv:2507.22925, 2025. URL https://arxiv.org/abs/2507.22925.
221. Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. arXiv preprint arXiv:2506.07398, 2025. URL https://arxiv.org/abs/2506.07398.
222. Tooraj Helmi. Decentralizing AI Memory: SHIMI, a Semantic Hierarchical Memory Index for Scalable Agent Reasoning. arXiv preprint arXiv:2504.06135, 2025. URL https://arxiv.org/abs/2504.06135.
223. Ruihong Zeng, Jinyuan Fang, Siwei Liu, and Zaiqiao Meng. On the Structural Memory of LLM Agents. arXiv preprint arXiv:2412.15266, 2024. URL https://arxiv.org/abs/2412.15266.
224. Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. arXiv preprint arXiv:2409.07429, 2024. URL https://arxiv.org/abs/2409.07429.
225. Ruhana Azam, Aditya Vempaty, and Ashish Jagmohan. Reflection-Based Memory For Web navigation Agents. arXiv preprint arXiv:2506.02158, 2025. URL https://arxiv.org/abs/2506.02158.
226. Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, et al. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. arXiv preprint arXiv:2509.25140, 2025. URL https://arxiv.org/abs/2509.25140.
227. Prakhar Dixit and Tim Oates. ISM:Self-Improving Strategy Memory for Continual Mathematical Reasoning. arXiv preprint arXiv:2606.31191, 2026. URL https://arxiv.org/abs/2606.31191.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

228. Kangning Shen, Jingyuan Zhang, Chenxi Sun, Wencong Zeng, and Yang Yue. Structurally Aligned Subtask-Level Memory for Software Engineering Agents. arXiv preprint arXiv:2602.21611, 2026. URL https://arxiv.org/abs/2602.21611.
229. Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, et al. Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv preprint arXiv:2508.19828, 2025. URL https://arxiv.org/abs/2508.19828.
230. Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, et al. MemRL: Self-Evolving Agents via Runtime Reinforcement Learning on Episodic Memory. arXiv preprint arXiv:2601.03192, 2026. URL https://arxiv.org/abs/2601.03192.
231. Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, et al. Memento: Fine-tuning LLM Agents without Fine-tuning LLMs. arXiv preprint arXiv:2508.16153, 2025. URL https://arxiv.org/abs/2508.16153.
232. Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. MemInsight: Autonomous Memory Augmentation for LLM Agents. arXiv preprint arXiv:2503.21760, 2025. URL https://arxiv.org/abs/2503.21760.
233. Haoran Xu, Jiacong Hu, Ke Zhang, Lei Yu, Yuxin Tang, Xinyuan Song, et al. SEDM: Scalable Self-Evolving Distributed Memory for Agents. arXiv preprint arXiv:2509.09498, 2025. URL https://arxiv.org/abs/2509.09498.
234. Guibin Zhang, Muxin Fu, and Shuicheng Yan. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. arXiv preprint arXiv:2509.24704, 2025. URL https://arxiv.org/abs/2509.24704.
235. Vaggelis Dorovatas, Nancy Kalaj, and Rahaf Aljundi. Continual Self-Improvement with Lightweight Experiential Latent Memories. arXiv preprint arXiv:2606.17803, 2026. URL https://arxiv.org/abs/2606.17803.
236. Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, et al. MEMORYLLM: Towards Self-Updatable Large Language Models. arXiv preprint arXiv:2402.04624, 2024. URL https://arxiv.org/abs/2402.04624.
237. Ali Behrouz, Farnoosh Hashemi, Adel Javanmard, and Vahab Mirrokni. Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories. arXiv preprint arXiv:2606.03979, 2026. URL https://arxiv.org/abs/2606.03979.
238. Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, et al. MemOS: A Memory OS for AI System. arXiv preprint arXiv:2507.03724, 2025. URL https://arxiv.org/abs/2507.03724.
239. Yunzhong Xiao, Yangmin Li, Hewei Wang, Yunlong Tang, and Zora Zhiruo Wang. ToolMem: Enhancing Multimodal Agents with Learnable Tool Capability Memory. arXiv preprint arXiv:2510.06664, 2025. URL https://arxiv.org/abs/2510.06664.
240. Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, et al. Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving. arXiv preprint arXiv:2507.06229, 2025. URL https://arxiv.org/abs/2507.06229.
241. Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, et al. ExpGraph: Model-Agnostic Experience Learning with Graph-Structured Memory for LLM Agents. arXiv preprint arXiv:2605.30712, 2026. URL https://arxiv.org/abs/2605.30712.
242. Gang Liao, Yujia He, Abdullah Ozturk, Zhouyang Li, Ying Wang, Zhitong Guo, et al. Experience Graphs: The Data Foundation for Self-Improving Agents. arXiv preprint arXiv:2606.29823, 2026. URL https://arxiv.org/abs/2606.29823.
243. Hieu Tran, Zonghai Yao, Nguyen Luong Tran, Zhichao Yang, Feiyun Ouyang, Shuo Han, Razieh Rahimi, and Hong Yu. PRIME: Planning and Retrieval-Integrated Memory for Enhanced Reasoning. arXiv preprint arXiv:2509.22315, 2025. URL https://arxiv.org/abs/2509.22315.
244. Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, et al. Skill-Weaver: Web Agents can Self-Improve by Discovering and Honing Skills. arXiv preprint arXiv:2504.07079, 2025. URL https://arxiv.org/abs/2504.07079.
245. Hongjin Qian and Zheng Liu. MetaAgent: Toward Self-Evolving Agent via Tool Meta-Learning. arXiv preprint arXiv:2508.00271, 2025. URL https://arxiv.org/abs/2508.00271.
246. Xintian Shen, Jiawei Chen, Lihao Zheng, Hao Ma, Tao Wei, and Kun Zhan. Evolving from Tool User to Creator via Training-Free Experience Reuse in Multimodal Reasoning. arXiv preprint arXiv:2602.01983, 2026. URL https://arxiv.org/abs/2602.01983.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

247. Shunyu Liu, Yaoru Li, Kongcheng Zhang, Zhenyu Cui, Wenkai Fang, Yuxuan Zheng, Tongya Zheng, and Mingli Song. Odyssey: Empowering Minecraft Agents with Open-World Skills. arXiv preprint arXiv:2407.15325, 2024. URL https://arxiv.org/abs/2407.15325.
248. Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation. arXiv preprint arXiv:2605.27366, 2026. URL https://arxiv.org/abs/2605.27366.
249. Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, et al. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. arXiv preprint arXiv:2606.01139, 2026. URL https://arxiv.org/abs/2606.01139.
250. Srishti Gautam, Arjun Radhakrishna, and Sumit Gulwani. SkillAxe: Sharpening LLM-Authored Agent Skills Through Evaluation-Guided Self-Refinement. arXiv preprint arXiv:2606.10546, 2026. URL https://arxiv.org/abs/2606.10546.
251. Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv preprint arXiv:2605.23904, 2026. URL https://arxiv.org/abs/2605.23904.
252. Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning Self-Evolving Skills for Coding Agents. arXiv preprint arXiv:2605.25430, 2026. URL https://arxiv.org/abs/2605.25430.
253. Min Yang, Jinghua Piao, Xu Xia, Xiaochong Lan, Jiaju Chen, Yongshun Gong, and Yong Li. SkillMaster: Toward Autonomous Skill Mastery in LLM Agents. arXiv preprint arXiv:2605.08693, 2026. URL https://arxiv.org/abs/2605.08693.
254. Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, et al. Skill-R1: Agent Skill Evolution via Reinforcement Learning. arXiv preprint arXiv:2605.09359, 2026. URL https://arxiv.org/abs/2605.09359.
255. Jiongxiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, et al. Reinforcement Learning for Self-Improving Agent with Skill Library. arXiv preprint arXiv:2512.17102, 2025. URL https://arxiv.org/abs/2512.17102.
256. Chuan Xiao, Zhengbo Jiao, Shaobo Wang, Wei Wang, Bing Zhao, Hu Wei, Linfeng Zhang, and Lin Qu. Socratic-SWE: Self-Evolving Coding Agents via Trace-Derived Agent Skills. arXiv preprint arXiv:2606.07412, 2026. URL https://arxiv.org/abs/2606.07412.
257. Ye Liu, Srijan Bansal, Bo Pang, Yang Li, Zeyu Leo Liu, Yifei Ming, et al. Procedural Memory Distillation: Online Reflection for Self-Improving Language Models. arXiv preprint arXiv:2607.01480, 2026. URL https://arxiv.org/abs/2607.01480.
258. Alvin Zhu, Yusuke Tanaka, Andrew Goldberg, and Dennis Hong. AURA: Autonomous Upskilling with Retrieval-Augmented Agents. arXiv preprint arXiv:2506.02507, 2025. URL https://arxiv.org/abs/2506.02507.
259. Yangbo Wei, Zhen Huang, Shaoqiang Lu, Junhong Qian, Qifan Wang, Chen Wu, and Lei He. SkillSmith: Co-Evolving Skills and Tools for Self-Improving Agent Systems. arXiv preprint arXiv:2606.01314, 2026. URL https://arxiv.org/abs/2606.01314.
260. Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Ratchet: How Reliable Must an LLM Judge Be to Retire a Skill? arXiv preprint arXiv:2605.22148, 2026. URL https://arxiv.org/abs/2605.22148.
261. Yurun Chen, Biao Yi, Keting Yin, and Shengyu Zhang. SkillHarness: Harnessing Safe Skills for Computer-Use Agents. arXiv preprint arXiv:2606.20636, 2026. URL https://arxiv.org/abs/2606.20636.
262. Yiyang Feng, Biddut Sarker Bijoy, Niranjan Balasubramanian, and Jiawei Zhou. Break It Down, Pass It On: Cross-Task Skill Transfer in LLM Agents. arXiv preprint arXiv:2608.20274, 2026. URL https://arxiv.org/abs/2608.20274.
263. Xueqiao Sun, Xiaohan Wang, Ludwig Schmidt, Serena Yeung-Levy, and Yuhui Zhang. Learning from Failure: Inference-Time Self-Improvement for Computer-Use Agents. arXiv preprint arXiv:2606.31270, 2026. URL https://arxiv.org/abs/2606.31270.
264. Ajay Patel, Markus Hofmarcher, Claudiu Leoveanu-Condrei, Marius-Constantin Dinu, Chris Callison-Burch, and Sepp Hochreiter. Large Language Models Can Self-Improve At Web Agent Tasks. arXiv preprint arXiv:2405.20309, 2024. URL https://arxiv.org/abs/2405.20309.
265. Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents. arXiv preprint arXiv:2408.07199, 2024. URL https://arxiv.org/abs/2408.07199.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

266. Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, et al. WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning. arXiv preprint arXiv:2411.02337, 2024. URL https://arxiv.org/abs/2411.02337.
267. Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. WebEvolver: Enhancing Web Agent Self-Improvement with Coevolving World Model. arXiv preprint arXiv:2504.21024, 2025. URL https://arxiv.org/abs/2504.21024.
268. Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, and Jiaqi Wang. SEAgent: Self-Evolving Computer Use Agent with Autonomous Learning from Experience. arXiv preprint arXiv:2508.04700, 2025. URL https://arxiv.org/abs/2508.04700.
269. Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, et al. RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning. arXiv preprint arXiv:2504.20073, 2025. URL https://arxiv.org/abs/2504.20073.
270. Mingyue Cheng, Shuo Yu, Daoyu Wang, Qingchuan Li, Xiaoyu Tao, Jie Ouyang, et al. Agent-R1: A Unified and Modular Framework for Agentic Reinforcement Learning. arXiv preprint arXiv:2511.14460, 2025. URL https://arxiv.org/abs/2511.14460.
271. Jeff Da, Clinton Wang, Xiang Deng, Yuntao Ma, Nikhil Barhate, and Sean Hendryx. Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards. arXiv preprint arXiv:2506.11425, 2025. URL https://arxiv.org/abs/2506.11425.
272. Genglin Liu, Shijie Geng, Sha Li, Hejie Cui, Sarah Zhang, Xin Liu, and Tianyi Liu. WebCoach: Self-Evolving Web Agents with Cross-Session Memory Guidance. arXiv preprint arXiv:2511.12997, 2025. URL https://arxiv.org/abs/2511.12997.
273. Cheng Yang, Xuemeng Yang, Licheng Wen, Daocheng Fu, Jianbiao Mei, Rong Wu, et al. Learning on the Job: An Experience-Driven Self-Evolving Agent for Long-Horizon Tasks. arXiv preprint arXiv:2510.08002, 2025. URL https://arxiv.org/abs/2510.08002.
274. Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, et al. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv preprint arXiv:2510.16079, 2025. URL https://arxiv.org/abs/2510.16079.
275. Xuechen Liang, Yangfan He, Yinghui Xia, Xinyuan Song, Jianhui Wang, Meiling Tao, et al. Self-evolving Agents with reflective and memory-augmented abilities. arXiv preprint arXiv:2409.00872, 2024. URL https://arxiv.org/abs/2409.00872.
276. Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, et al. Building Self-Evolving Agents via Experience-Driven Lifelong Learning: A Framework and Benchmark. arXiv preprint arXiv:2508.19005, 2025. URL https://arxiv.org/abs/2508.19005.
277. Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. arXiv preprint arXiv:2402.07456, 2024. URL https://arxiv.org/abs/2402.07456.
278. Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, et al. Alita: Generalist Agent Enabling Scalable Agentic Reasoning with Minimal Predefinition and Maximal Self-Evolution. arXiv preprint arXiv:2505.20286, 2025. URL https://arxiv.org/abs/2505.20286.
279. Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, et al. Symbolic Learning Enables Self-Evolving Agents. arXiv preprint arXiv:2406.18532, 2024. URL https://arxiv.org/abs/2406.18532.
280. Emre Can Acikgoz, Cheng Qian, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Self-Improving LLM Agents at Test-Time. arXiv preprint arXiv:2510.07841, 2025. URL https://arxiv.org/abs/2510.07841.
281. Yuanyuan Xu, Wenjie Zhang, Yin Chen, Xuemin Lin, and Ying Zhang. Self-Evolving Agents as Dynamic Graph Transformation: A Survey and New Perspective. arXiv preprint arXiv:2608.18104, 2026. URL https://arxiv.org/abs/2608.18104.
282. Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A Self-Improving Coding Agent. arXiv preprint arXiv:2504.15228, 2025. URL https://arxiv.org/abs/2504.15228.
283. Huahui Yi, Kun Wang, Qiankun Li, Miao Yu, Liang Lin, Gongli Xi, et al. SaFeR-VLM: Toward Safety-aware Fine-grained Reasoning in Multimodal Models. arXiv preprint arXiv:2510.06871, 2025. URL https://arxiv.org/abs/2510.06871v2.
284. Toby Simonds, Kevin Lopez, Akira Yoshiyama, and Dominique Garmier. RLSR: Reinforcement Learning from Self Reward. arXiv preprint arXiv:2505.08827, 2025. URL https://arxiv.org/abs/2505.08827.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

285. Haolin Chen, Yihao Feng, Zuxin Liu, Weiran Yao, Akshara Prabhakar, Shelby Heinecke, et al. Language Models are Hidden Reasoners: Unlocking Latent Reasoning Capabilities via Self-Rewarding. arXiv preprint arXiv:2411.04282, 2024. URL https://arxiv.org/abs/2411.04282.
286. Kongcheng Zhang, Qi Yao, Shunyu Liu, Yingjie Wang, Baisheng Lai, Jieping Ye, Mingli Song, and Dacheng Tao. Consistent Paths Lead to Truth: Self-Rewarding Reinforcement Learning for LLM Reasoning. arXiv preprint arXiv:2506.08745, 2025. URL https://arxiv.org/abs/2506.08745.
287. Yi-Chen Li, Tian Xu, Yang Yu, Xuqin Zhang, Xiong-Hui Chen, Zhongxiang Ling, et al. Generalist Reward Models: Found Inside Large Language Models. arXiv preprint arXiv:2506.23235, 2025. URL https://arxiv.org/abs/2506.23235.
288. Chunyang Jiang, Yonggang Zhang, Yiyang Cai, Chi-Min Chan, Yulong Liu, Mingming Chen, Wei Xue, and Yike Guo. Semantic Voting: A Self-Evaluation-Free Approach for Efficient LLM Self-Improvement on Unverifiable Open-ended Tasks. arXiv preprint arXiv:2509.23067, 2025. URL https://arxiv.org/abs/2509.23067.
289. Shuaijie She, Yu Bao, Yu Lu, Lu Xu, Tao Li, Wenhao Zhu, et al. DuPO: Enabling Reliable LLM Self-Verification via Dual Preference Optimization. arXiv preprint arXiv:2508.14460, 2025. URL https://arxiv.org/abs/2508.14460.
290. Shi Feng, Hanlin Zhang, Fan Nie, Sham Kakade, and Yiling Chen. Peer-Predictive Self-Training for Language Model Reasoning. arXiv preprint arXiv:2604.13356, 2026. URL https://arxiv.org/abs/2604.13356.
291. Chen Henry Wu and Aditi Raghunathan. Self-Trained Verification for Training- and Test-Time Self-Improvement. arXiv preprint arXiv:2605.30290, 2026. URL https://arxiv.org/abs/2605.30290.
292. Zhiheng Xi, Dingwen Yang, Jixuan Huang, Jiafu Tang, Guanyu Li, Yiwen Ding, et al. Enhancing LLM Reasoning via Critique Models with Test-Time and Training-Time Supervision. arXiv preprint arXiv:2411.16579, 2024. URL https://arxiv.org/abs/2411.16579.
293. Wei Xiong, Hanning Zhang, Chenlu Ye, Lichang Chen, Nan Jiang, and Tong Zhang. Self-rewarding correction for mathematical reasoning. arXiv preprint arXiv:2502.19613, 2025. URL https://arxiv.org/abs/2502.19613.
294. Yuhua Jiang, Yuwen Xiong, Yufeng Yuan, Chao Xin, Wenyuan Xu, Yu Yue, Qianchuan Zhao, and Lin Yan. PAG: Multi-Turn Reinforced LLM Self-Correction with Policy as Generative Verifier. arXiv preprint arXiv:2506.10406, 2025. URL https://arxiv.org/abs/2506.10406.
295. Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. ReVeal: Self-Evolving Code Agents via Reliable Self-Verification. arXiv preprint arXiv:2506.11442, 2025. URL https://arxiv.org/abs/2506.11442.
296. Sanjiban Choudhury. Process Reward Models for LLM Agents: Practical Framework and Directions. arXiv preprint arXiv:2502.10325, 2025. URL https://arxiv.org/abs/2502.10325.
297. Nakyung Lee, Sangwoo Hong, and Jungwoo Lee. Efficient Process Reward Modeling via Contrastive Mutual Information. arXiv preprint arXiv:2604.10660, 2026. URL https://arxiv.org/abs/2604.10660.
298. KaShun Shum, Binyuan Hui, Jiawei Chen, Lei Zhang, X. W., Jiaxi Yang, et al. SWE-RM: Execution-free Feedback For Software Engineering Agents. arXiv preprint arXiv:2512.21919, 2025. URL https://arxiv.org/abs/2512.21919.
299. Tevin Wang and Chenyan Xiong. AutoRule: Reasoning Chain-of-thought Extracted Rule-based Rewards Improve Preference Learning. arXiv preprint arXiv:2506.15651, 2025. URL https://arxiv.org/abs/2506.15651.
300. Jiuzhou Han, Wray Buntine, and Ehsan Shareghi. VerifiAgent: a Unified Verification Agent in Language Model Reasoning. arXiv preprint arXiv:2504.00406, 2025. URL https://arxiv.org/abs/2504.00406.
301. Yuxuan Wan, Tianqing Fang, Zaitang Li, Yintong Huo, Wenxuan Wang, Haitao Mi, Dong Yu, and Michael R. Lyu. Inference-Time Scaling of Verification: Self-Evolving Deep Research Agents via Test-Time Rubric-Guided Verification. arXiv preprint arXiv:2601.15808, 2026. URL https://arxiv.org/abs/2601.15808.
302. Aojie Yuan, Yi Nian, Haiyue Zhang, Zijian Su, and Yue Zhao. SEVA: Self-Evolving Verification Agent with Process Reward for Fact Attribution. arXiv preprint arXiv:2606.29713, 2026. URL https://arxiv.org/abs/2606.29713.
303. Yinjie Wang, Ling Yang, Ye Tian, Ke Shen, and Mengdi Wang. Co-Evolving LLM Coder and Unit Tester via Reinforcement Learning. arXiv preprint arXiv:2506.03136, 2025. URL https://arxiv.org/abs/2506.03136.
304. Ethan Mendes and Alan Ritter. Language Models can Self-Improve at State-Value Estimation for Better Search. arXiv preprint arXiv:2503.02878, 2025. URL https://arxiv.org/abs/2503.02878.
305. Yuan Sui and Bryan Hooi. Conversation for Non-verifiable Learning: Self-Evolving LLMs through Meta-Evaluation. arXiv preprint arXiv:2601.21464, 2026. URL https://arxiv.org/abs/2601.21464.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

306. Zihan Tan, Leixin Sun, Zitong Shi, Yitao Liu, Jiajun Wu, Nathaniel Brooks, et al. MetaRSI / RSI2: A Meta-Recursive Self-Improving System for Recursive Self-Improving Systems Themselves. arXiv preprint arXiv:2609.06396, 2026. URL https://arxiv.org/abs/2609.06396.
307. Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement. arXiv preprint arXiv:2410.04444, 2024. URL https://arxiv.org/abs/2410.04444.
308. Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine. arXiv preprint arXiv:2510.21614, 2025. URL https://arxiv.org/abs/2510.21614.
309. Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-Evolution of Agent Memory Systems. arXiv preprint arXiv:2512.18746, 2025. URL https://arxiv.org/abs/2512.18746.
310. Tu Hu, Ronghao Chen, Shuo Zhang, Jianghao Yin, Mou Xiao Feng, Jingping Liu, et al. Controlled Self-Evolution for Algorithmic Code Optimization. arXiv preprint arXiv:2601.07348, 2026. URL https://arxiv.org/abs/2601.07348.
311. Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA: Self Improving AI with Harness & Weight Updates. arXiv preprint arXiv:2605.27276, 2026. URL https://arxiv.org/abs/2605.27276.
312. Boyun Zhang, Chao Wang, and Kai Wu. EVOM: Agentic Meta-Evolution of Actor-Critic Architectures for Reinforcement Learning. arXiv preprint arXiv:2606.26327, 2026. URL https://arxiv.org/abs/2606.26327.
313. Ziyang Liu, Xinyan Guo, Xuchen Wei, Han Hao, and Liu Yang. Escher-Loop: Mutual Evolution by Closed-Loop Self-Referential Optimization. arXiv preprint arXiv:2604.23472, 2026. URL https://arxiv.org/abs/2604.23472.
314. Alex Iacob, Andrej Jovanović, William F. Shen, Daniel Burkhardt, Meghdad Kurmanji, Nurbek Tastan, et al. The Red Queen Gödel Machine: Co-Evolving Agents and Their Evaluators. arXiv preprint arXiv:2606.26294, 2026. URL https://arxiv.org/abs/2606.26294.
315. Yuan-Hang Zhang, Chesson Sipling, and Massimiliano Di Ventra. Scientific discovery as meta-optimization: a combinatorial optimization case study. arXiv preprint arXiv:2606.26728, 2026. URL https://arxiv.org/abs/2606.26728.
316. Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, et al. AgentBench: Evaluating LLMs as Agents. arXiv preprint arXiv:2308.03688, 2023. URL https://arxiv.org/abs/2308.03688.
317. Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for General AI Assistants. arXiv preprint arXiv:2311.12983, 2023. URL https://arxiv.org/abs/2311.12983.
318. Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint arXiv:2310.06770, 2023. URL https://arxiv.org/abs/2310.06770.
319. Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation. arXiv preprint arXiv:2310.03302, 2023. URL https://arxiv.org/abs/2310.03302.
320. Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv preprint arXiv:2410.07095, 2024. URL https://arxiv.org/abs/2410.07095.
321. Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, et al. ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery. arXiv preprint arXiv:2410.05080, 2024. URL https://arxiv.org/abs/2410.05080.
322. Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, et al. RewardBench: Evaluating Reward Models for Language Modeling. arXiv preprint arXiv:2403.13787, 2024. URL https://arxiv.org/abs/2403.13787.
323. Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, et al. ProcessBench: Identifying Process Errors in Mathematical Reasoning. arXiv preprint arXiv:2412.06559, 2024. URL https://arxiv.org/abs/2412.06559.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

324. Fan Zhang, Guangming Yao, Jinyang Wu, Hao Wu, Zheng Lian, Xinyu Geng, et al. VideoGAIA: A Benchmark for General AI Assistants on Agentic Video Understanding. arXiv preprint arXiv:2608.14718, 2026. URL https://arxiv.org/abs/2608.14718.
325. Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Tengfei Wang, et al. SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment. arXiv preprint arXiv:2604.08988, 2026. URL https://arxiv.org/abs/2604.08988.
326. Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. SEAGym: An Evaluation Environment for Self-Evolving LLM Agents. arXiv preprint arXiv:2606.17546, 2026. URL https://arxiv.org/abs/2606.17546.
327. Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv preprint arXiv:2307.13854, 2023. URL https://arxiv.org/abs/2307.13854.
328. Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, et al. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. arXiv preprint arXiv:2409.08264, 2024. URL https://arxiv.org/abs/2409.08264.
329. Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045, 2024. URL https://arxiv.org/abs/2406.12045.
330. John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv preprint arXiv:2405.15793, 2024. URL https://arxiv.org/abs/2405.15793.
331. Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv preprint arXiv:2407.01489, 2024. URL https://arxiv.org/abs/2407.01489.
332. Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, et al. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv preprint arXiv:2509.16941, 2025. URL https://arxiv.org/abs/2509.16941.
333. Spandan Garg, Benjamin Steenhoek, and Yufan Huang. Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation. arXiv preprint arXiv:2510.08996, 2025. URL https://arxiv.org/abs/2510.08996.
334. Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv preprint arXiv:2412.21139, 2024. URL https://arxiv.org/abs/2412.21139.
335. Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. arXiv preprint arXiv:2504.07164, 2025. URL https://arxiv.org/abs/2504.07164.
336. Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. EnvScaler: Scaling Tool-Interactive Environments for LLM Agent via Programmatic Synthesis. arXiv preprint arXiv:2601.05808, 2026. URL https://arxiv.org/abs/2601.05808.
337. Xiao Wang, Yuansen Zhang, Tianze Chen, Songyang Gao, Senjie Jin, Xianjun Yang, et al. TRACE: A Comprehensive Benchmark for Continual Learning in Large Language Models. arXiv preprint arXiv:2310.06762, 2023. URL https://arxiv.org/abs/2310.06762.
338. Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM Agents as Lifelong Learners. arXiv preprint arXiv:2505.11942, 2025. URL https://arxiv.org/abs/2505.11942.
339. Jiayi Zhang, Yiran Peng, Fanqi Kong, Cheng Yang, Yifan Wu, Zhaoyang Yu, et al. AutoEnv: Automated Environments for Measuring Cross-Environment Agent Learning. arXiv preprint arXiv:2511.19304, 2025. URL https://arxiv.org/abs/2511.19304.
340. Zhe Ren, Yibo Yang, Yimeng Chen, Zijun Zhao, Benshuo Fu, Zhihao Shu, et al. GateMem: Benchmarking Memory Governance in Multi-Principal Shared-Memory Agents. arXiv preprint arXiv:2606.18829, 2026. URL https://arxiv.org/abs/2606.18829.
341. Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. Tool-Genesis: A Task-Driven Tool Creation Benchmark for Self-Evolving Language Agent. arXiv preprint arXiv:2603.05578, 2026. URL https://arxiv.org/abs/2603.05578.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

342. Xinyu Lu, Tianshu Wang, Pengbo Wang, zujie wen, Zhiqiang Zhang, Jun Zhou, et al. The Meta-Agent Challenge: Are Current Agents Capable of Autonomous Agent Development? arXiv preprint arXiv:2606.04455, 2026. URL https://arxiv.org/abs/2606.04455.
343. Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqiu Wang, Mingju Gao, Kaisen Yang, et al. AI4AI-Bench: Benchmarking LLM Agents in Algorithmic Design for Recursive Self-Improvement. arXiv preprint arXiv:2608.20318, 2026. URL https://arxiv.org/abs/2608.20318.
344. Siyuan Wang, Zhuohan Long, Zhihao Fan, Zhongyu Wei, and Xuanjing Huang. Benchmark Self-Evolving: A Multi-Agent Framework for Dynamic LLM Evaluation. arXiv preprint arXiv:2402.11443, 2024. URL https://arxiv.org/abs/2402.11443.
345. Dadi Guo, Tianyi Zhou, Dongrui Liu, Chen Qian, Qihan Ren, Shuai Shao, et al. Towards Self-Evolving Benchmarks: Synthesizing Agent Trajectories via Test-Time Exploration under Validate-by-Reproduce Paradigm. arXiv preprint arXiv:2510.00415, 2025. URL https://arxiv.org/abs/2510.00415.
346. Yangzhen Wu, Aaron J. Li, Wenjie Ma, Li Cao, Ziheng Zhou, Mert Cemri, et al. BenchEvolver: Frontier Task Synthesis via Solution-Centric Evolution. arXiv preprint arXiv:2606.01286, 2026. URL https://arxiv.org/abs/2606.01286.
347. Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv preprint arXiv:2408.03314, 2024. URL https://arxiv.org/abs/2408.03314.
348. Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, et al. SWE-Effi: Re-Evaluating Software AI Agent System Effectiveness Under Resource Constraints. arXiv preprint arXiv:2509.09853, 2025. URL https://arxiv.org/abs/2509.09853.
349. Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv preprint arXiv:2403.07974, 2024. URL https://arxiv.org/abs/2403.07974.
350. François Chollet. On the Measure of Intelligence. arXiv preprint arXiv:1911.01547, 2019. URL https://arxiv.org/abs/1911.01547.
351. Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv preprint arXiv:2504.13837, 2025. URL https://arxiv.org/abs/2504.13837.
352. Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv preprint arXiv:2310.01798, 2023. URL https://arxiv.org/abs/2310.01798.
353. Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv preprint arXiv:2306.05685, 2023. URL https://arxiv.org/abs/2306.05685.
354. Fan Zhang, Vireo Zhang, Shengju Qian, Haoxuan Li, Zheng Lian, Hao Wu, et al. Struct-Searcher: Agentic Structural Thinking Advances Multimodal Deep Information Seeking. arXiv preprint arXiv:2606.07689, 2026. URL https://arxiv.org/abs/2606.07689.
355. Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The Curse of Recursion: Training on Generated Data Makes Models Forget. arXiv preprint arXiv:2305.17493, 2023. URL https://arxiv.org/abs/2305.17493.
356. Sina Alemohammad, Josue Casco-Rodriguez, Lorenzo Luzi, Ahmed Imtiaz Humayun, Hossein Babaei, Daniel LeJeune, Ali Siahkoohi, and Richard G. Baraniuk. Self-Consuming Generative Models Go MAD. arXiv preprint arXiv:2307.01850, 2023. URL https://arxiv.org/abs/2307.01850.
357. Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Henry Sleight, John Hughes, et al. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv preprint arXiv:2404.01413, 2024. URL https://arxiv.org/abs/2404.01413.
358. Jianzhe Lin. Self-Improvement Can Self-Regress: The Rise-and-Collapse Failure Mode of LLM Self-Training. arXiv preprint arXiv:2606.21090, 2026. URL https://arxiv.org/abs/2606.21090.
359. Jianzhe Lin, Fei Wang, Xiaolin Li, Rajeshkumar Golani, and Jubin Chheda. Repeated post-training is not Self-improving: Diagnosing Scientific Amnesia in Continual DPO Pipelines. arXiv preprint arXiv:2606.21089, 2026. URL https://arxiv.org/abs/2606.21089.
360. Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Kangrui Wang, Ruochen Zhou, et al. Why Do LLM Agents Fail in Exploring New Environments? A World-Modeling Perspective. arXiv preprint arXiv:2510.15047, 2025. URL https://arxiv.org/abs/2510.15047.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

361. Chuyi Tan, Peiwen Yuan, Xinglin Wang, Yiwei Li, Shaoxiong Feng, Yueqi Zhang, et al. Breaking the Self-Confirming Loop: Diagnosing and Mitigating Systemic Reward Bias in Self-Rewarding RL. arXiv preprint arXiv:2510.08977, 2025. URL https://arxiv.org/abs/2510.08977.
362. Liangjie Huang, Dawei Li, Huan Liu, and Lu Cheng. Beyond Accuracy: The Role of Calibration in Self-Improving Large Language Models. arXiv preprint arXiv:2504.02902, 2025. URL https://arxiv.org/abs/2504.02902.
363. Bentley DeVilling. The Mirror Loop: Recursive Non-Convergence in Generative Reasoning Systems. arXiv preprint arXiv:2510.21861, 2025. URL https://arxiv.org/abs/2510.21861.
364. Sophia Xiao Pu, Zhaotian Weng, Chengzhi Liu, Jayanth Srinivasa, Gaowen Liu, William Yang Wang, and Xin Eric Wang. Survive or Collapse: The Asymmetric Roles of Data Gating and Reward Grounding in Self-Play RL. arXiv preprint arXiv:2605.22217, 2026. URL https://arxiv.org/abs/2605.22217.
365. Wei Liu, Siya Qi, Yali Du, and Yulan He. Self-Play Only Evolves When Self-Synthetic Pipeline Ensures Learnable Information Gain. arXiv preprint arXiv:2603.02218, 2026. URL https://arxiv.org/abs/2603.02218.
366. Srishti Gureja, Elena Tommasone, Jingyi He, Sara Hooker, Matthias Gallé, and Marzieh Fadaee. Verification Limits Code LLM Training. arXiv preprint arXiv:2509.20837, 2025. URL https://arxiv.org/abs/2509.20837.
367. Harsh Patel. A Verifiable Search Is Not a Learnable Chain-of-Thought. arXiv preprint arXiv:2606.21884, 2026. URL https://arxiv.org/abs/2606.21884.
368. Shuai Shao, Qihan Ren, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, et al. Your Agent May Misevolve: Emergent Risks in Self-evolving LLM Agents. arXiv preprint arXiv:2509.26354, 2025. URL https://arxiv.org/abs/2509.26354.
369. Siwei Han, Kaiwen Xiong, Jiaqi Liu, Xinyu Ye, Yaofeng Su, Wenbo Duan, et al. Alignment Tipping Process: How Self-Evolution Pushes LLM Agents Off the Rails. arXiv preprint arXiv:2510.04860, 2025. URL https://arxiv.org/abs/2510.04860.
370. Dongyoon Hahm, Taywon Min, Woogyeol Jin, and Kimin Lee. Unintended Misalignment from Agentic Fine-Tuning: Risks and Mitigation. arXiv preprint arXiv:2508.14031, 2025. URL https://arxiv.org/abs/2508.14031.
371. Ruixiao Lin, Xinhao Deng, Qingming Li, Jianan Ma, Yunhao Feng, Yuqi Qing, et al. Safety in Self-Evolving LLM Agent Systems: Threats, Amplification, and Case Studies. arXiv preprint arXiv:2606.23075, 2026. URL https://arxiv.org/abs/2606.23075.
372. Shiyi Yang, Zhibo Hu, Xinshu Li, Chen Wang, Tong Yu, Xiwei Xu, Liming Zhu, and Lina Yao. DrunkAgent: Stealthy Memory Corruption in LLM-Powered Recommender Agents. arXiv preprint arXiv:2503.23804, 2025. URL https://arxiv.org/abs/2503.23804.
373. Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling Privacy Risks in LLM Agent Memory. arXiv preprint arXiv:2502.13172, 2025. URL https://arxiv.org/abs/2502.13172.
374. Balachandra Devarangadi Sunil, Isheeta Sinha, Piyush Maheshwari, Shantanu Todmal, Shreyan Mallik, and Shuchi Mishra. Memory Poisoning Attack and Defense on Memory Based LLM-Agents. arXiv preprint arXiv:2601.05504, 2026. URL https://arxiv.org/abs/2601.05504.
375. Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, et al. A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv preprint arXiv:2510.02373, 2025. URL https://arxiv.org/abs/2510.02373.
376. Youngduk Kim, Minkyoo Song, and Seungwon Shin. SkillMutator: Benchmarking and Defending Language-and-Code Cross-modal Attacks on LLM Agent Skills. arXiv preprint arXiv:2606.14154, 2026. URL https://arxiv.org/abs/2606.14154.
377. Xinfeng Li, Dong Huang, Jie Li, Hongyi Cai, Zhenhong Zhou, Wei Dong, XiaoFeng Wang, and Yang Liu. A Vision for Access Control in LLM-based Agent Systems. arXiv preprint arXiv:2510.11108, 2025. URL https://arxiv.org/abs/2510.11108.
378. Zonglin Yang, Xingtong Liu, and Xinyan Xu. SciIntegrity-Bench: A Benchmark for Evaluating Academic Integrity in AI Scientist Systems. arXiv preprint arXiv:2605.10246, 2026. URL https://arxiv.org/abs/2605.10246.
379. Sy-Tuyen Ho, Minghui Liu, Huy Nghiem, and Furong Huang. SoundnessBench: Can Your AI Scientist Really Tell Good Research Ideas from Bad Ones? arXiv preprint arXiv:2605.30329, 2026. URL https://arxiv.org/abs/2605.30329.
380. Sasi Kiran Gaddipati, Diyana Muhammed, Farhana Keya, Gollam Rabby, and Sören Auer. MLReplicate: Benchmarking Autonomous Research Systems for Machine Learning Reproducibility. arXiv preprint arXiv:2605.16616, 2026. URL https://arxiv.org/abs/2605.16616.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

381. Dhruv Trehan and Paras Chopra. Why LLMs Aren't Scientists Yet: Lessons from Four Autonomous Research Attempts. arXiv preprint arXiv:2601.03315, 2026. URL https://arxiv.org/abs/2601.03315.
382. Jiayi Geng, Howard Chen, Dilip Arumugam, and Thomas L. Griffiths. Are Large Language Models Reliable AI Scientists? Assessing Reverse-Engineering of Black-Box Systems. arXiv preprint arXiv:2505.17968, 2025. URL https://arxiv.org/abs/2505.17968.
383. Razeen A Rasheed, Somnath Banerjee, Animesh Mukherjee, and Rima Hazra. From Fluent to Verifiable: Claim-Level Auditability for Deep Research Agents. arXiv preprint arXiv:2602.13855, 2026. URL https://arxiv.org/abs/2602.13855.
384. Hiroyasu Usami, Keisuke Hara, Ayato Tsuboi, and Naohiko Matsuda. LLM Judges Have Dark Current: A Psychometric Datasheet for LLM-as-a-Judge Evaluation. arXiv preprint arXiv:2606.15610, 2026. URL https://arxiv.org/abs/2606.15610.
385. Harshit Bisht, Vinay Kumar, Kevin Maik Jablonka, Mausam, and N. M. Anoop Krishnan. Agentic AI Scientists Are Not Built For Autonomous Scientific Discovery. arXiv preprint arXiv:2605.08956, 2026. URL https://arxiv.org/abs/2605.08956.
386. Kargi Chauhan. Dead Science Walking: Publication Bias and the AI Scientist Pipeline. arXiv preprint arXiv:2606.04220, 2026. URL https://arxiv.org/abs/2606.04220.
387. Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, et al. Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models. arXiv preprint arXiv:2312.06585, 2023. URL https://arxiv.org/abs/2312.06585.
388. Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking. arXiv preprint arXiv:2403.09629, 2024. URL https://arxiv.org/abs/2403.09629.
389. Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive Introspection: Teaching Language Model Agents How to Self-Improve. arXiv preprint arXiv:2407.18219, 2024. URL https://arxiv.org/abs/2407.18219.
390. Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, et al. Training Language Models to Self-Correct via Reinforcement Learning. arXiv preprint arXiv:2409.12917, 2024. URL https://arxiv.org/abs/2409.12917.
391. Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, et al. TTRL: Test-Time Reinforcement Learning. arXiv preprint arXiv:2504.16084, 2025. URL https://arxiv.org/abs/2504.16084.
392. Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-Adapting Language Models. arXiv preprint arXiv:2506.10943, 2025. URL https://arxiv.org/abs/2506.10943.
393. Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, et al. AgentGym: Evolving Large Language Model-based Agents across Diverse Environments. arXiv preprint arXiv:2406.04151, 2024. URL https://arxiv.org/abs/2406.04151.
394. Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, et al. AgentEvolver: Towards Efficient Self-Evolving Agent System. arXiv preprint arXiv:2511.10395, 2025. URL https://arxiv.org/abs/2511.10395.
395. Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents. arXiv preprint arXiv:2502.12110, 2025. URL https://arxiv.org/abs/2502.12110.
396. Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. arXiv preprint arXiv:2504.07952, 2025. URL https://arxiv.org/abs/2504.07952.
397. Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv preprint arXiv:2510.04618, 2025. URL https://arxiv.org/abs/2510.04618.
398. Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, et al. MetaClaw: Just Talk – An Agent That Meta-Learns and Evolves in the Wild. arXiv preprint arXiv:2603.17187, 2026. URL https://arxiv.org/abs/2603.17187.
399. Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv preprint arXiv:2310.03714, 2023. URL https://arxiv.org/abs/2310.03714.
400. Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, et al. AFlow: Automating Agentic Workflow Generation. arXiv preprint arXiv:2410.10762, 2024. URL https://arxiv.org/abs/2410.10762.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 30 September 2026 doi:10.20944/preprints202609.2680.v1

401. Seth Karten, Alex L. Zhang, Kevin Thomas, Sebastian Müller, Elie Bakouch, Daniel Auras, et al. Prime Agent: A Self-Improving RLM Harness. arXiv preprint arXiv:2608.23552, 2026. URL https://arxiv.org/abs/2608.23552.
402. Tianlu Wang, Ilia Kulikov, Olga Golovneva, Ping Yu, Weizhe Yuan, Jane Dwivedi-Yu, et al. Self-Taught Evaluators. arXiv preprint arXiv:2408.02666, 2024. URL https://arxiv.org/abs/2408.02666.
403. Zhaoyang Wang, Weilei He, Zhiyuan Liang, Xuchao Zhang, Chetan Bansal, Ying Wei, Weitong Zhang, and Huaxiu Yao. CREAM: Consistency Regularized Self-Rewarding Language Models. arXiv preprint arXiv:2410.12735, 2024. URL https://arxiv.org/abs/2410.12735.
404. Shimao Zhang, Xiao Liu, Xin Zhang, Junxiao Liu, Zheheng Luo, Shujian Huang, and Yeyun Gong. Process-based Self-Rewarding Language Models. arXiv preprint arXiv:2503.03746, 2025. URL https://arxiv.org/abs/2503.03746.
405. Yaonan Qu and Meng Lu. Bilevel Autoresearch: Meta-Autoresearching Itself. arXiv preprint arXiv:2603.23420, 2026. URL https://arxiv.org/abs/2603.23420.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

© 2026 by the author(s). Distributed under a Creative Commons CC BY license.